算法设计与分析

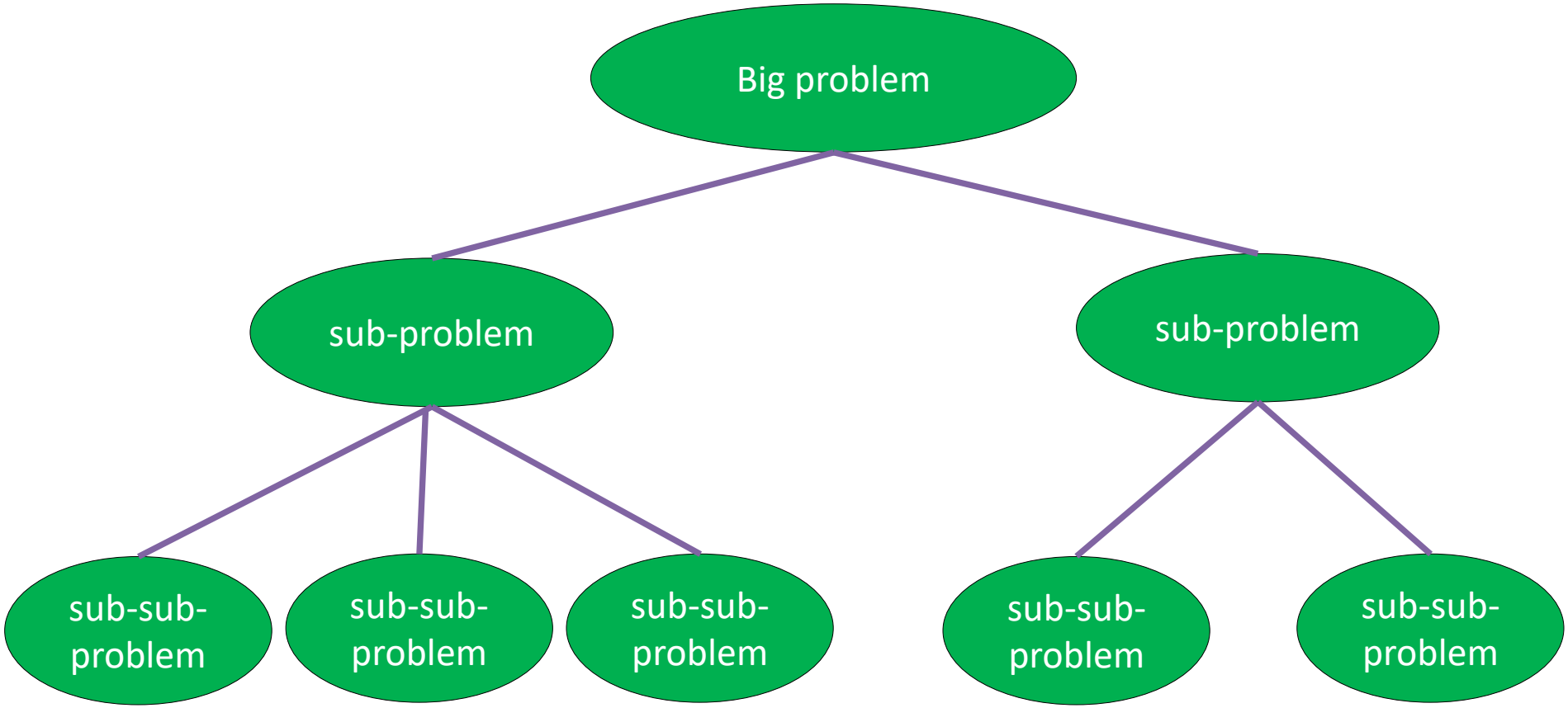
第五章 贪心算法

Greedy Algorithm

哈尔滨工业大学（深圳）
陈勇勇

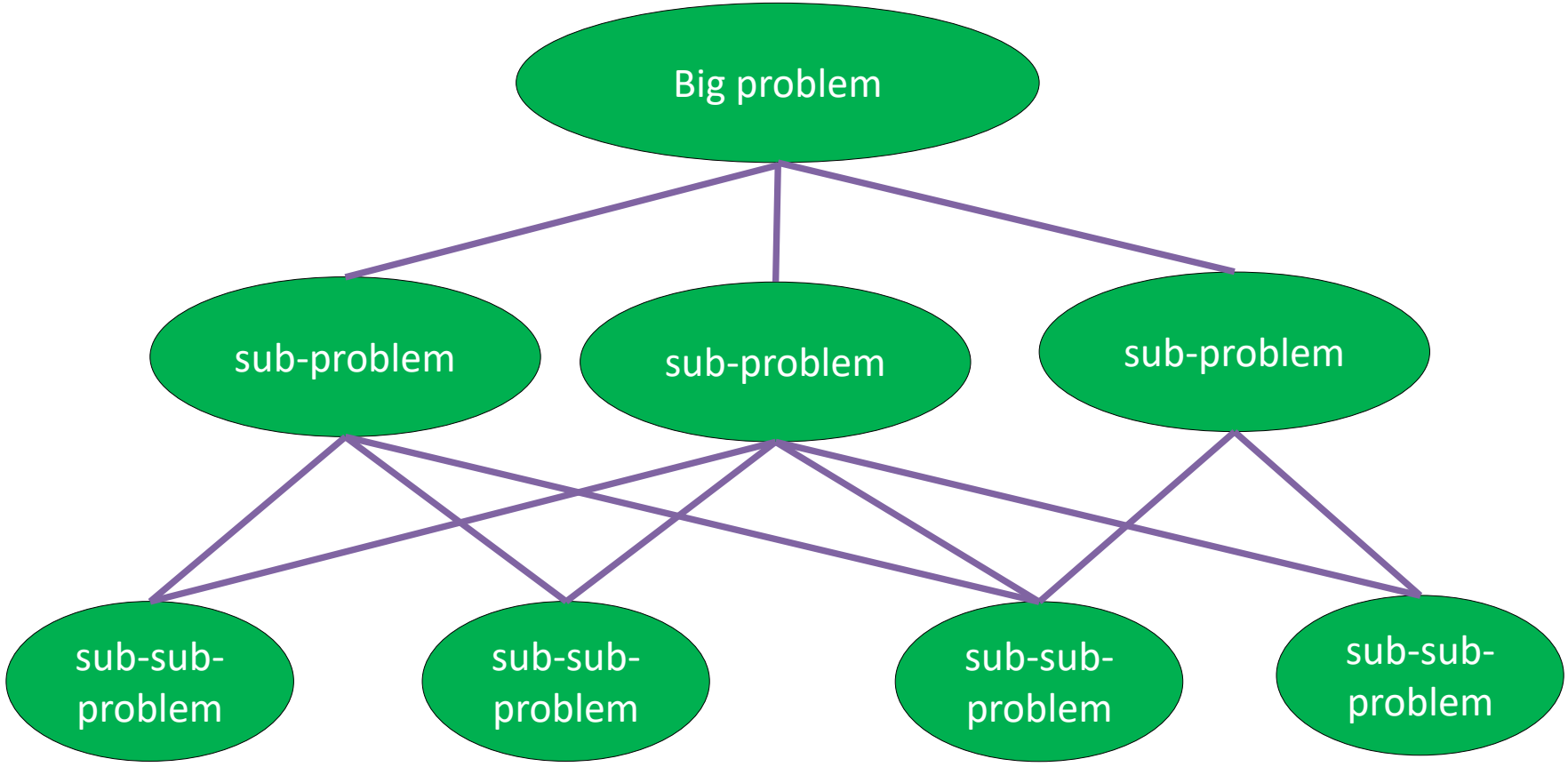
《算法导论》第16章

分治算法的子问题



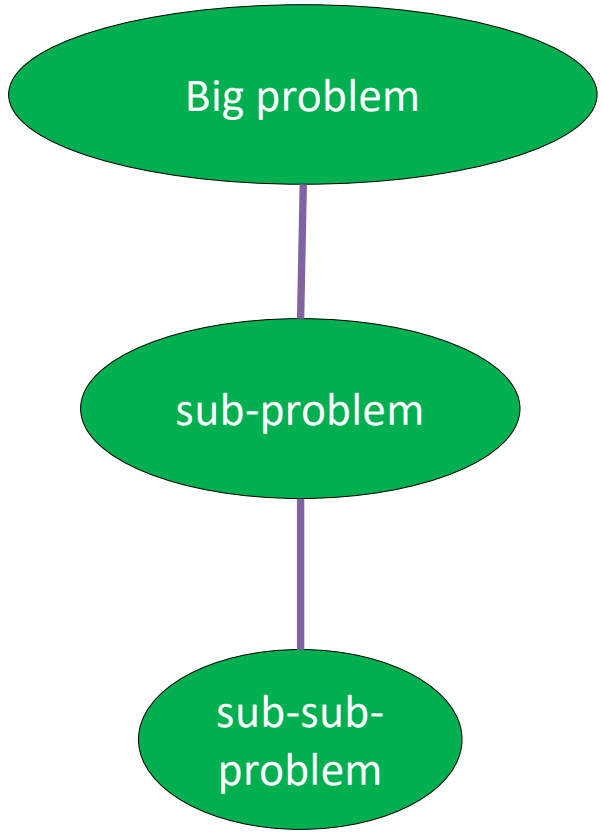
分治法（Divide and Conquer Algorithm）是把一个复杂问题分解为若干个相互独立的子问题，通过求解子问题并将子问题的解合并得到原问题的解。

动态规划的子问题



动态规划法（Dynamic Programming）是把一个复杂问题分解为若干个相互重叠的子问题，通过求解子问题形成的一系列决策得到原问题的解。

贪心算法的子问题



- 不仅需要满足优化子结构
- 同时每个大问题只依赖于一个子问题
- 贪心法（Greedy Algorithm）是把一个复杂问题分解为一系列较为简单的局部最优选择，每一步选择都是对当前解的一个扩展，直到获得问题的完整解。贪心法的典型应用是求解最优化问题，而且对许多问题都能得到整体最优解，即使不能得到整体最优解，通常也是最优解的很好近似。

本讲内容

5.1 贪心法的基本原理

5.2 任务安排问题

5.3 哈夫曼编码问题

5.4 最小生成树

贪心算法的基本概念

- 贪心算法的基本思想
  - 求解最优化问题的算法包含一系列步骤
  - 每一步都有一组选择
  - 每一步作出在当前看来最好的选择（从不回头看）
  - 希望通过作出局部优化选择达到全局优化选择
  - 贪心算法不一定总产生优化解
  - 贪心算法是否产生优化解，需严格证明
- 贪心算法产生优化解的条件
  - 贪心选择性
  - 优化子结构

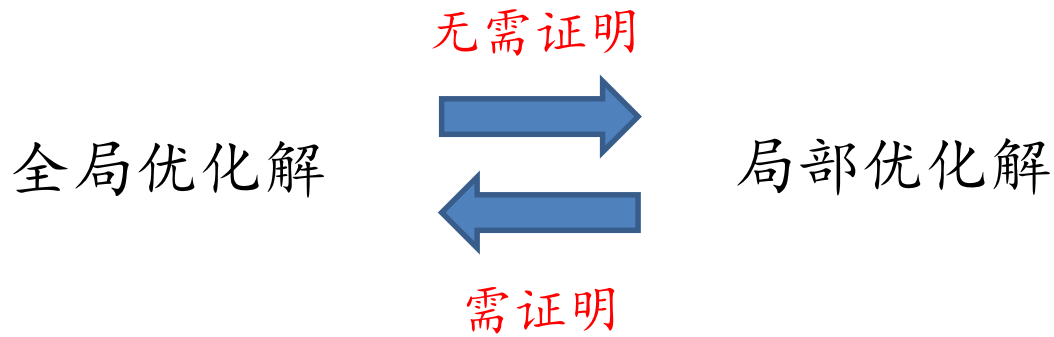


贪心选择性

- 贪心选择性

  若一个优化问题的全局优化解可以通过局部优化选择得到，则该问题称为具有贪心选择性。

- 一个问题是否具有贪心选择性需证明



优化子结构

若一个优化问题的优化解包含它的子问题的优化解，则称其具有优化子结构。

与动态规划方法的比较

- 动态规划方法可用的条件
  - 优化子结构
  - 子问题重叠性
  - 子问题空间小
- 贪心方法可用的条件
  - 优化子结构（比动态规划，有更高的优化子结构要求）
  - 贪心选择性
- 可用贪心方法时，动态规划方法可能不适用
- 可用动态规划方法时，贪心方法可能不适用（0-1背包问题）

与动态规划方法的比较

- 共同点：贪心算法和动态规划算法都要求问题具有优化子结构性质
- 动态规划方法
  - 在每一步所做的选择通常依赖于子问题的解
  - 以自底向上方式，先解小子问题，再求解大子问题
- 贪心方法
  - 在每一步先做出当前看起来最好的选择
  - 然后再求解本次选择后产生的剩余子问题
  - 每次选择既不依赖于子问题的解，也不依赖于未来的选择
  - 以自顶向下方式，逐步进行贪心选择，不断减少子问题规模

贪心算法正确性证明方法

- 证明算法所求解的问题具有优化子结构
- 证明算法所求解的问题具有贪心选择性
- 证明算法确实按照贪心选择性进行局部优化选择

本讲内容

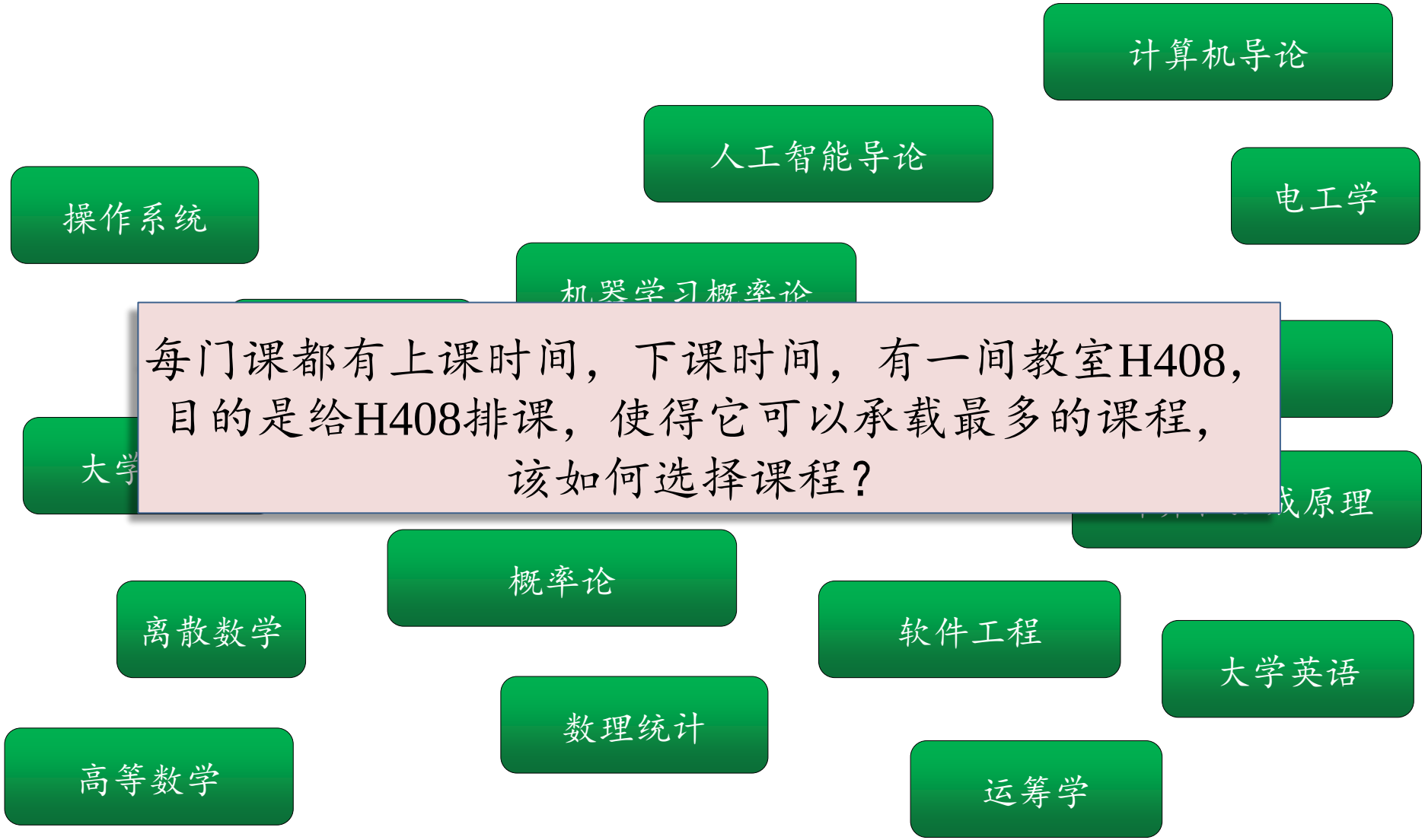
5.1 贪心法的基本原理

5.2 任务安排问题

5.3 哈夫曼编码问题

5.4 最小生成树

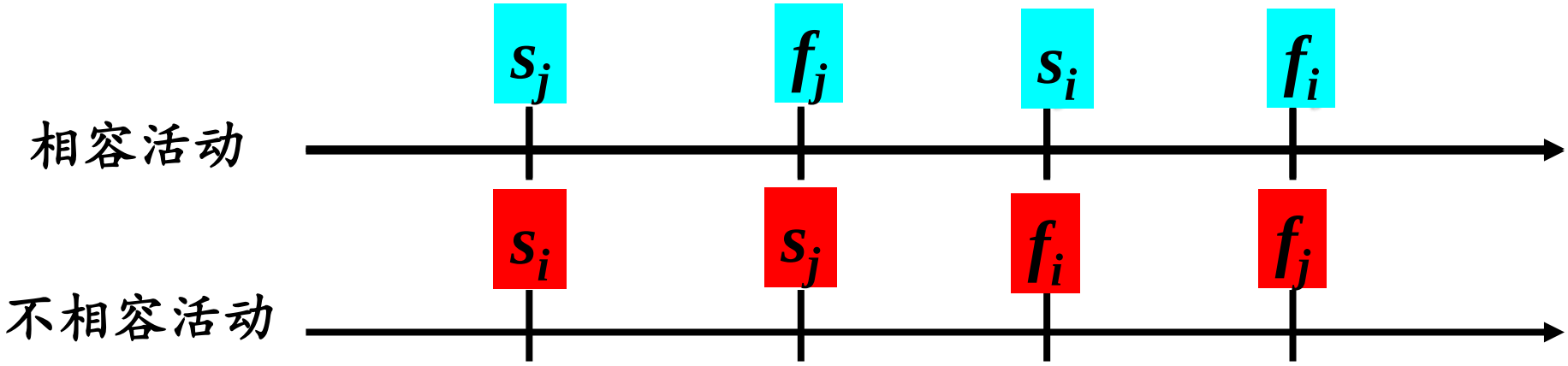
问题描述



每门课都有上课时间，下课时间，有一间教室H408，目的是给H408排课，使得它可以承载最多的课程，该如何选择课程？

问题的定义

- 活动
  - 设$S=\{1,2,\cdots,n\}$是n个活动的集合，各个活动使用同一个资源，资源在同一时间只能为一个活动使用。
  - 每个活动i有起始时间s_i，终止时间f_i，$s_i \le f_i$
- 相容活动
  - 活动 i 和 j 是相容的，若$s_i \ge f_j$ 或$s_j \ge f_i$，即



问题的定义

- 输入：$S=\{1,2,\cdots,n\}$, $\{s_i\}$, $\{f_i\}$, $1\le i\le n$
- 输出： S的最大相容集合

贪心思想：

① 最早开始时间：这样可以增大资源的利用率。

② 最早结束时间：这样可以使下一个活动尽早开始。

为了选择最多的相容活动，每次选f_i最小的活动，使我们能够选更多的活动。

优化解的结构分析

引理1：

设$S=\{1,2,\cdots,n\}$是n个活动集合，$[s_i,f_i]$是活动i的起始终止时间，且$f_1\le f_2\le\cdots\le f_n$，$S$的活动选择问题的某个优化解包括活动 1（活动1是结束时间最早的活动）。

证明：

- 设A是一个优化解，按结束时间排序A中活动，设其第一个活动为k，第二个活动为j。
- 如果$k=1$，引理成立。
- 如果$k\neq 1$，令$B=(A-\{k\})\cup\{1\}$，由于A中活动相容，$f_1\le f_k\le s_j$，B中活动相容。因为$|B|=|A|$，所以B是一个优化解，且包括活动1。

引理2：

设$S=\{1,2,\cdots,n\}$是n个活动集合，$[s_i,f_i]$是活动i的起始终止时间，且$f_1\le f_2\le\cdots\le f_n$，设$A$是$S$的调度问题的一个优化解且包括**活动1**，则$A'=A-\{1\}$是$S'=\{i\in S|s_i>f_1\}$的调度问题的优化解。

引理2说明活动选择问题具有优化子结构

证明：

- 显然，$A'=A-\{1\}$中的活动是相容的。
- 我们仅需要证明A'是S'最大的。设不然，存在一个S'的活动选择问题的优化解B'，$|B'|>|A'|$。
- 令$B=\{1\}\cup B'$，对于$\forall i\in S'$，$s_i>f_1$，B中活动相容。B是S的一个解。由于$|A|=|A'|+1$，$|B|=|B'|+1>|A'|+1=|A|$，与A最大矛盾。

贪心选择性

引理3:

设 $S=\{1,2,\cdots,n\}$是n个活动集合，其中 $f_1 \le f_2 \le \cdots \le f_n$，$f_0=0$，$l_i$是$S_i=\{j \in S \mid s_j \ge f_{l_{i-1}}\}$ 中具有最小结束时间 f_{l_i}的活动，设A是S的包含活动1的优化解，则 $A=\cup_{i=1}^{k}\{l_i\}$。

证明:

对$|A|$作归纳法。

引理1: 设$S=\{1,2,\cdots,n\}$是n个活动集合，$[s_i,f_i]$是活动的起始终止时间，且 $f_1 \le f_2 \le \cdots \le f_n$， S的活动选择问题的某个优化解包括活动1。

- 当$|A|=1$时，由引理1，命题成立。
- 设$|A|<k$时，命题成立。当$|A|=k$时，由引理2，$A=\{1\}\cup A_1$，A_1是$S_2=\{j \in S \mid s_j \ge f_1\}$的优化解。
- 由归纳假设，$A_1=\cup_{i=2}^{k}\{l_i\}$。于是，$A=\cup_{i=1}^{k}\{l_i\}$。

引理2: 设$S=\{1,2,\cdots,n\}$是n个活动集合，$[s_i,f_i]$是活动i的起始终止时间，且$f_1 \le f_2 \le \ldots. \le f_n$，设$A$是$S$的调度问题的一个优化解且包括活动1，则$A'=A-\{1\}$是$S'=\{i \in S \mid s_i > f_1\}$的调度问题的优化解。

算法的设计

- 贪心思想

  为了选择最多的相容活动，每次选f_i最小的活动，使我们能够选更多的活动。

算法

Greedy-Activity-Selector

Input: 活动的开始，结束时间数组S，F，设$f_1 \le f_2 \le \cdots \le f_n$（已排序）

Output: 选择的活动集合

执行: Greedy-Activity-Selector(S, F)

过程: Greedy-Activity-Selector(S, F)

```
1. n ← length(S)
2. A ← {1}
3. j ← 1
4. For  i←2  To  n  Do
5.    IF  s_i ≥ f_j (当前第i个活动开始的时间大于等于第j个活动结束的时间)
6.    Then  A←A∪{i}; j←i;
7. Return  A
```

复杂性分析

- 如果结束时间$\{f_i\}_{i=1}^n$已排序

$$T(n) = \Theta(n)$$

- 如果结束时间$\{f_i\}_{i=1}^n$未排序 （先对序列进行排序）

$$T(n) = \Theta(n) + \Theta(n\lg n) = \Theta(n\lg n)$$

算法正确性证明

- 需要证明
  - 活动选择问题具有优化子结构
  - 活动选择问题具有贪心选择性
  - 算法按照贪心选择性计算优化解

算法正确性证明

定理： Greedy-Activity-Selector算法能够产生最优解。

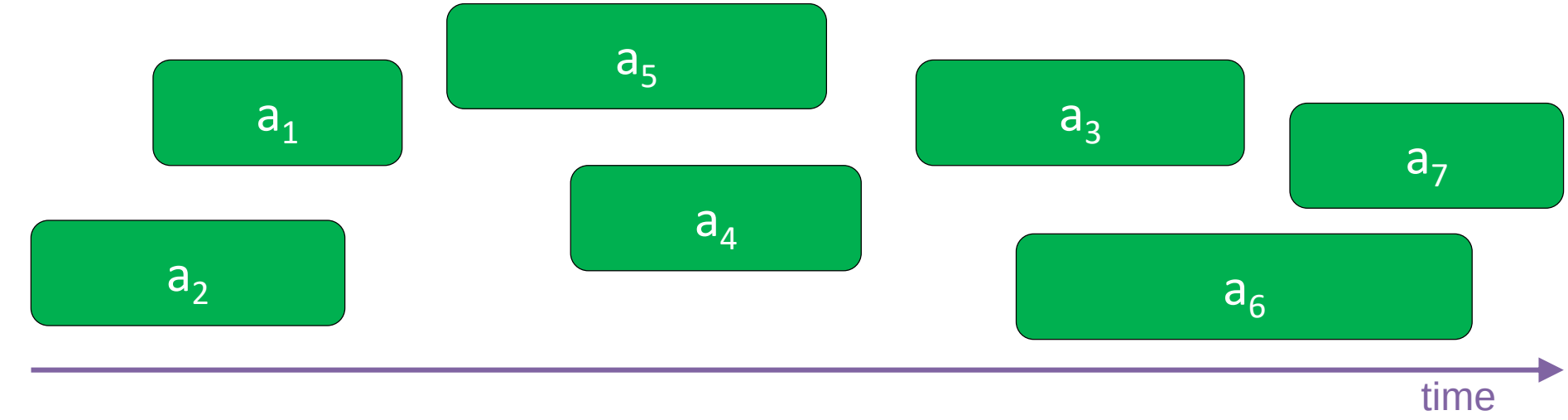
证明：

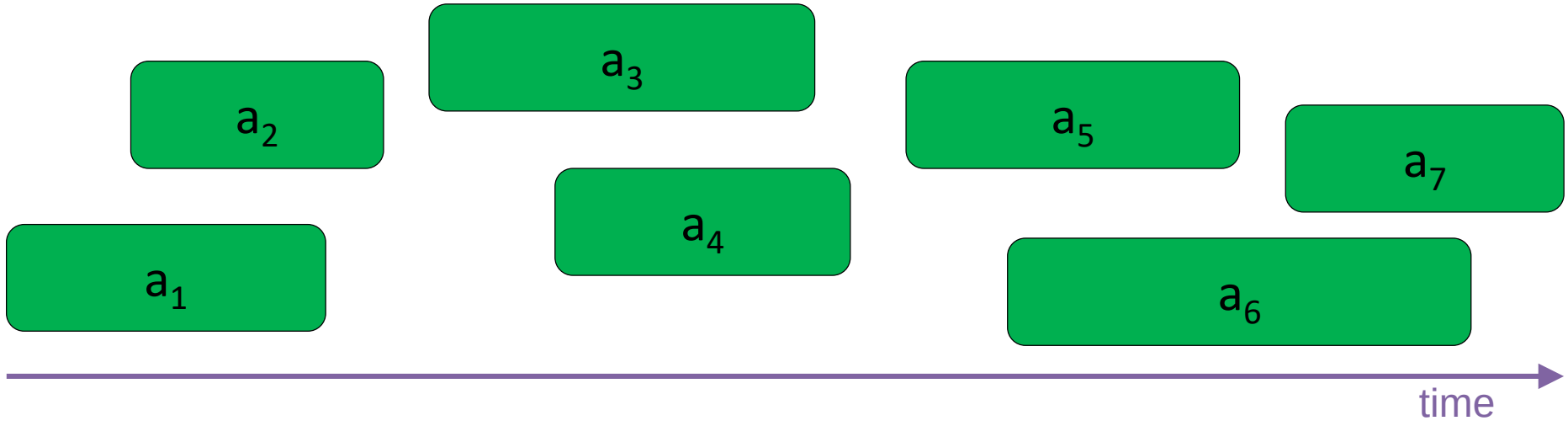
（1）由引理2可知活动选择问题具有优化子结构；

（2）由引理3知活动选择问题具有贪心选择性；

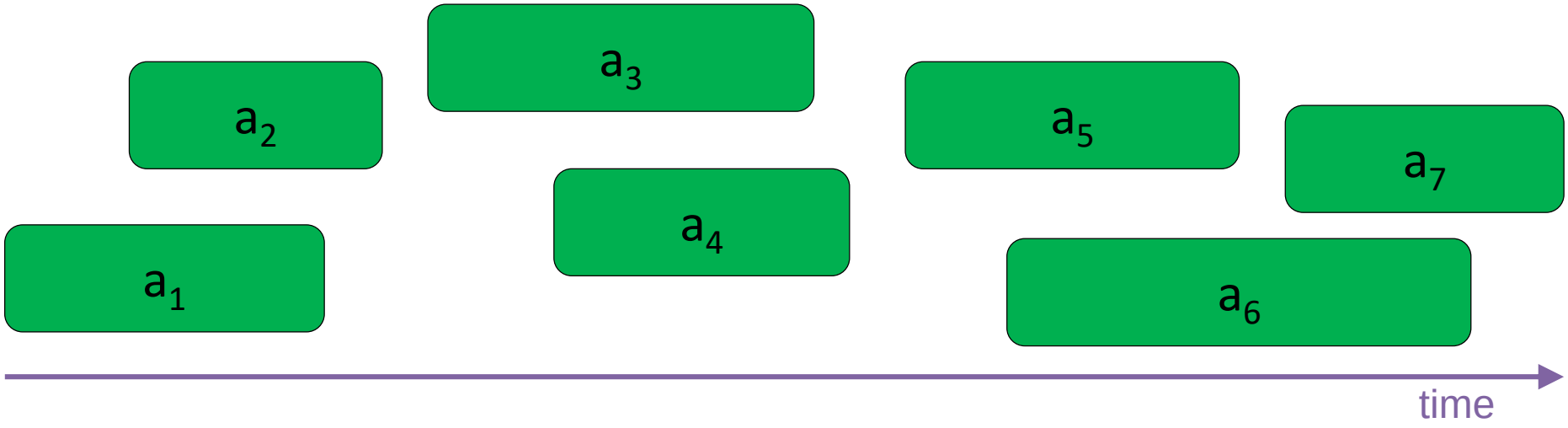
（3）Greedy-Activity-Selector算法按照引理3的贪心选择性进行局部优化选择。

示例





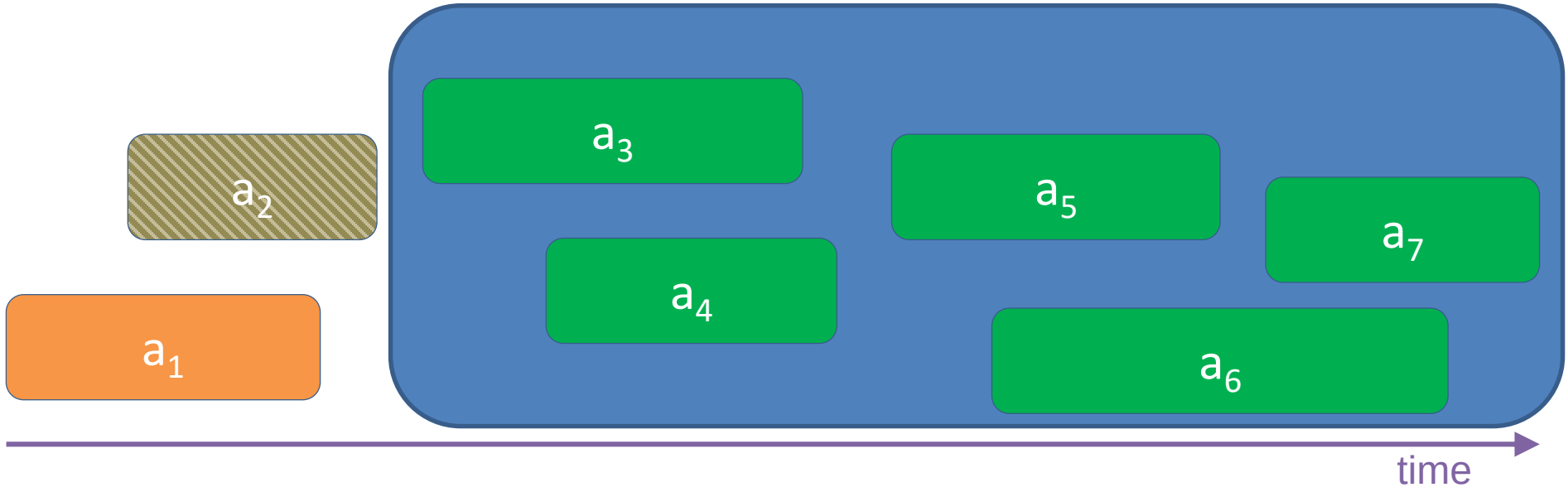
示例



引理3:

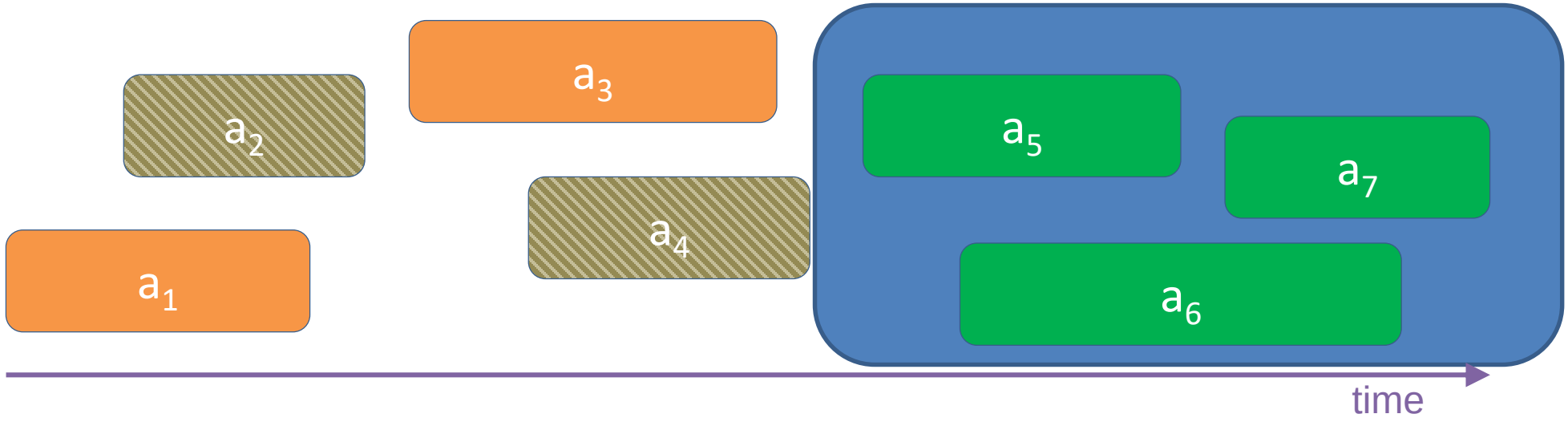
设 $S=\{1,2,\cdots,n\}$是n个活动集合，其中 $f_1 \le f_2 \le \cdots \le f_n$，$f_0=0$，$l_i$是$S_i=\{j \in S \mid s_j \ge f_{l_{i-1}}\}$ 中具有最小结束时间 f_{l_i}的活动，设A是S的包含活动1的优化解，则 $A=\cup_{i=1}^{k}\{l_i\}$。

示例



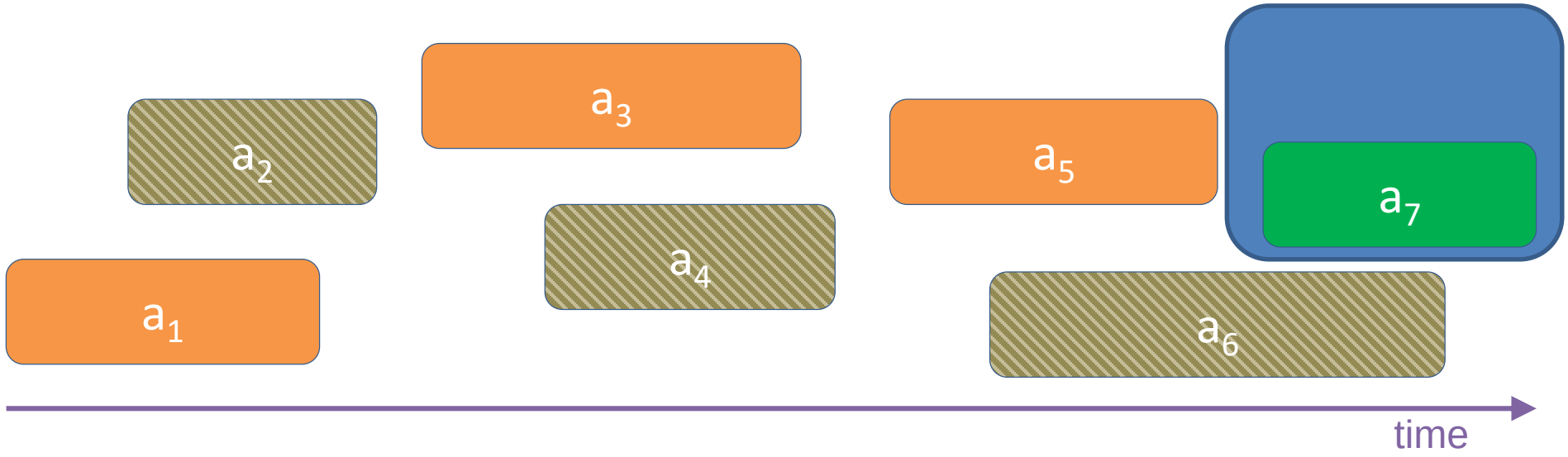
$$S_1 = \{j \in S \mid s_j \geq f_{l_{1-1}} = f_0\} = \{1, 2, \cdots, 7\} = S \quad l_1 = 1$$

示例



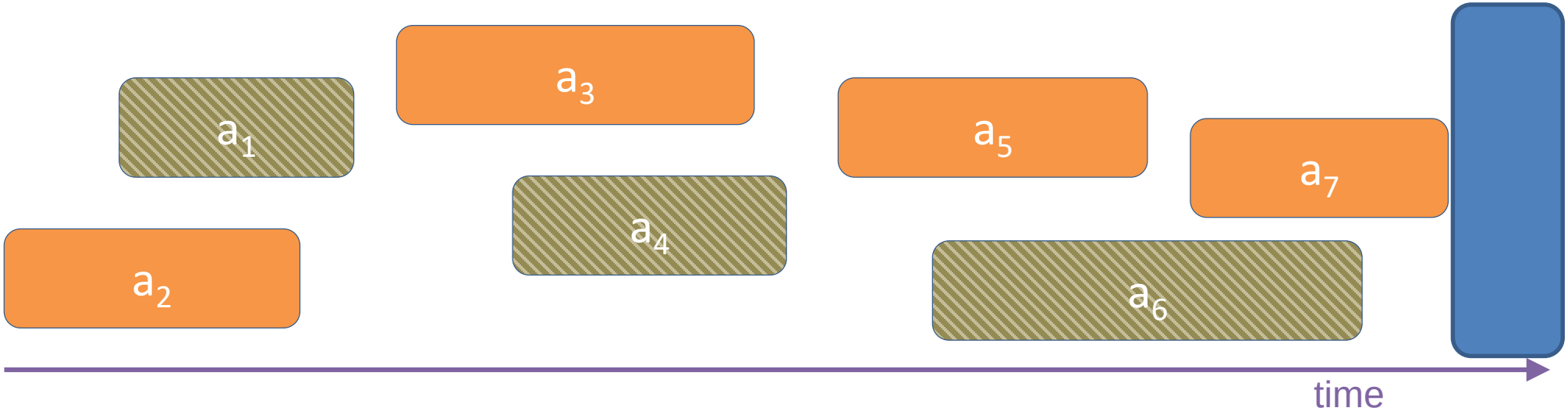
$$S_2 = \{j \in S \mid s_j \geq f_{l_{2-1}} = f_{l_1} = f_1\} = \{3, 4, 5, 6, 7\} \qquad l_2 = 3$$

示例



$$S_3 = \{j \in S \mid s_j \geq f_{l_{3-1}} = f_{l_2} = f_3\} = \{5, 6, 7\} \qquad l_3 = 5$$

示例



$$S_4 = \{j \in S \mid s_j \geq f_{l_{4-1}} = f_{l_3} = f_5\} = \{7\} \qquad l_4 = 7$$



Capacity: 10

Item:	🐢	💡	🍉	🌮	🚒
Weight:	6	2	4	3	11
Value:	20	8	14	13	35

- 对完全背包问题使用动态规划策略:
  - 所有物品，都有无限份
  - 使用动态规划，装背包的方案如下：

🌮 🌮 💡 💡

Total weight: 10
Total value: 42

因此，贪心算法不是总能产生优化解！

- 对完全背包问题使用贪心策略:
  - 由于Tacos具有最高的value/weight
  - 贪心策略是持续装入Tacos，直到不能装，方案如下：

🌮 🌮 🌮

Total weight: 9
Total value: 39

本讲内容

5.1 贪心法的基本原理

5.2 任务安排问题

5.3 哈夫曼编码问题

5.4 最小生成树

计算机编码

- 二进制字符编码
  - 每个字符用一个二进制0、1串来表示。
- 固定长编码
  - 每个字符都用相同长的0、1串表示。
- 可变长编码
  - 经常出现的字符用短码，不经常出现的用长码。
- 前缀编码
  - 无任何字符的编码是另一个字符编码的前缀。

计算机编码

- everyday english sentence
- 01100101 01110110 01100101 01110010 01111001 01100100 01100001 01111001 00100000 01100101 01101110 01100111 01101100 01101001 01110011 01101000 00100000 01110011 01100101 01101110 01110100 01100101 01101110 01100011 01100101

- qwertyui_opasdfg+hjklzxcv
- 01110001 01110111 01100101 01110010 01110100 01111001 01110101 01101001 01011111 01101111 01110000 01100001 01110011 01100100 01100110 01100111 00101011 01101000 01101010 01101011 01101100 01111010 01111000 01100011 01110110

计算机编码

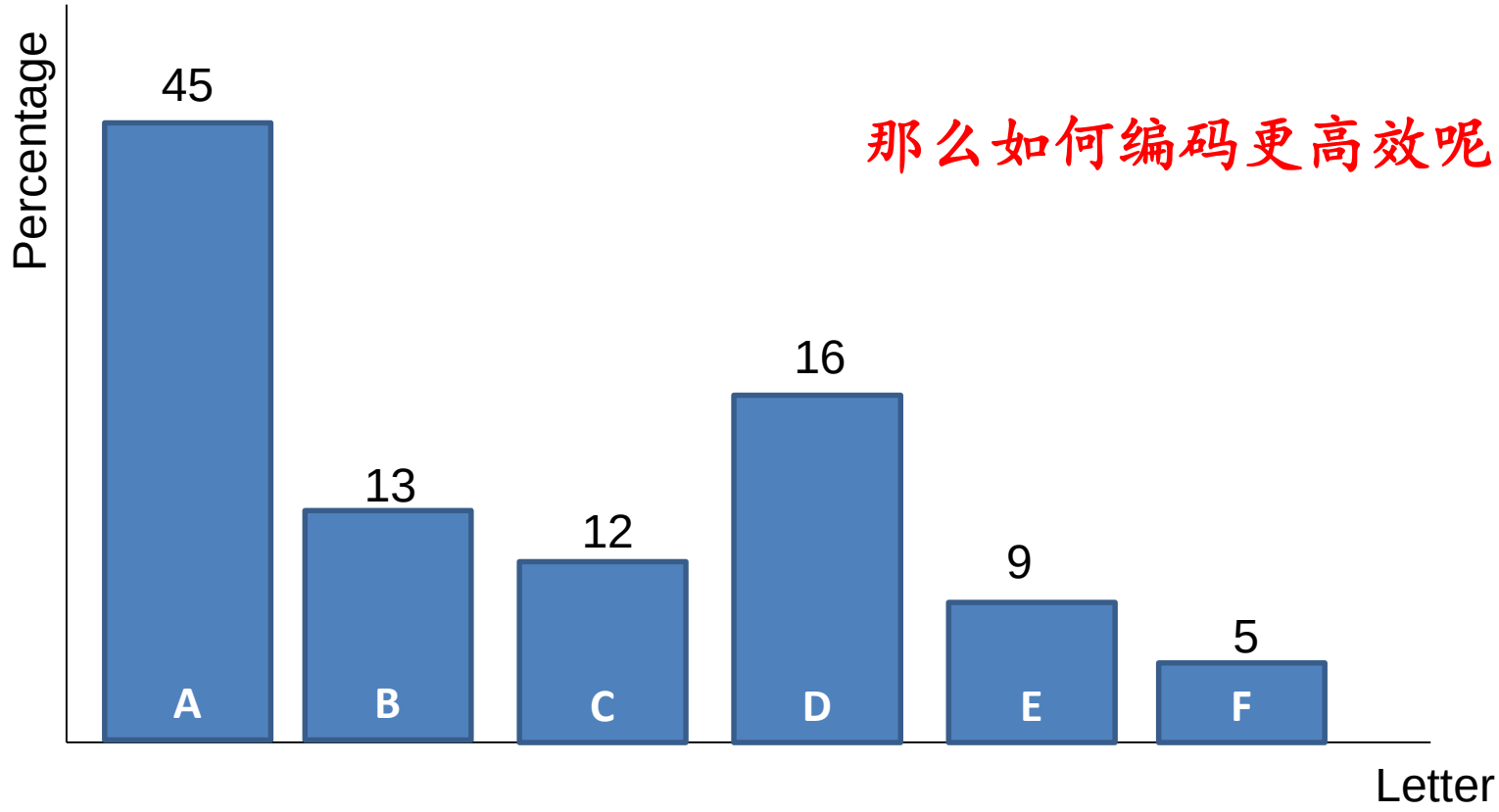
如果 e 出现多次的话，
ASCII码将会非常低效！

- everyday english sentence
- 01100101 01110110 01100101 01110010 01111001 01100100 01100001 01111001 00100000 01100101 01101110 01100111 01101100 01101001 01110011 01101000 00100000 01110011 01100101 01101110 01110100 01100101 01101110 01100011 01100101

- qwertyui_opasdfg+hjklzxcv
- 01110001 01110111 01100101 01110010 01110100 01111001 01110101 01101001 01011111 01101111 01110000 01100001 01110011 01100100 01100110 01100111 00101011 01101000 01101010 01101011 01101100 01111010 01111000 01100011 01110110

计算机编码

- 字符在使用中有一个使用频率的分布，例如：



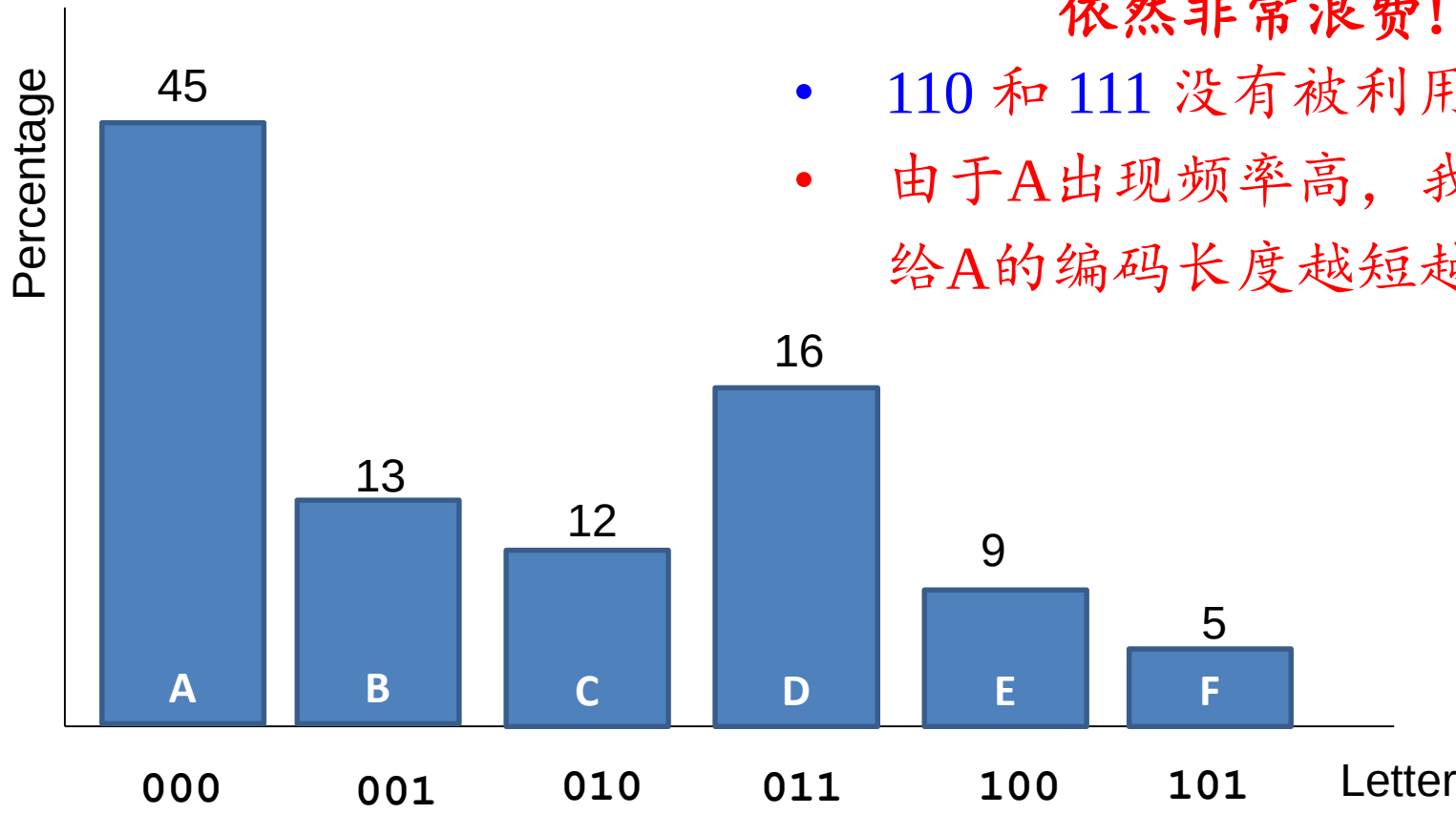
那么如何编码更高效呢？

计算机编码

（Try 0：固定长编码）

固定长编码：

- 模仿ASCII码，但我们用位数更少的编码方式，每个字母分配一个固定3位二进制。



依然非常浪费！

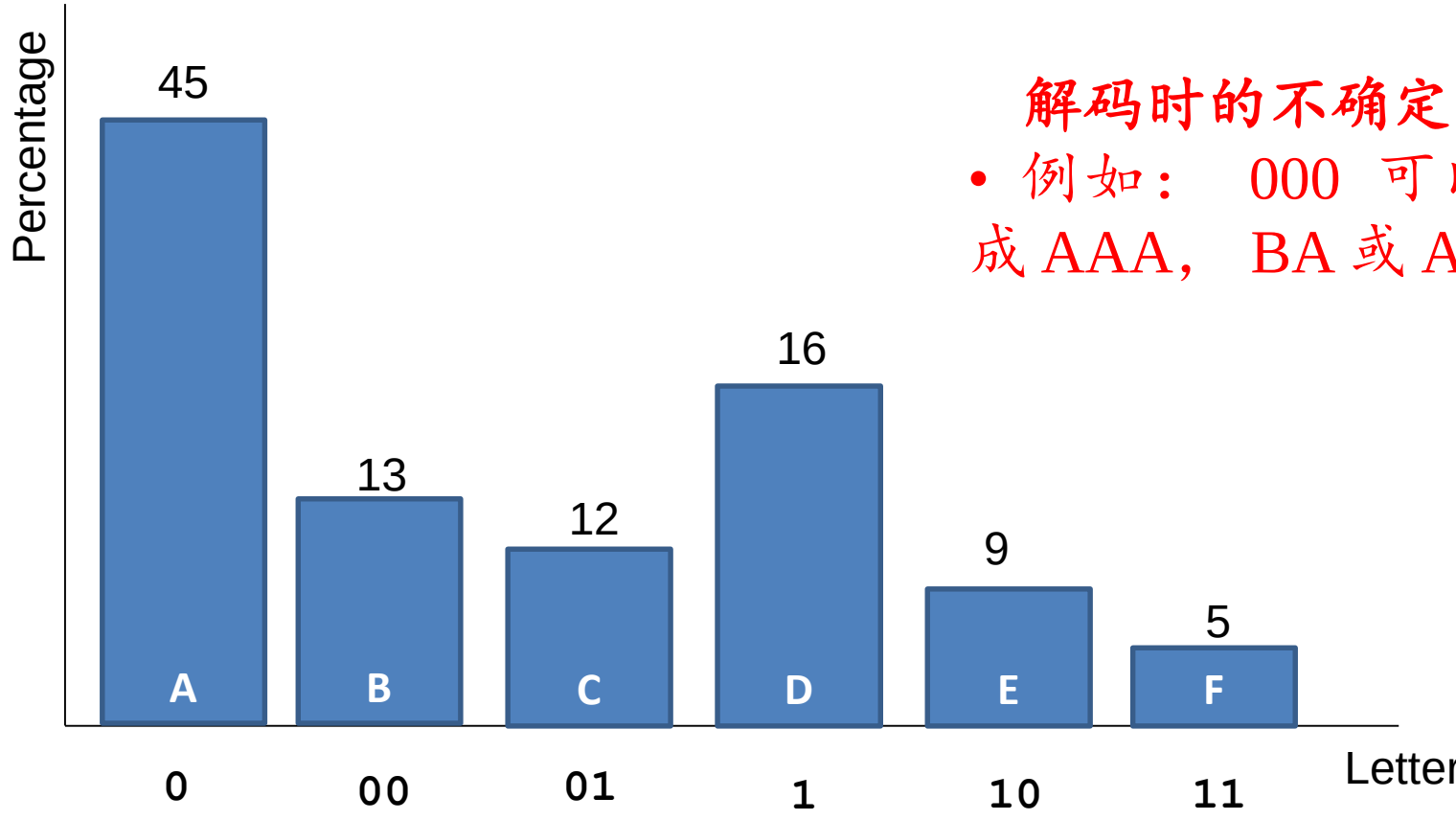
- 110 和 111 没有被利用。
- 由于A出现频率高，我们应该给A的编码长度越短越好。

计算机编码

（Try 1：可变长编码）

可变长编码：

- 每个字符分配一个二进制0/1串
- 出现频率越高的分配的字符串长度越短



解码时的不确定性！

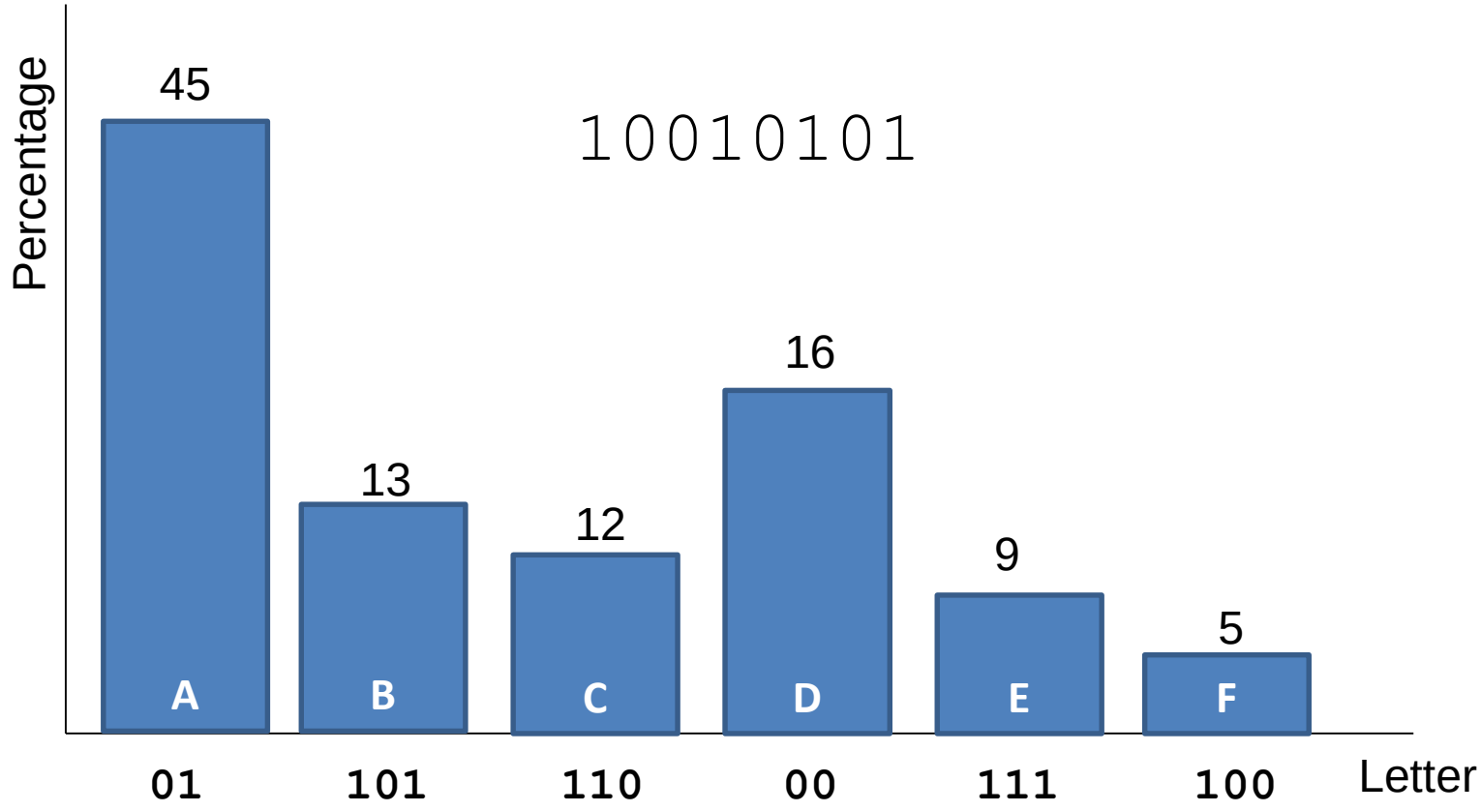
- 例如： 000 可以解码成 AAA， BA 或 AB？

计算机编码

（Try 2：前缀编码）

前缀编码：

- 每个字母用二进制串来表示。
- 出现频率越长的字母，他的编码长度越短。
- 每个字母的编码不允许是另一个字母编码的前缀。

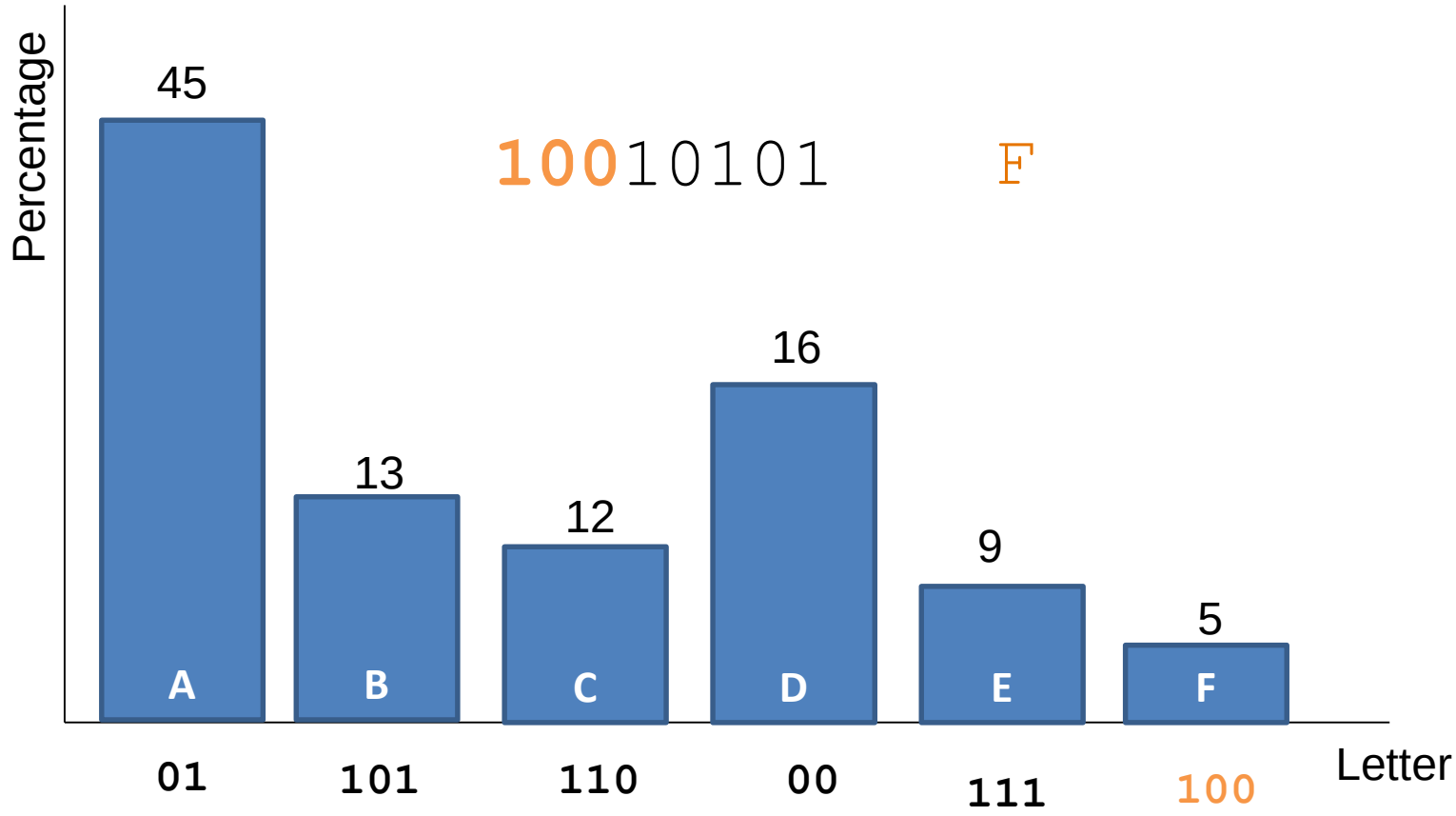


计算机编码

（Try 2：前缀编码）

前缀编码：

- 每个字母用二进制串来表示。
- 出现频率越长的字母，他的编码长度越短。
- 每个字母的编码不允许是另一个字母编码的前缀。



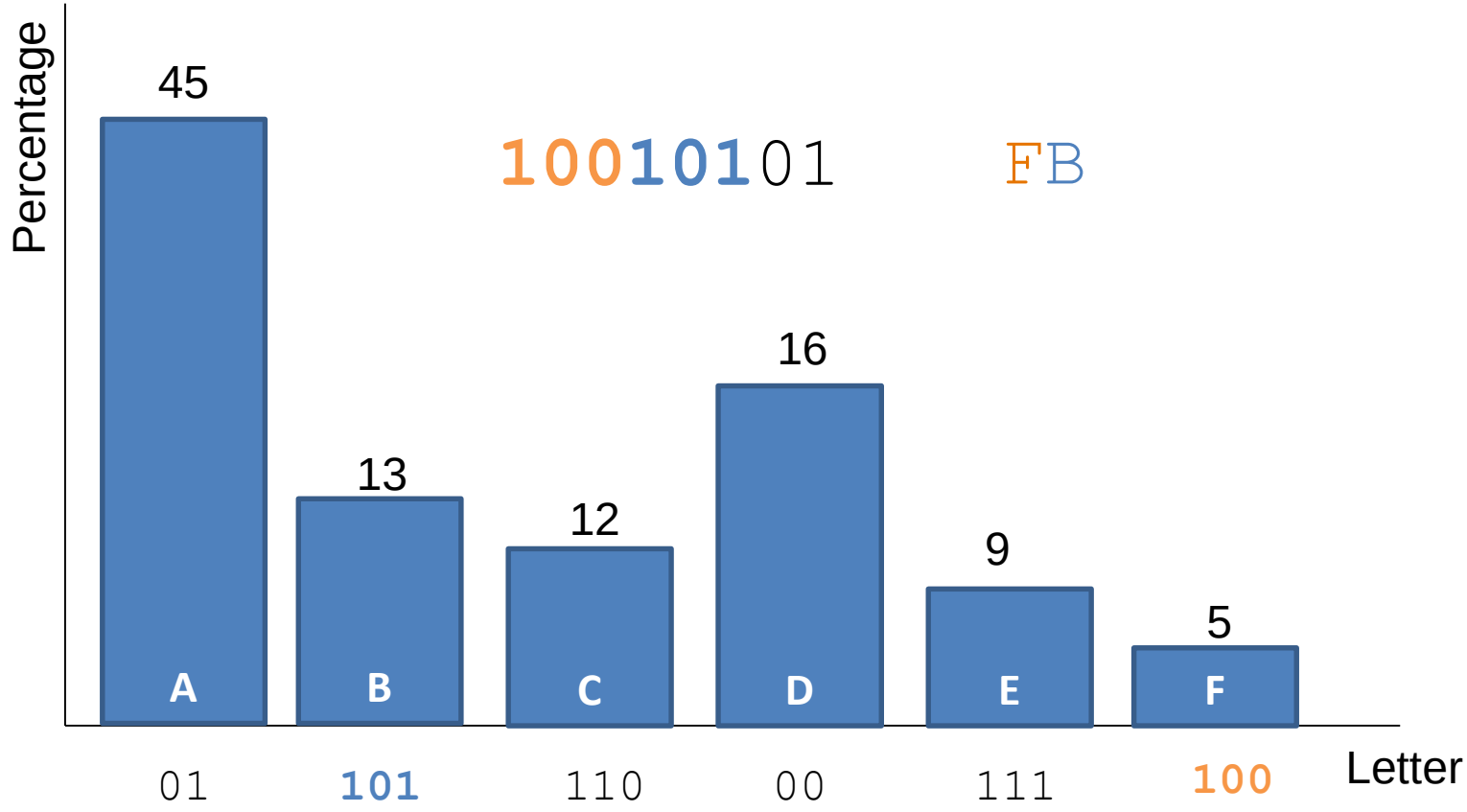
100**10101** F

计算机编码

（Try 2：前缀编码）

前缀编码：

- 每个字母用二进制串来表示。
- 出现频率越长的字母，他的编码长度越短。
- 每个字母的编码不允许是另一个字母编码的前缀。

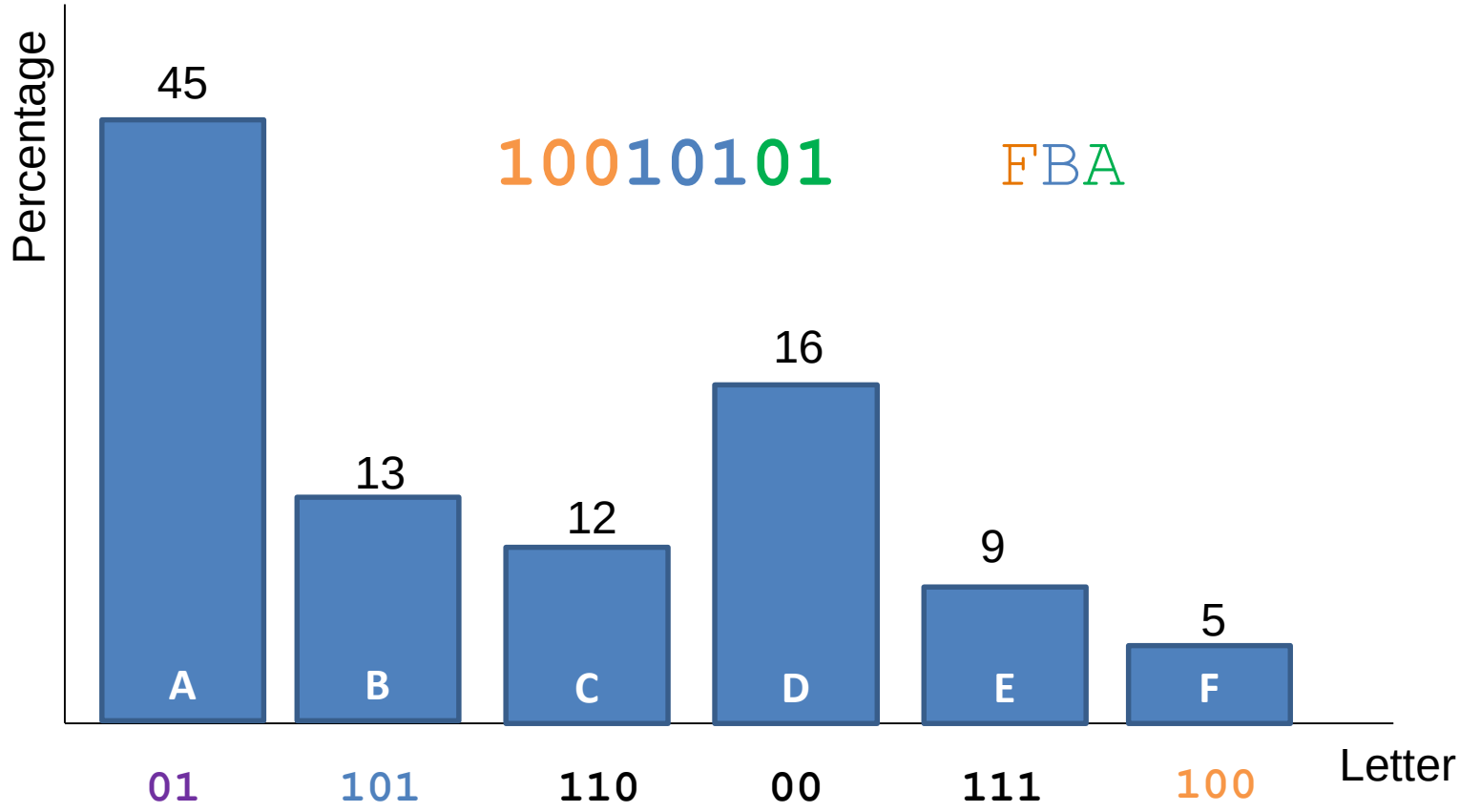


计算机编码

（Try 2：前缀编码）

前缀编码：

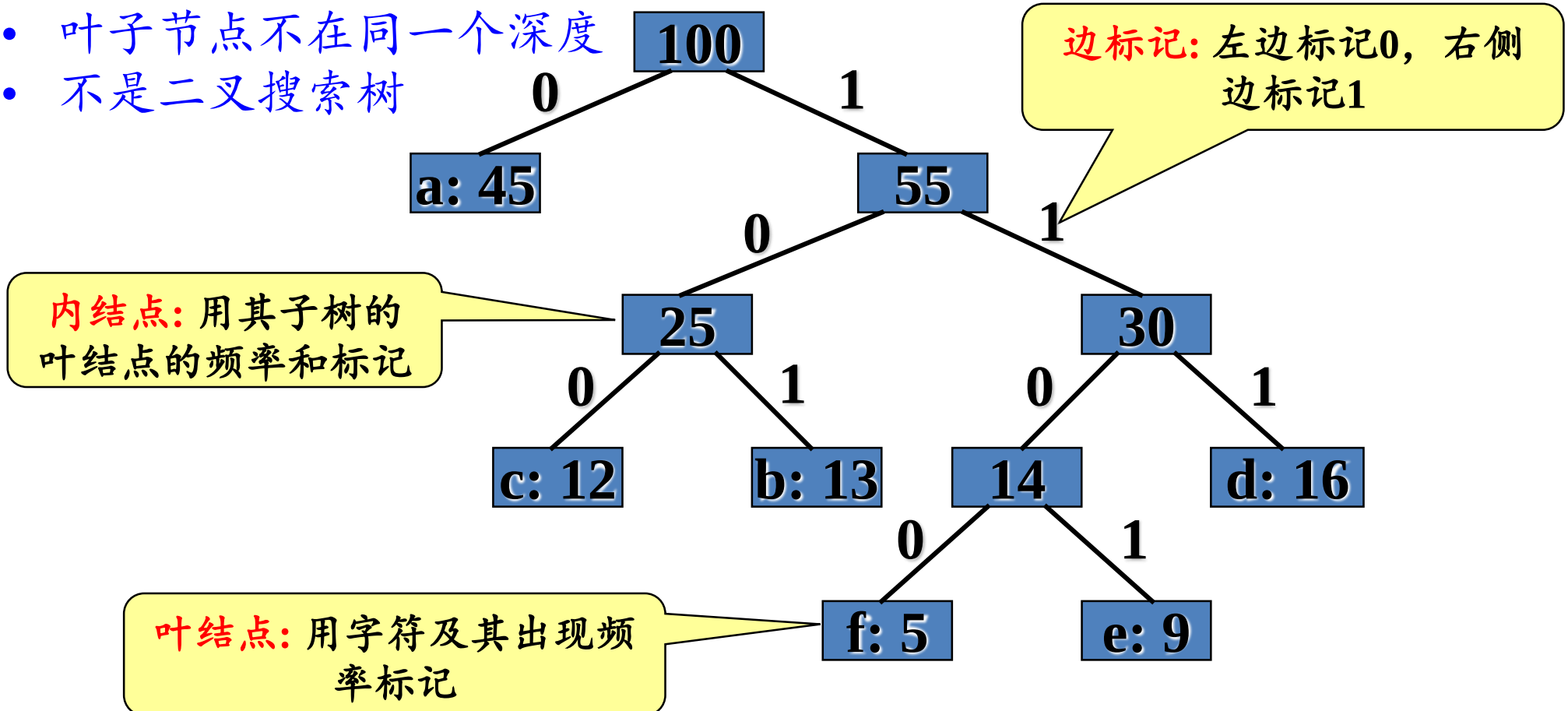
- 每个字母用二进制串来表示。
- 出现频率越长的字母，他的编码长度越短。
- 每个字母的编码不允许是另一个字母编码的前缀。



计算机编码

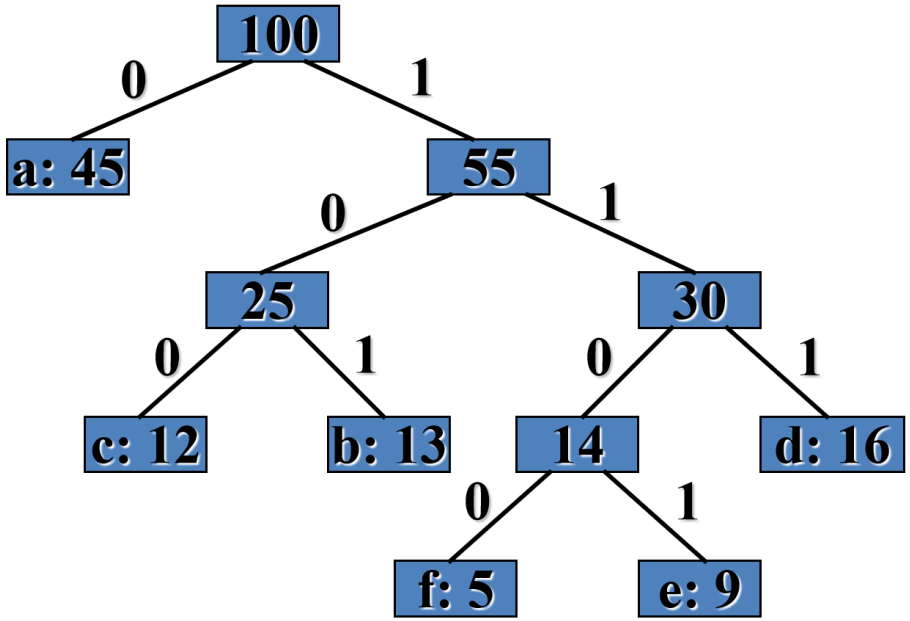
（Try 2：前缀编码）

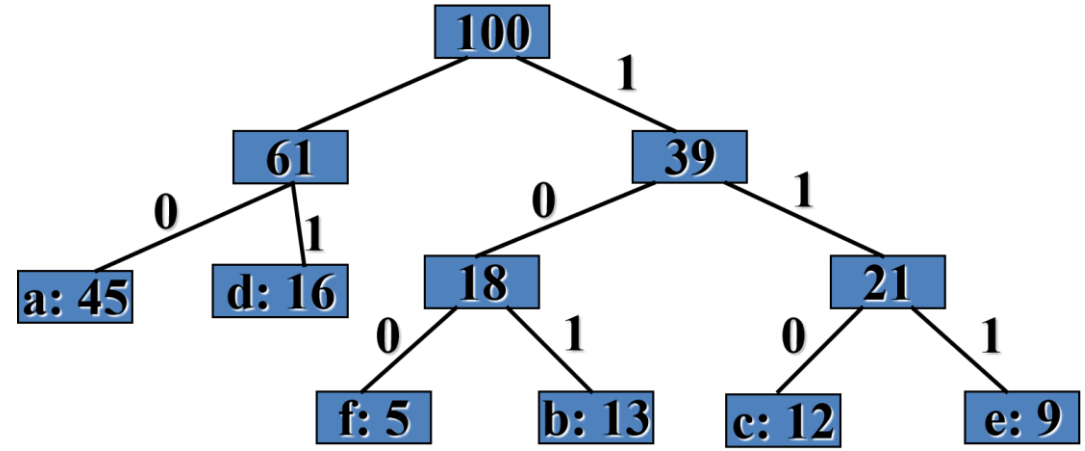
- 前缀编码可以表示成一个二叉树
  - 每个字符是树的叶子
  - 从根结点到叶子结点的路径为对应字符的编码
  - 前缀编码树不唯一
  - 叶子节点不在同一个深度
  - 不是二叉搜索树



计算机编码

（Try 2：前缀编码）





这两个前缀编码树，有没有优劣之分？

编码树T的代价

- 设C是字母表，$\forall c \in C$
- $f(c)$是c在文件中出现的频率
- $d_T(c)$是叶子c在树T中的深度，即c的编码长度
- T的代价是编码所有字符的期望编码长度：

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

问题的定义

输入：字母表 $C = \{c_1, c_2, \cdots, c_n\}$，

对应的频率表 $F = \{f(c_1), f(c_2), \cdots, f(c_n)\}$

输出：具有最小$B(T)$的字母表C的前缀编码树，即C的哈夫曼编码树

哈夫曼算法的设计

贪心思想:循环地选择具有最低频率的两个结点，生成一棵子树，直至所有结点形成一棵树。

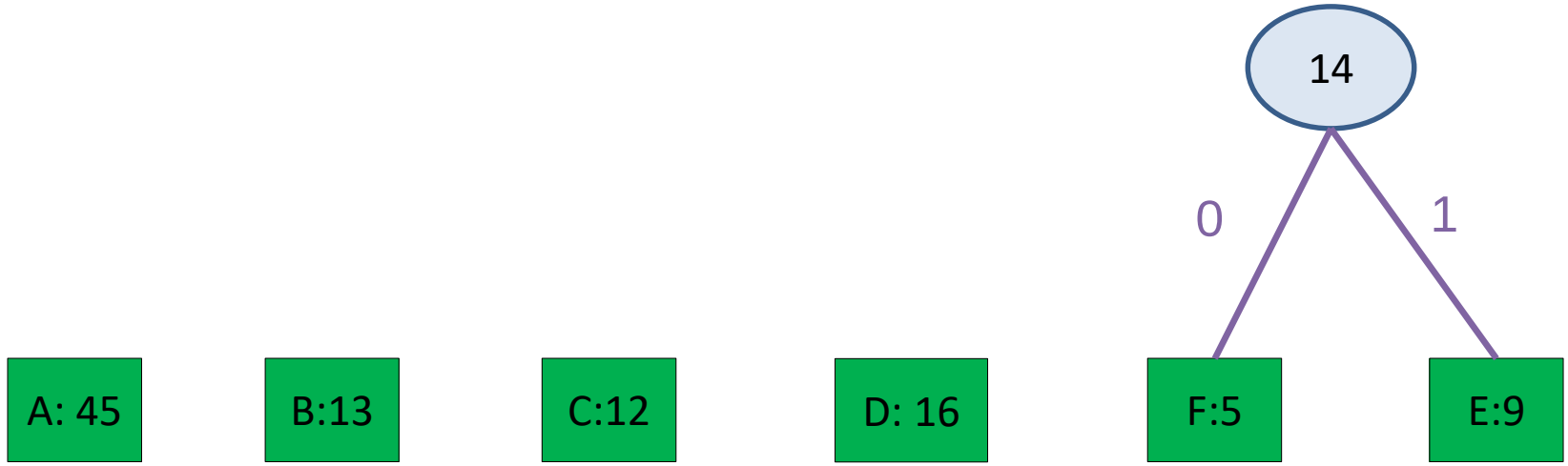
构造Huffman树的步骤：

1. 根据给定的n个权值 ，构造n棵只有一个根结点的二叉树，n个权值分别是这些二叉树根结点的权。
2. 设F是由这n棵二叉树构成的集合，在F中选取两棵根结点树值最小的树作为左、右子树，构造一颗新的二叉树，置新二叉树根的权值=左、右子树根结点权值之和；
3. 从F中删除这两颗树，并将新树加入F；
4. 重复 2、3，直到F中只含一颗树为止； 这棵树便是Huffman树

示例

例如：**f:5，e:9，c:12，b:13，d:16，a:45**

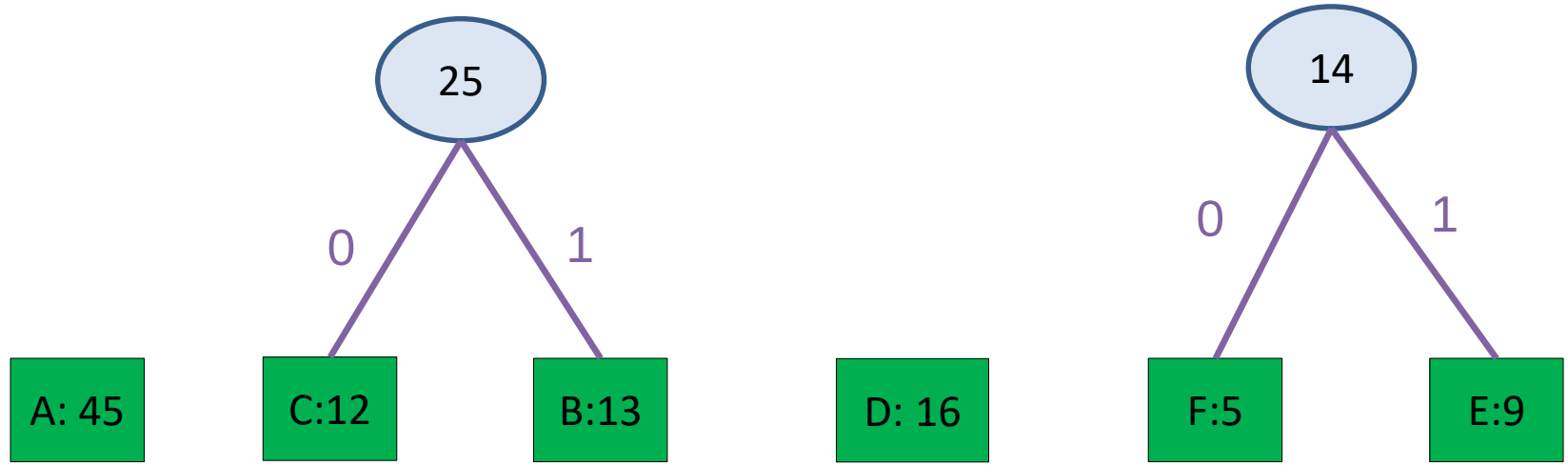
循环地选择具有最低频率的两个结点，生成一棵子树，直至所有结点形成一棵树



示例

例如：f:5，e:9，c:12，b:13，d:16，a:45

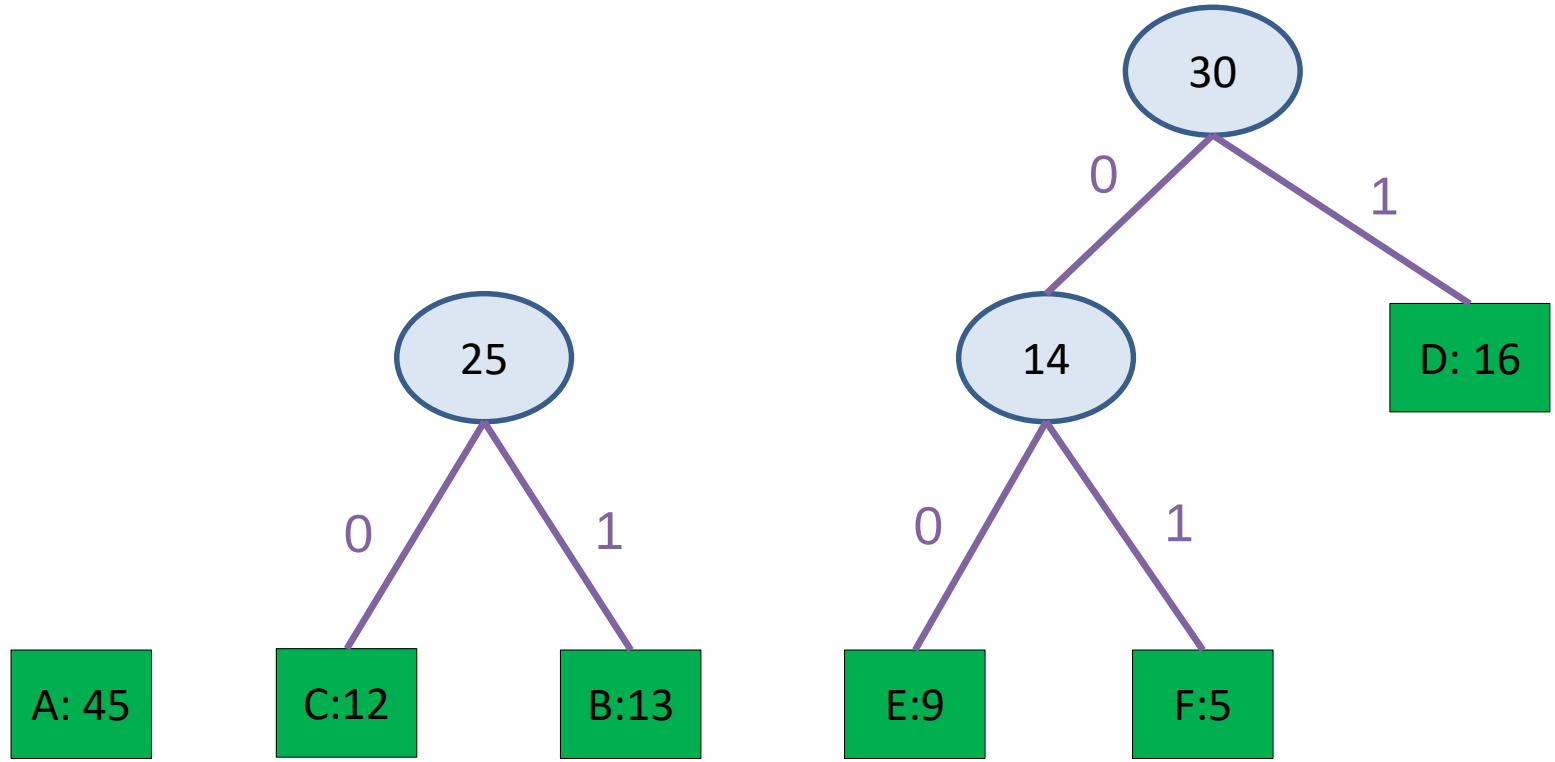
循环地选择具有最低频率的两个结点，生成一棵子树，直至所有结点形成一棵树



示例

例如：f:5，e:9，c:12，b:13，d:16，a:45

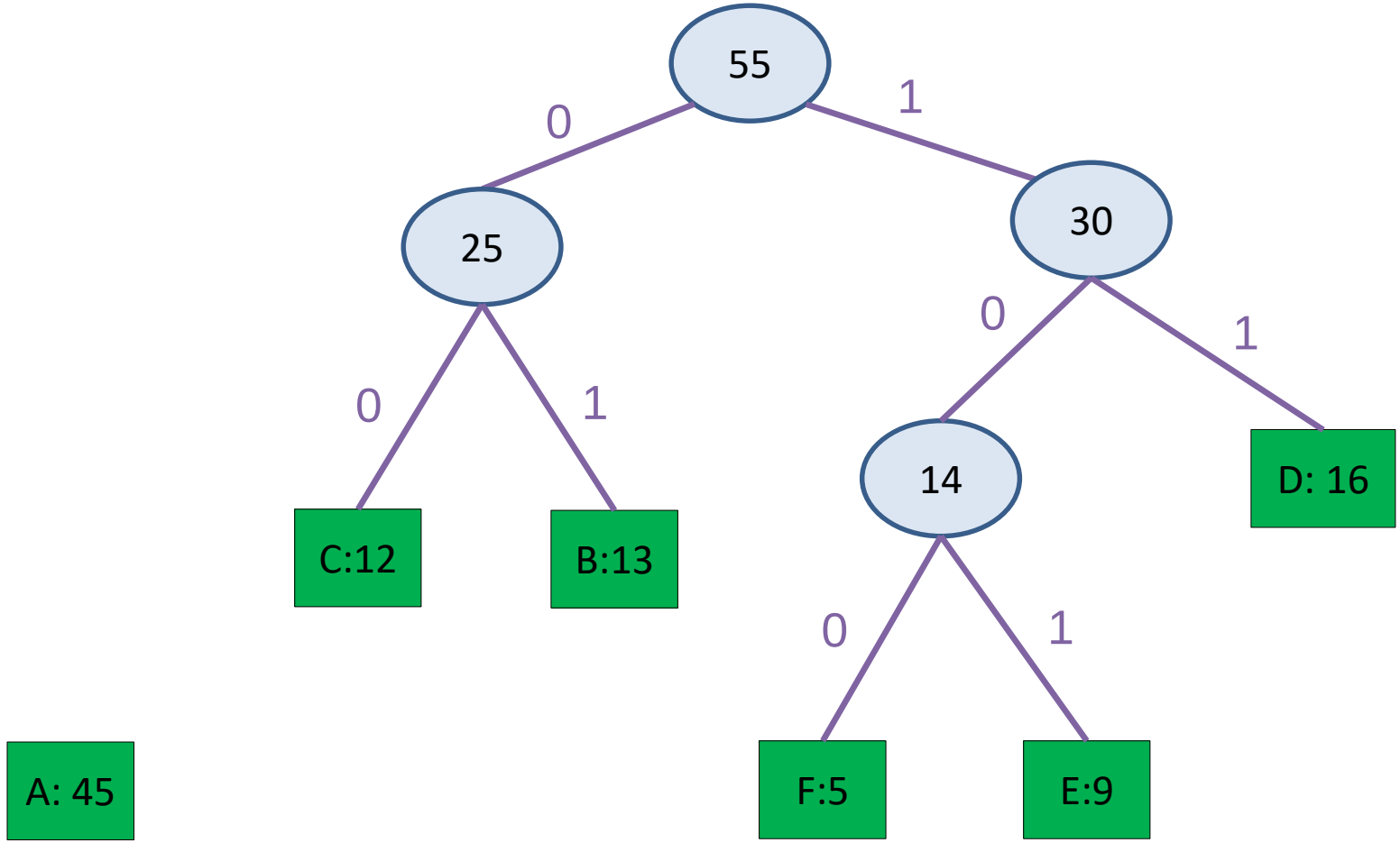
循环地选择具有最低频率的两个结点，生成一棵子树，直至所有结点形成一棵树



示例

例如：f:5，e:9，c:12，b:13，d:16，a:45

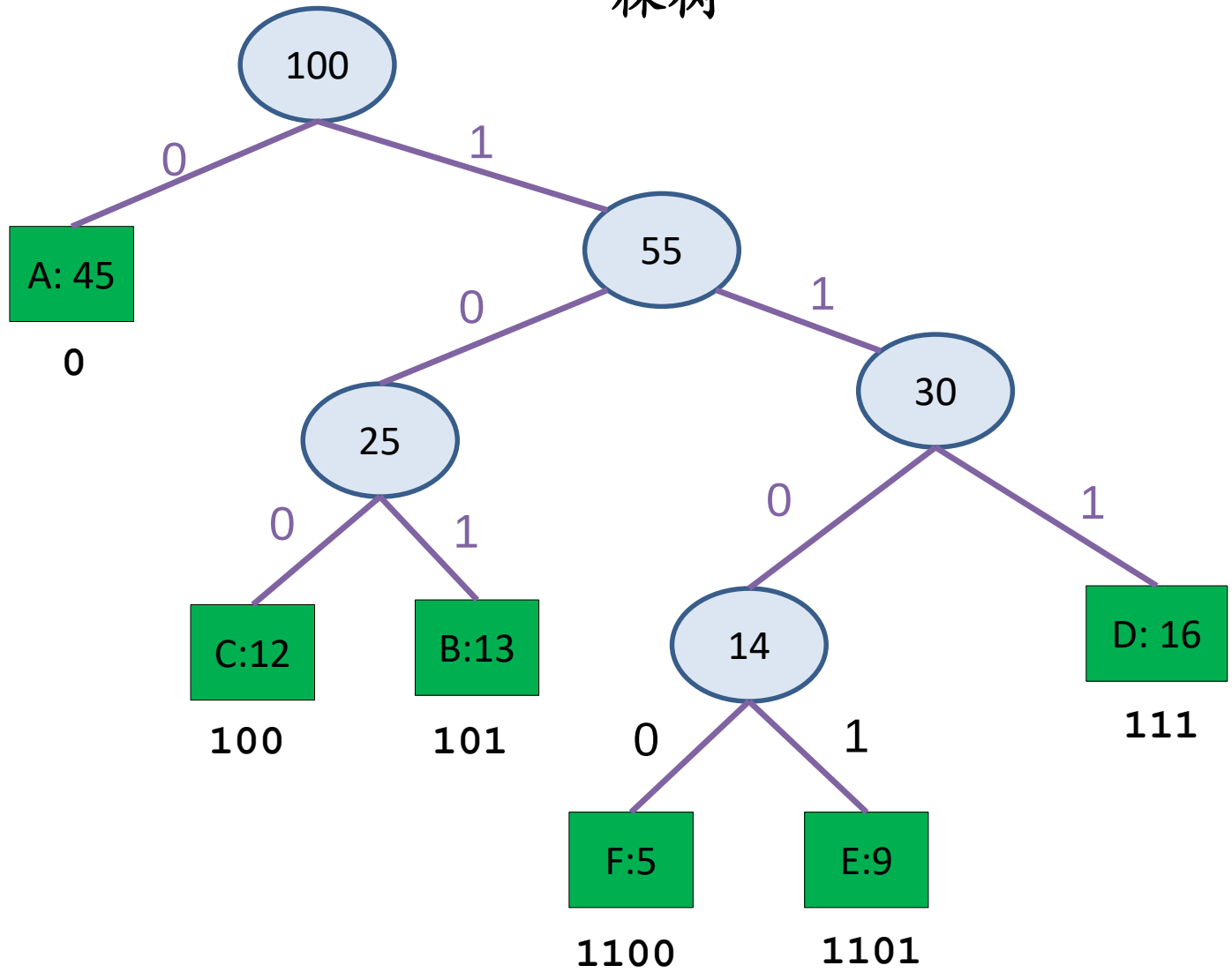
循环地选择具有最低频率的两个结点，生成一棵子树，直至所有结点形成一棵树



示例

例如：f:5，e:9，c:12，b:13，d:16，a:45

循环地选择具有最低频率的两个结点，生成一棵子树，直至所有结点形成一棵树



哈夫曼算法

Input: 字母表 C，对应的频率表 F

Output: 存储哈夫曼编码树的堆Q

过程： **Huffman(C, F)**

```
1.  n←|C|
2.  Q←C  /* 用BUILD-HEAP建立堆 */
3.  FOR i←1 To n-1 Do
4.      z←Allocate-Node( )
5.      x←left[z]←Extract-MIN(Q) /* 堆操作*/
6.      y←right[z]←Extract-MIN(Q) /* 堆操作*/
7.      f(z)←f(x) + f(y)
8.      Insert(Q, z)/* 堆操作*/
9.  Return Q
```

复杂性分析

- 设Q由一个堆实现
- 第2步 用堆排序的BUILD-HEAP实现: $O(n)$
- 每个堆操作要求$O(\lg n)$，循环$n-1$次: $O(n\lg n)$
- $T(n) = O(n) + O(n\lg n) = O(n\lg n)$

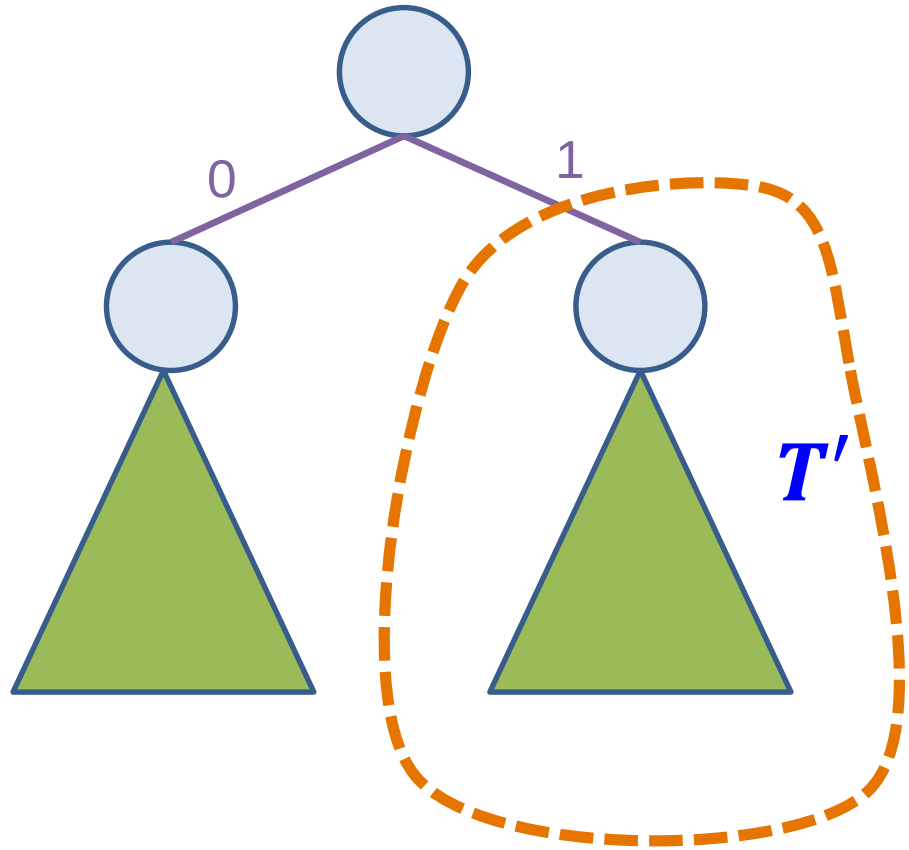
关于堆的相关知识在《算法导论》第6章

优化解的结构分析

- 我们需要证明
  - 哈夫曼编码算法具有优化子结构
  - 哈夫曼编码算法具有贪心选择性
  - 哈夫曼编码算法按照贪心选择性计算优化解

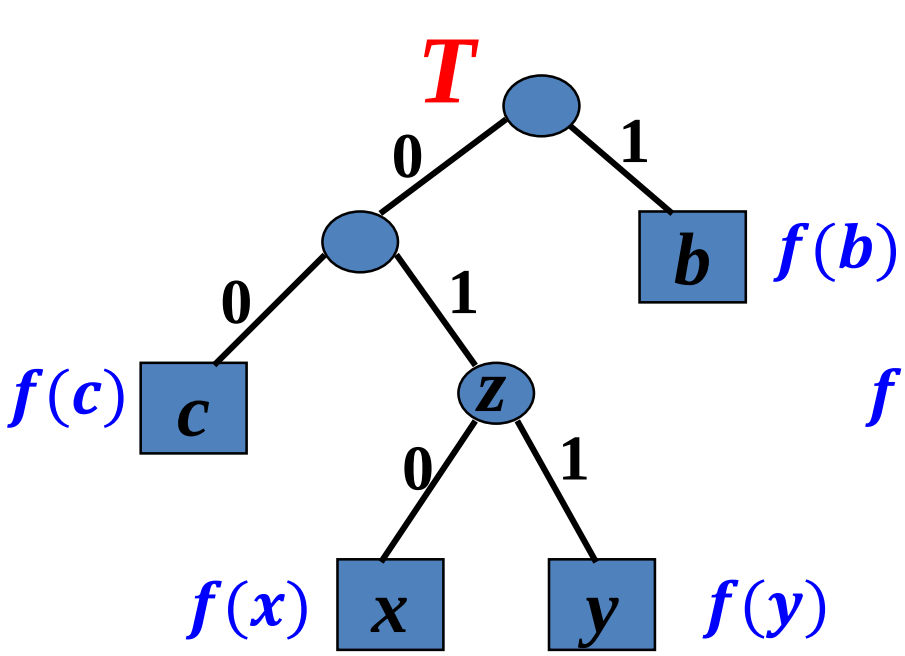
优化子结构

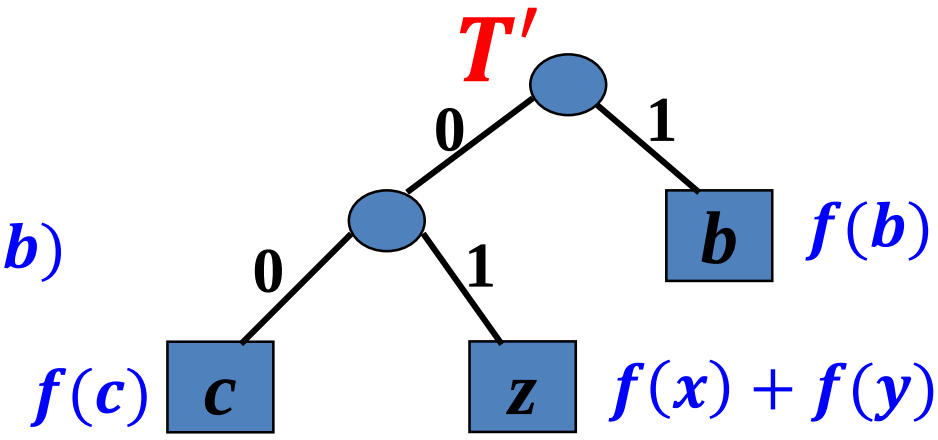
引理1： 设C是字母表，$\forall c \in C$，c具有频率$f(c)$，任意C的一个优化前缀树T的子树T'是他对应字符的优化前缀树。



优化子结构

引理2： 设T是字母表C的优化前缀编码树，$\forall c \in C$，$f(c)$是c在文件中出现的频率。设x、y是T中任意两个相邻叶结点，z是它们的父结点，则z编码频率是$f(z) = f(x) + f(y)$的字符，$T' = T - \{x, y\}$是字母表$C' = (C - \{x, y\}) \cup \{z\}$的优化前缀编码树。





优化子结构

证：先证$B(T) = B(T') + f(x) + f(y)$。

（后面用到）

- 对于$\forall v \in C - \{x, y\}$，有：

$d_{T(v)} = d_{T'(v)}, f(v)d_{T(v)} = f(v)d_{T'(v)}$

- 由 $d_T(x) = d_T(y) = d_{T'}(z) + 1$，有：

$$f(x)d_T(x) + f(y)d_T(y)$$

$$= \big(f(x) + f(y)\big)\big(d_{T'}(z) + 1\big)$$

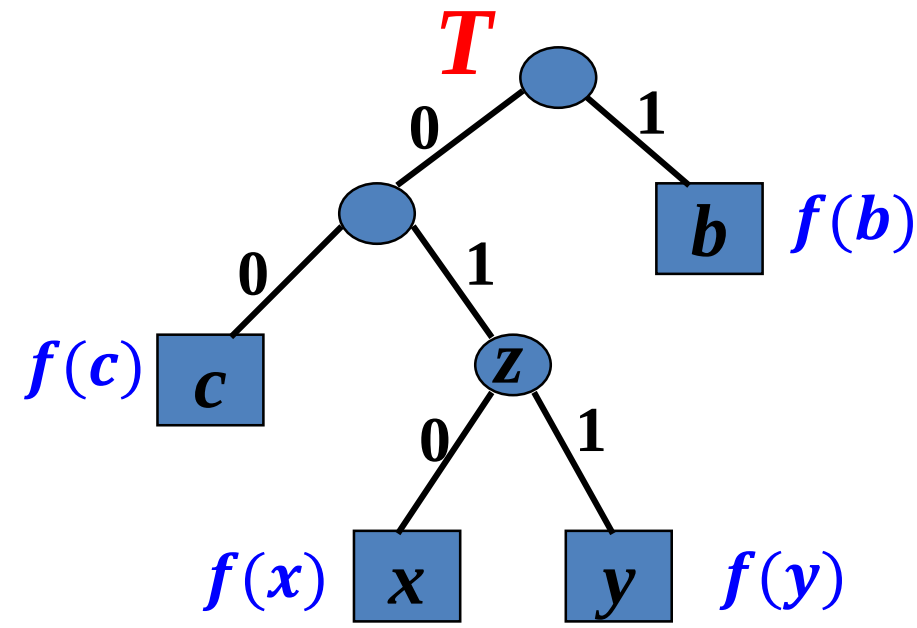
$$= (f(x) + f(y))d_{T'}(z) + (f(x) + f(y))$$

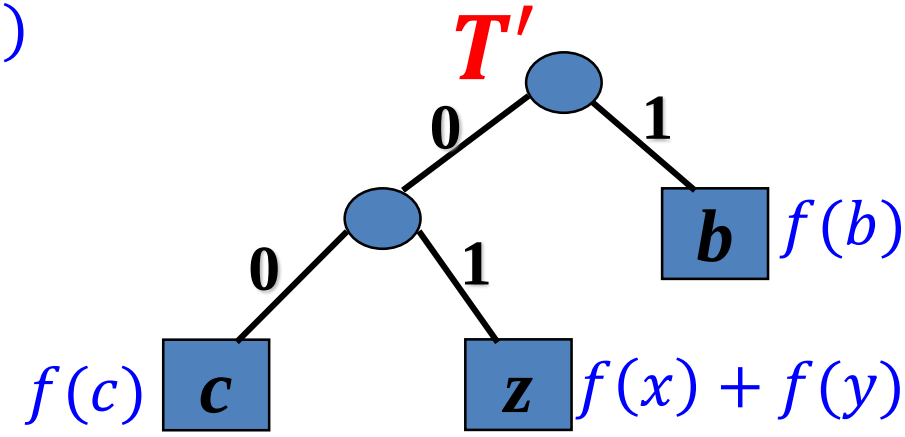
- 由 $f(x) + f(y) = f(z)$，有：

$$f(x)d_{T(x)} + f(y)d_{T(y)}$$

$$= f(z)d_{T'}(z) + (f(x) + f(y))$$

于是$B(T) = B(T') + f(x) + f(y)$。

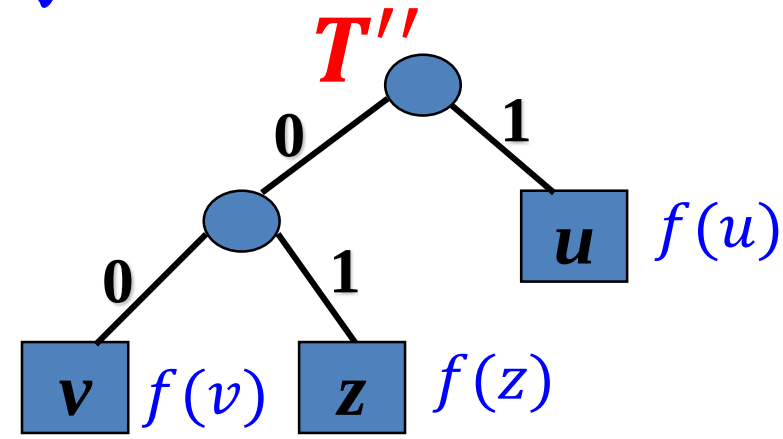




优化子结构

再证T'是C'的优化前缀编码树。

- 若T'不是C'的优化前缀编码树，则必存在T''，使$B(T'') < B(T')$。
- 因为z是C'中字符，它必为T''中的叶子结点。把结点x与y加入T''，作为z的左右子结点，则得到C的一个如下前缀编码树T'''（T'''是字母表C新的编码树）：



$$B(T''')$$

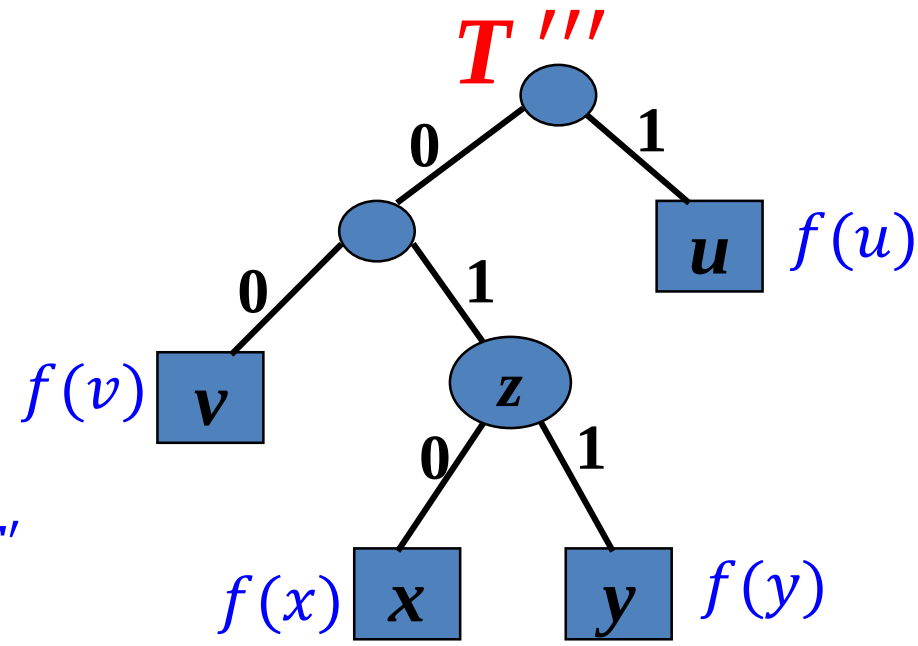
$$= \cdots + (f(x) + f(y))(d_{T''}(z) + 1)$$

$$= \cdots + f(z)d_{T''}(z) + (f(x) + f(y))$$

$$= B(T'') + f(x) + f(y)$$

$$< B(T') + f(x) + f(y) = B(T)$$

这与“T是字母表C的优化前缀编码树”相矛盾，故T'是C'的优化前缀编码树。



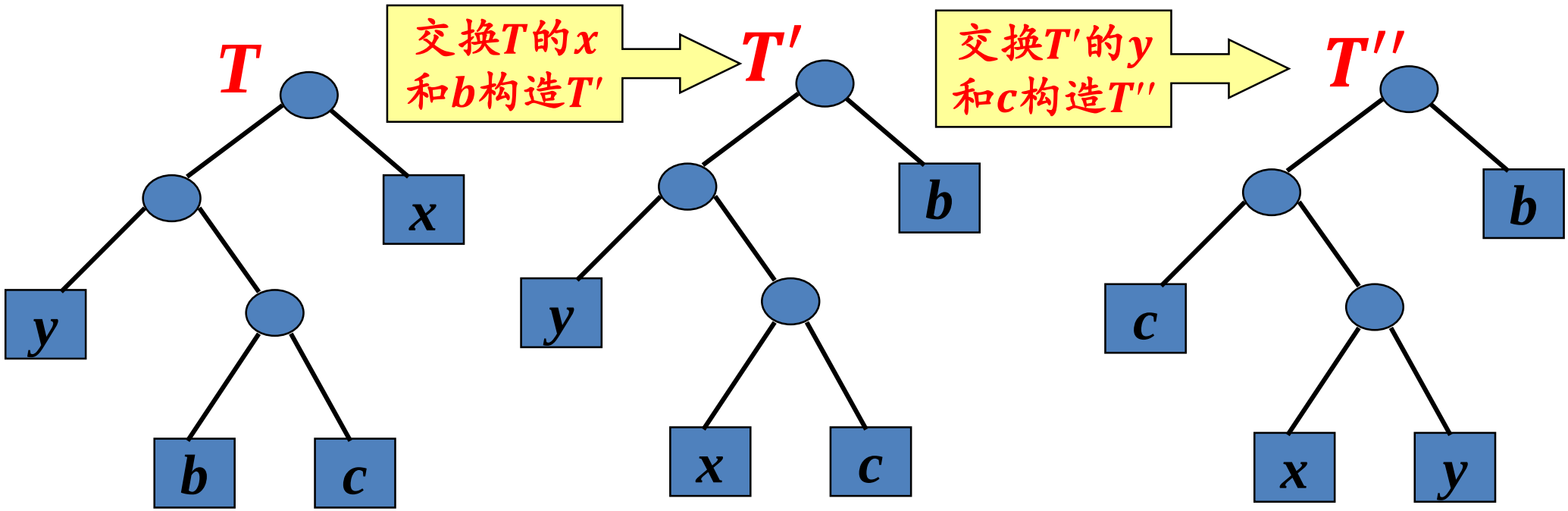
贪心选择性

引理3: 设C是字母表，$\forall c \in C$，c具有频率$f(c)$，x、y是C中具有最小频率的两个字符，则存在一个C的优化前缀编码树，x与y的编码具有相同长度，且仅在最末一位不同。

引理3说明：我们每次选取最小频率的两个字符作为优化编码树的当前的最深叶子结点，直到包含所有字符，就是我们要得到的哈夫曼编码树。

证明思路：只要找到一个符合条件的优化编码树即可。从字母表C的一个优化编码树出发，对其进行修改，得到一个满足条件（x与y的编码具有相同长度，且仅在最末一位不同）的新的编码树即可。

证： 设T是C的优化前缀树，且b和c是具有最大深度的两个兄弟字符：



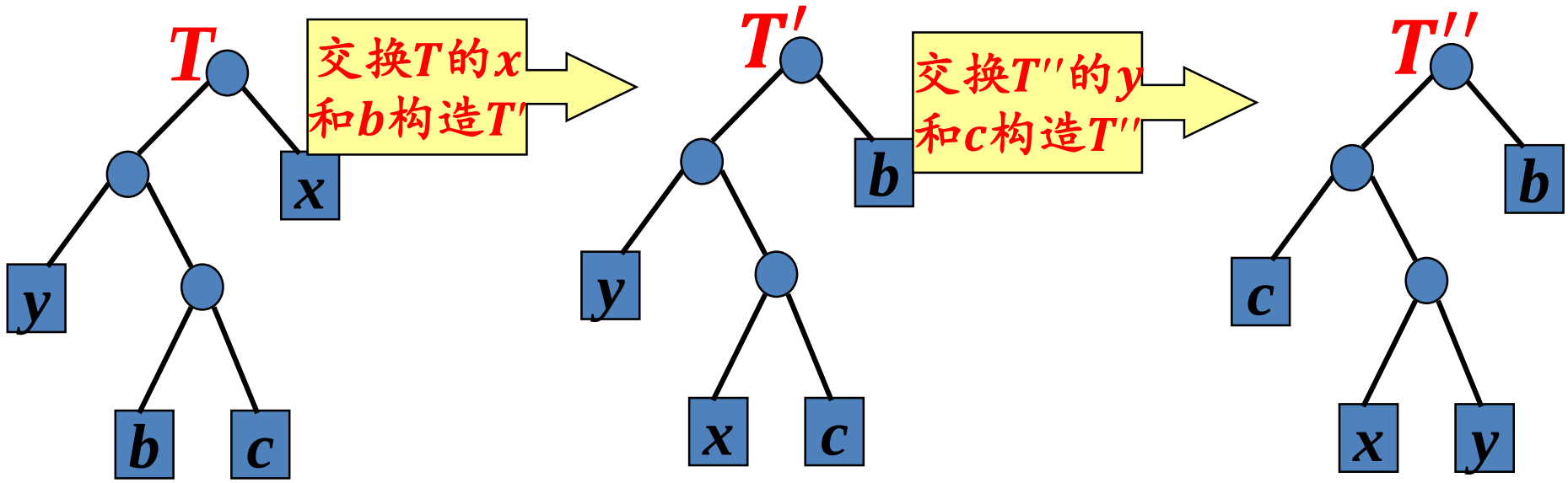
- 不失一般性，设$f(b) \le f(c)$，$f(x) \le f(y)$。因x与y是具有最低频率的字符，$f(b) \ge f(x)$，$f(c) \ge f(y)$。交换T的x和b构造T'，交换T'的y和c构造T''。
- 在T''中，x和y具有相同长度编码，且仅编码最后一位不同！现在需要证明T''是优化前缀编码树！（只要证明$B(T) = B(T'')$）

往证T''是最优化前缀树。

$$B(T)-B(T')=\sum_{c\in C}f(c)d_T(c)-\sum_{c\in C}f(c)d_{T'}(c)$$

$$=f(x)d_T(x)+f(b)d_T(b)-f(x)d_{T'}(x)-f(b)d_{T'}(b)$$

$$=f(x)d_T(x)+f(b)d_T(b)-f(x)d_T(b)-f(b)d_T(x)=(f(b)-f(x))(d_T(b)-d_T(x))$$



∵ $f(b)\geq f(x), d_T(b)\geq d_T(x)$（因为$b$的深度最大）∴ $B(T)-B(T')\geq 0, B(T)\geq B(T')$

同理可证$B(T')\geq B(T'')$。于是$B(T)\geq B(T'')$。

由于T是最优化的，所以$B(T)\leq B(T'')$。于是，$B(T)=B(T'')$，T''是C的最优前缀编码树。

在T''中，x和y具有相同长度编码，且仅最后一位不同。

贪心选择性

- **引理2:**
  设T是字母表C的优化前缀树，$\forall c \in C$，$f(c)$是c在文件中出现的频率。设x、y是T中任意两个相邻叶结点，z是它们的父结点，则z作为频率是$f(z)=f(x)+f(y)$的字符，$T'=T-\{x,y\}$是字母表$C'=C-\{x,y\}\cup\{z\}$的优化前缀编码树。
- **引理3:**
  设C是字母表，$\forall c \in C$，c具有频率$f(c)$，x、y是C中具有最小频率的两个字符，则存在一个C的优化前缀树，x与y的编码具有相同长度，且仅在最末一位不同。
- **引理2，3一起使用表明:**
  我们可以贪心的从最小频率的字符开始，迭代进行（归纳），从而得到所有字符的优化前缀树。

正确性证明

定理. Huffman算法产生一个优化前缀编码树。

证明. 由于引理2、引理3成立，而且Huffman算法按照引理2的贪心选择性确定的规则进行局部优化选择，所以Huffman算法产生一个优化前缀编码树。

本讲内容

5.1 贪心法的基本原理

5.2 任务安排问题

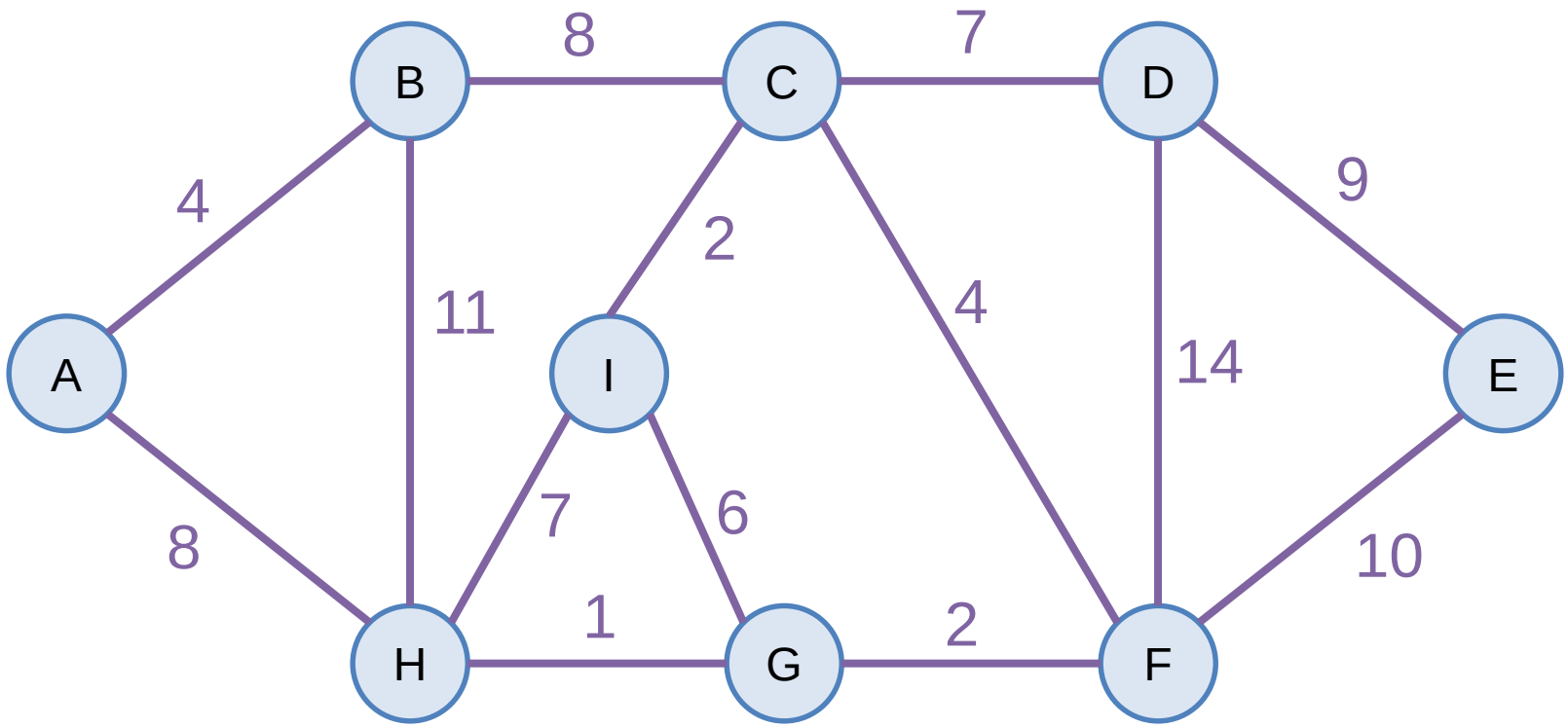
5.3 哈夫曼编码问题

5.4 最小生成树

《算法导论》第23章

最小生成树(Minimum Spanning Tree)

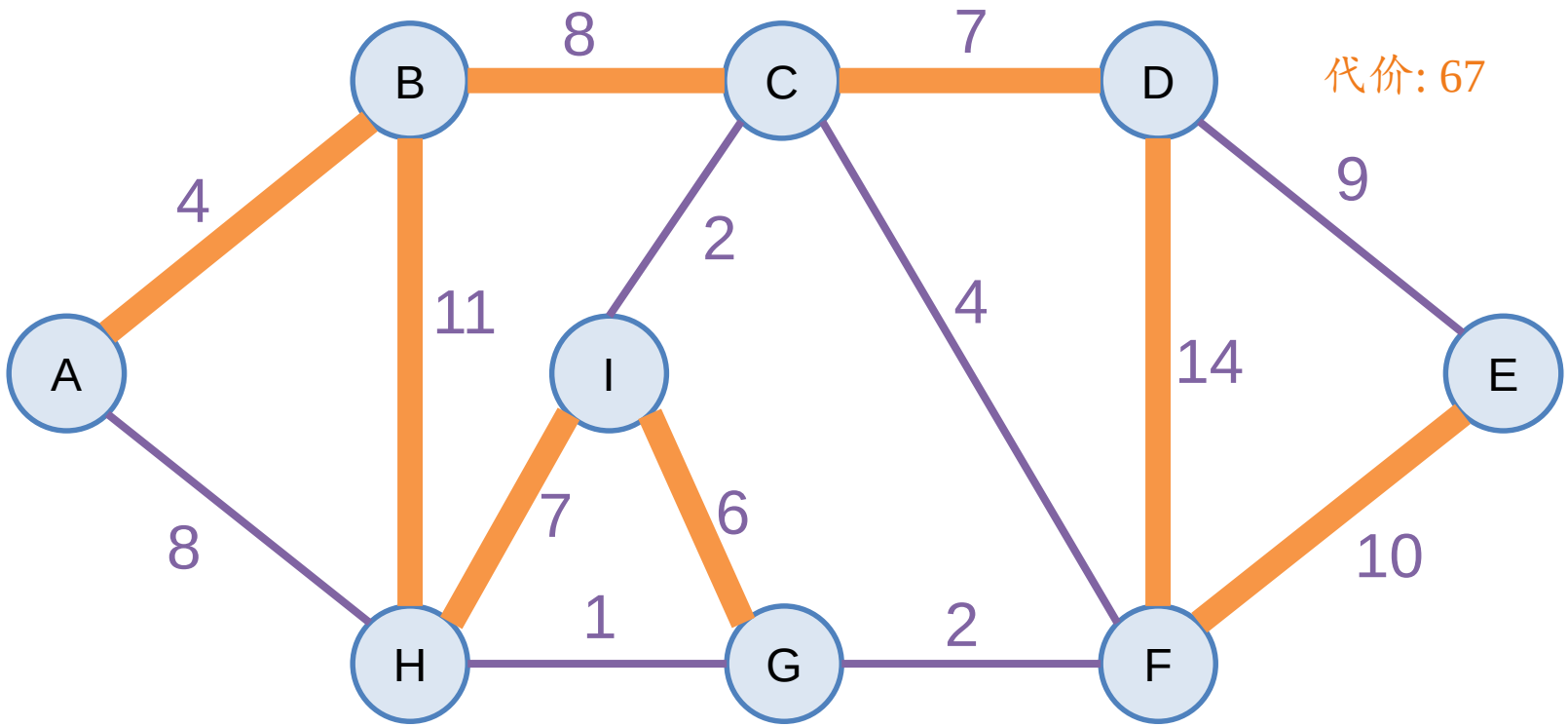
$G=(V,E)$是一个边加权无向连通图



- 生成树：连通图G的一个子图如果是一棵包含G的所有顶点的树，则该子图称为G的生成树。
- 生成树是连通图的极小连通子图。所谓极小是指：若在树中任意增加一条边，则将出现一个回路；若去掉一条边，将会使之变成非连通图。

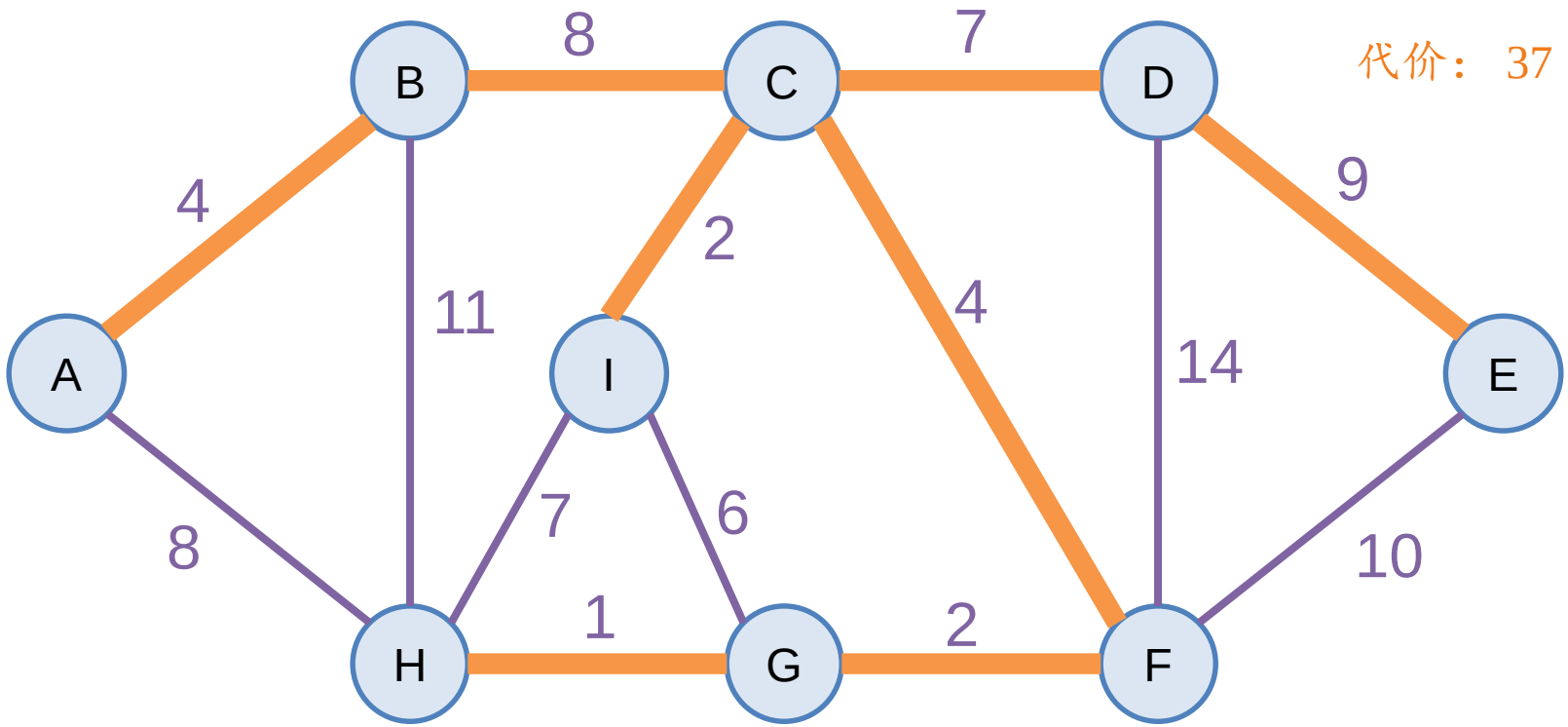
最小生成树(Minimum Spanning Tree)

- 生成树是一个连接了图中G所有结点V的树
- **生成树的代价是生成树上所有边的权重之和**



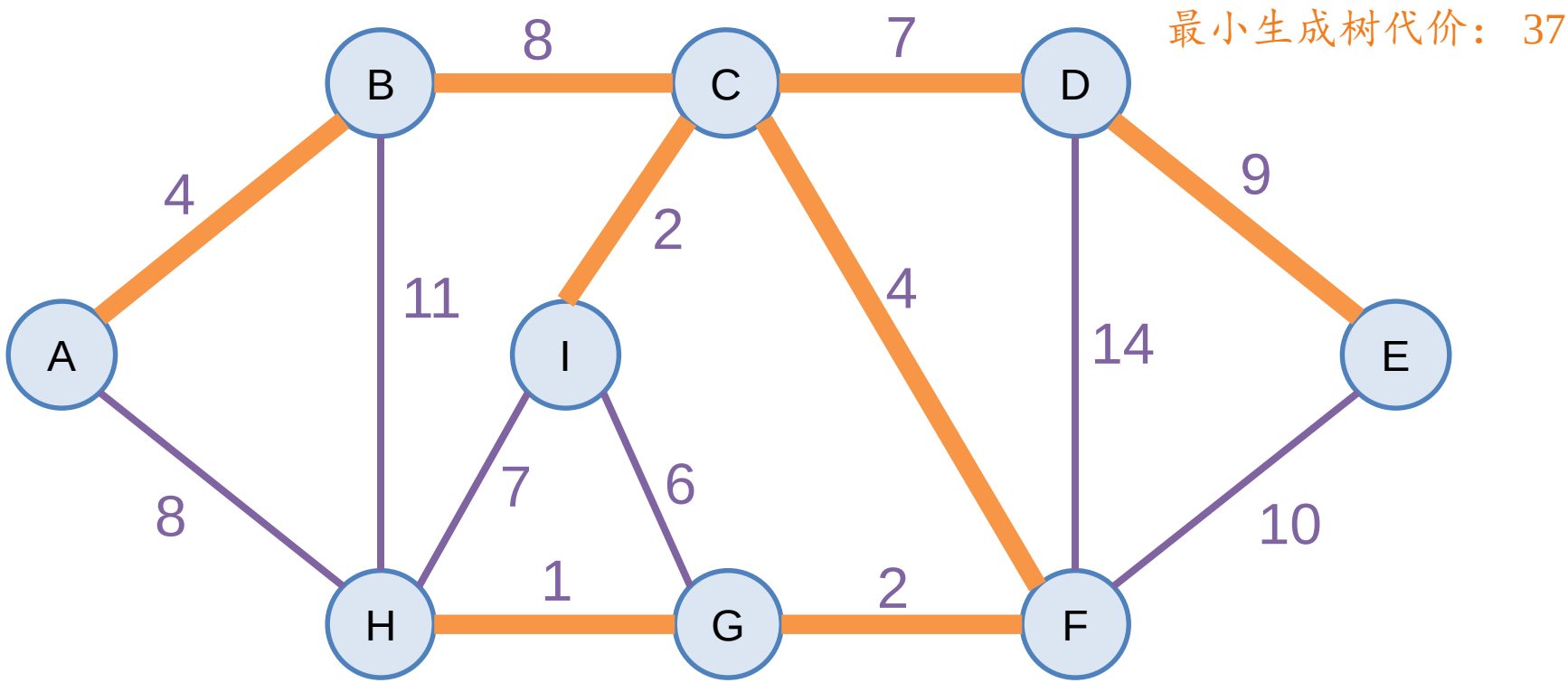
最小生成树(Minimum Spanning Tree)

- 生成树是一个连接图中所有结点的树
- **生成树的代价是生成树上所有边的权重之和**



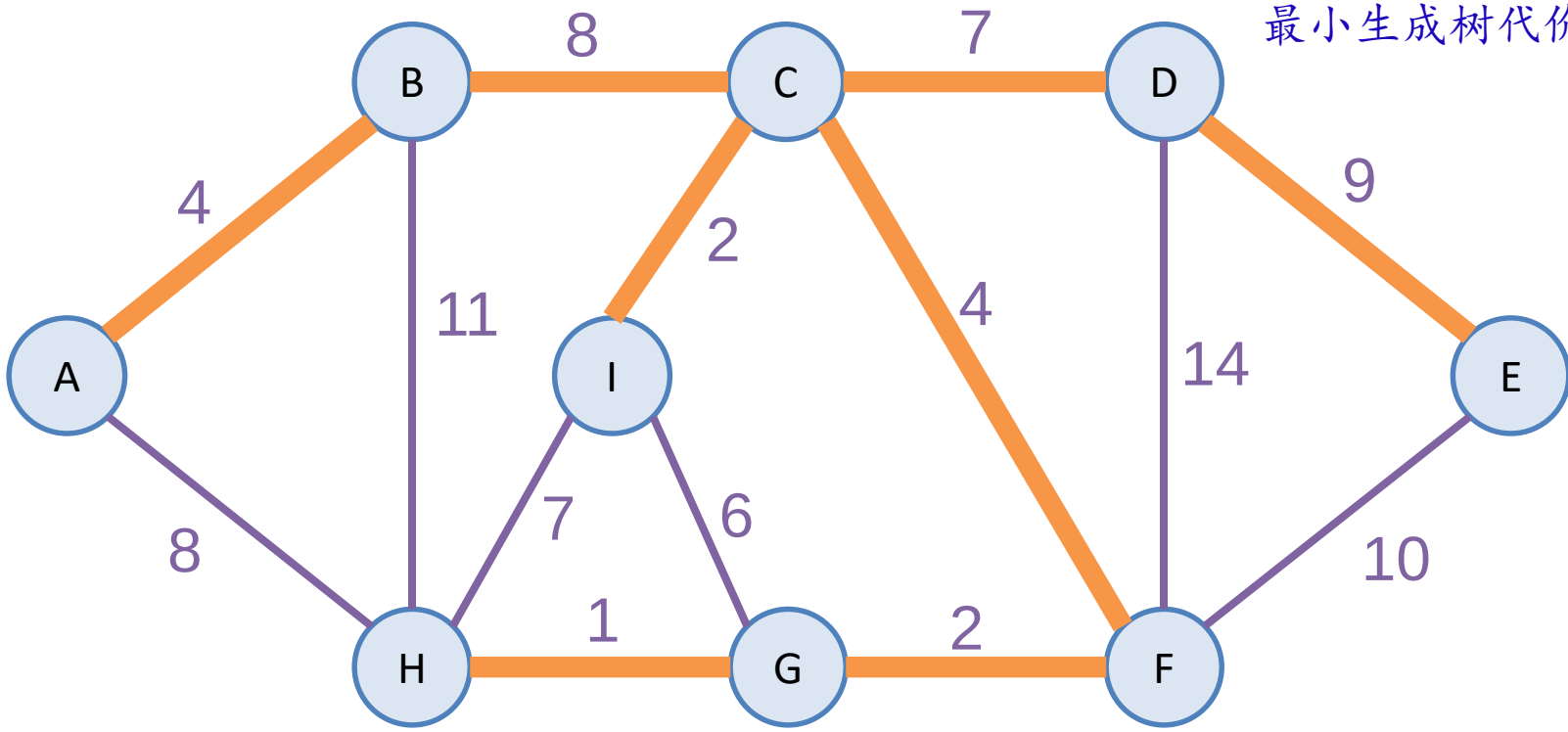
最小生成树(Minimum Spanning Tree)

- 最小生成树是具有最小代价的生成树



应用：旅行商问题、最优路径规划、国家的电力网、公路网、通信网

问题定义

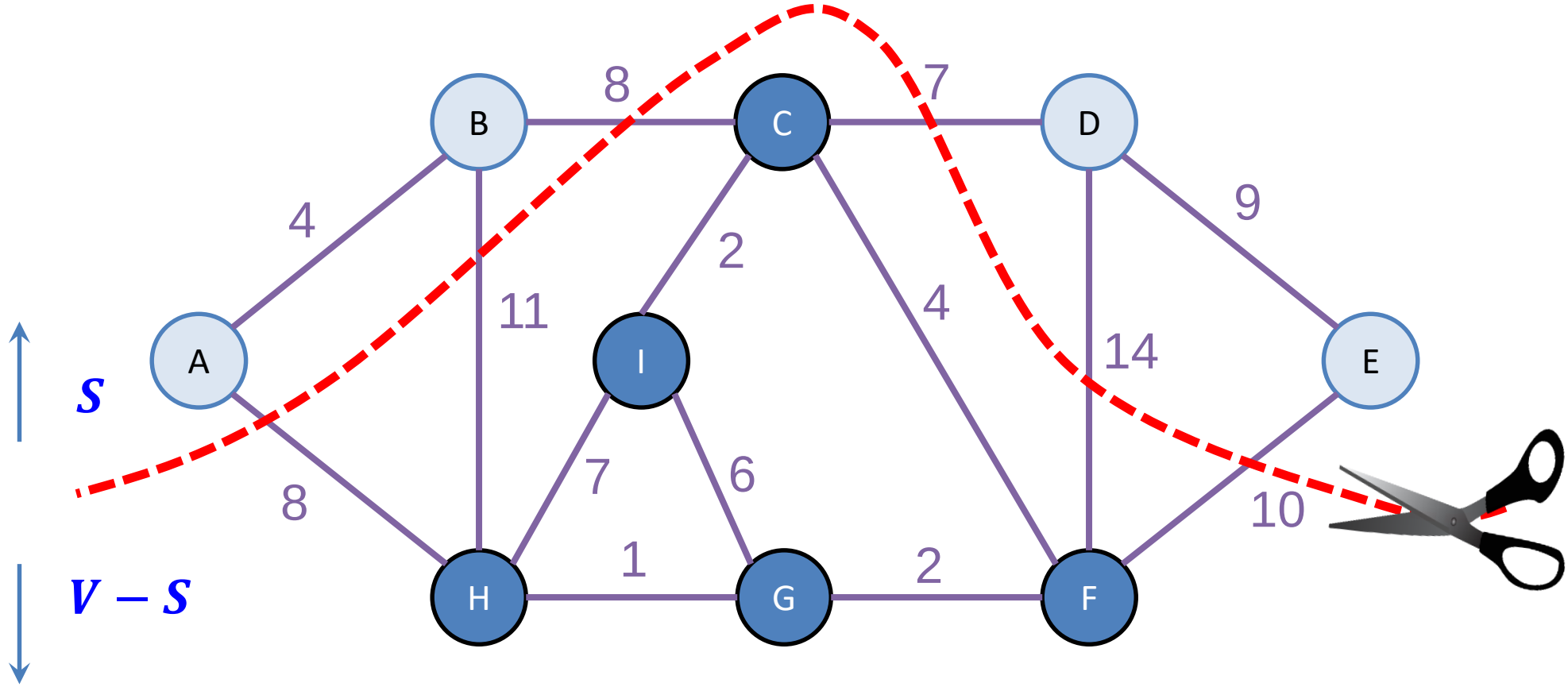


输入：一个边加权无向连通图$G=(V,E)$

输出：最小代价的生成树（不唯一）

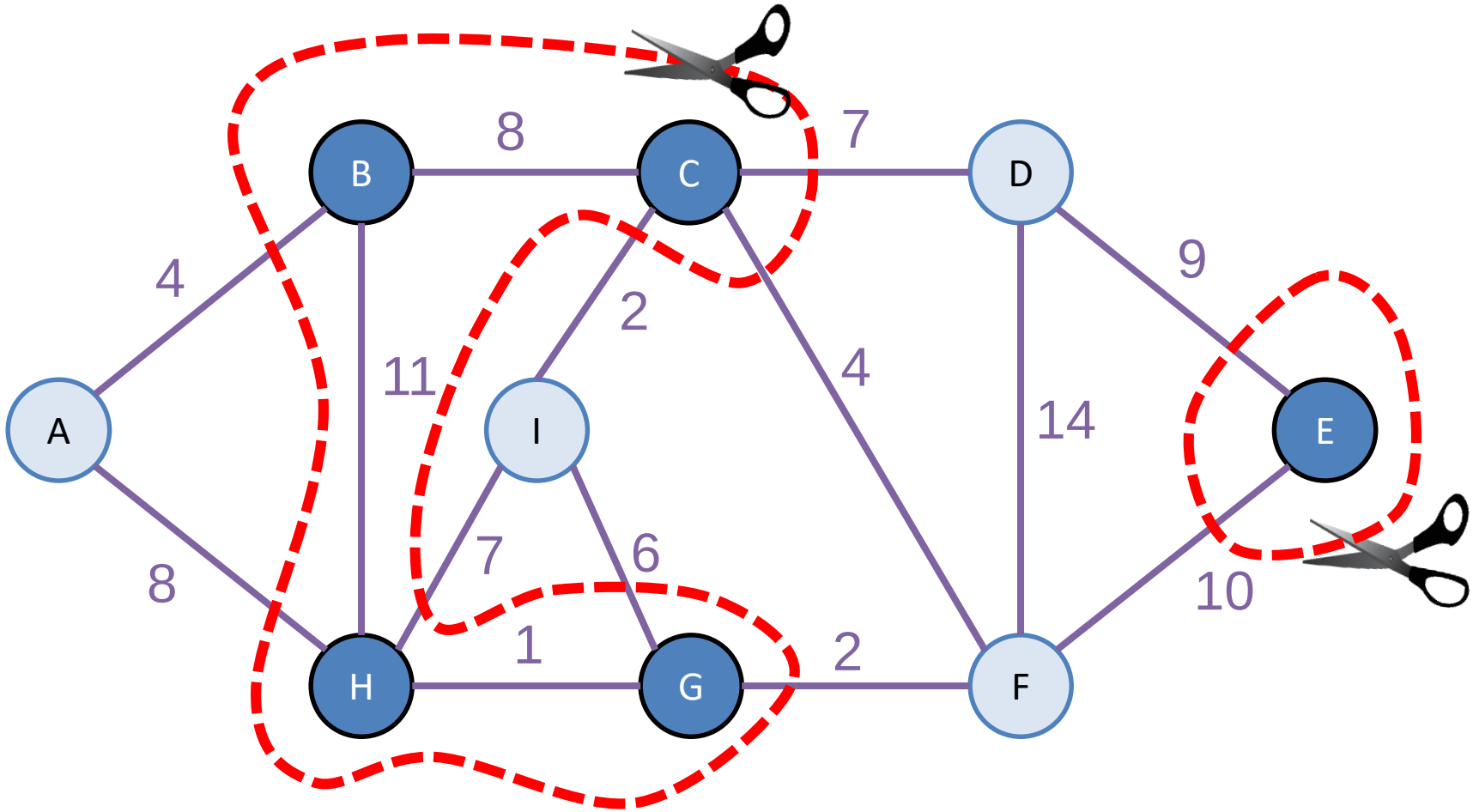
需要用到的几个概念

图割
（Graph Cut）



“$\{A, B, D, E\}$ 和$\{C, I, H, G, F\}$” 的割

图割
（Graph Cut）

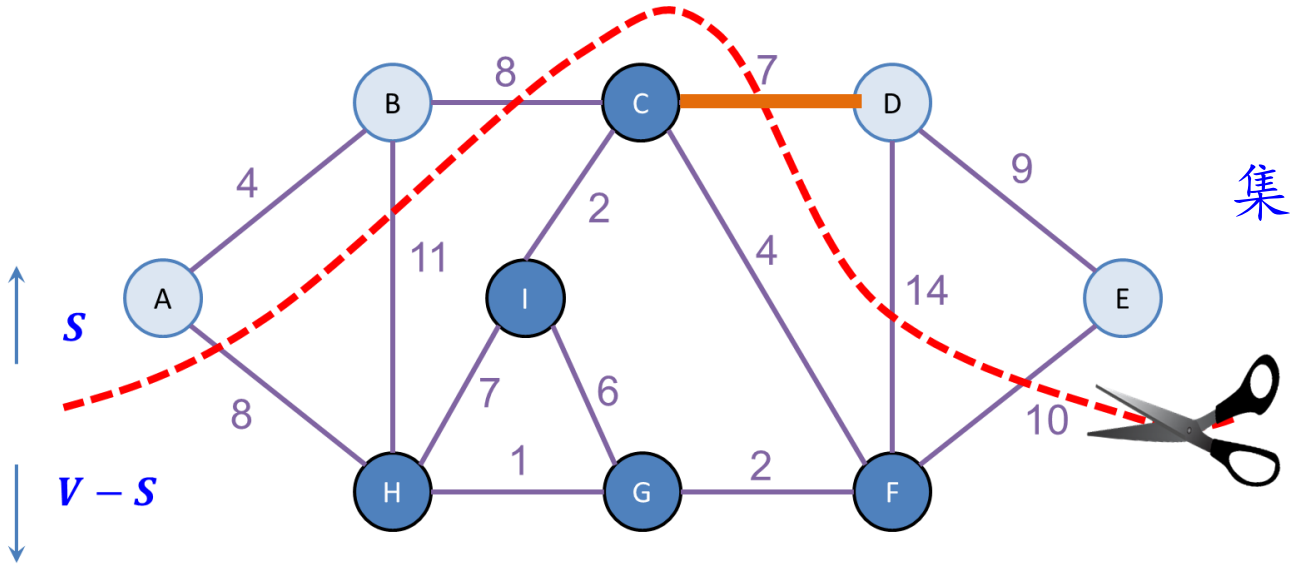


“$\{B, C, H, G, E\}$ 和$\{A, I, D, F\}$” 的割

图割

无向图$G=(V,E)$的一个割$(S,V-S)$，是集合V的一个划分：

- 如果E中一条边(u,v)的一个端点在S中，另一个端点在$V-S$中，则称这条边横跨割$(S,V-S)$。
- 如果集合A中不存在横跨割的边，则称该割尊重集合A。
- 在横跨该割所有边中，权重最小的边称为轻量级边（不唯一）。



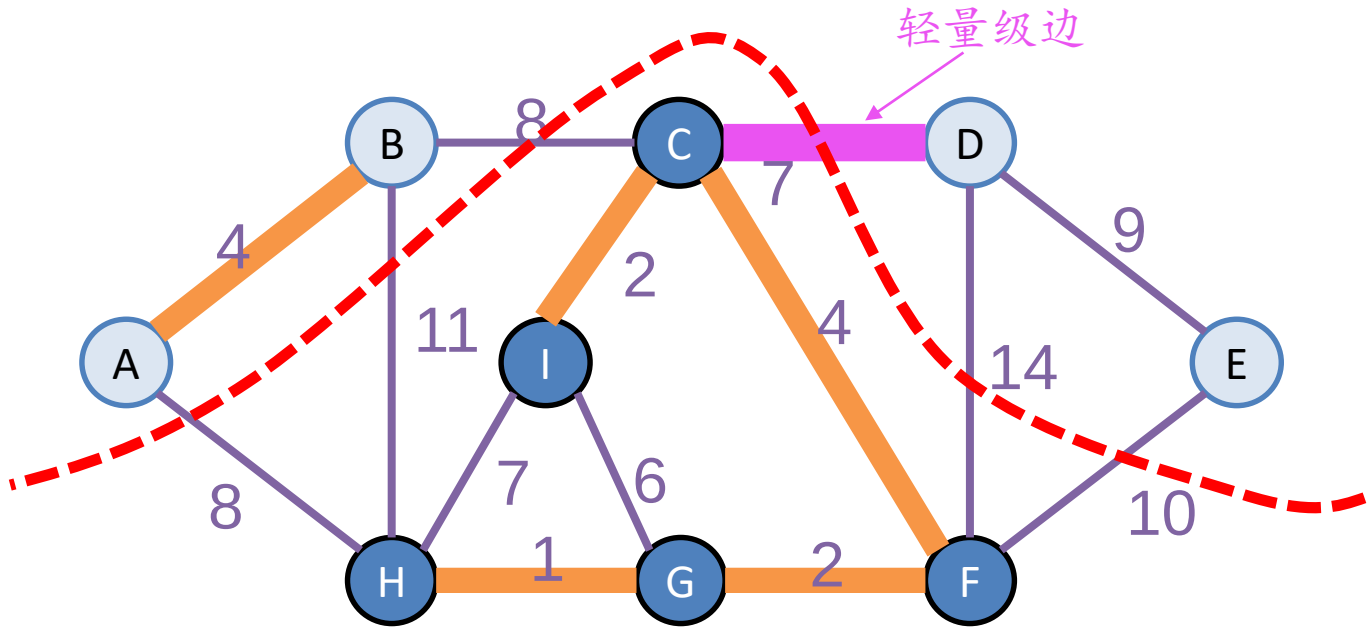
集合$A=\{(A,B),(C,I),(C,F)\}$

引理1

引理1：令S是图$G=(V,E)$的边集合E的子集，并且存在一个割尊重S，

- 如果存在G的一个最小生成树包含S
- 令(u,v)是该割的轻量级边

则**存在**一个G的最小生成树包含$S \cup \{(u,v)\}$。



S是橙色的边

证明：

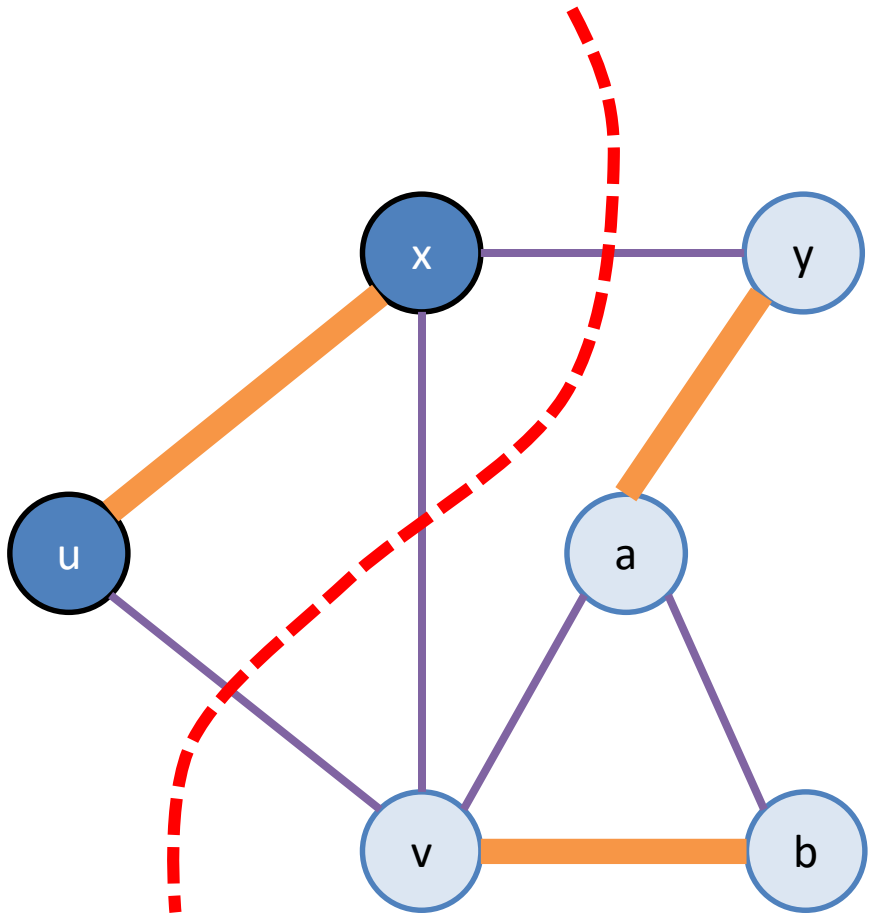
- 假设我们有：
  - 图G的一个割尊重 S
  - S是图G的一个最小生成树T的子集

引理1

引理1：令S是图G的边的子集，并且存在一个割尊重S，

- 如果存在G的一个最小生成树包含S
- 令(u, v)是该割的轻量级边

则存在一个G的最小生成树包含$S \cup \{(u, v)\}$。



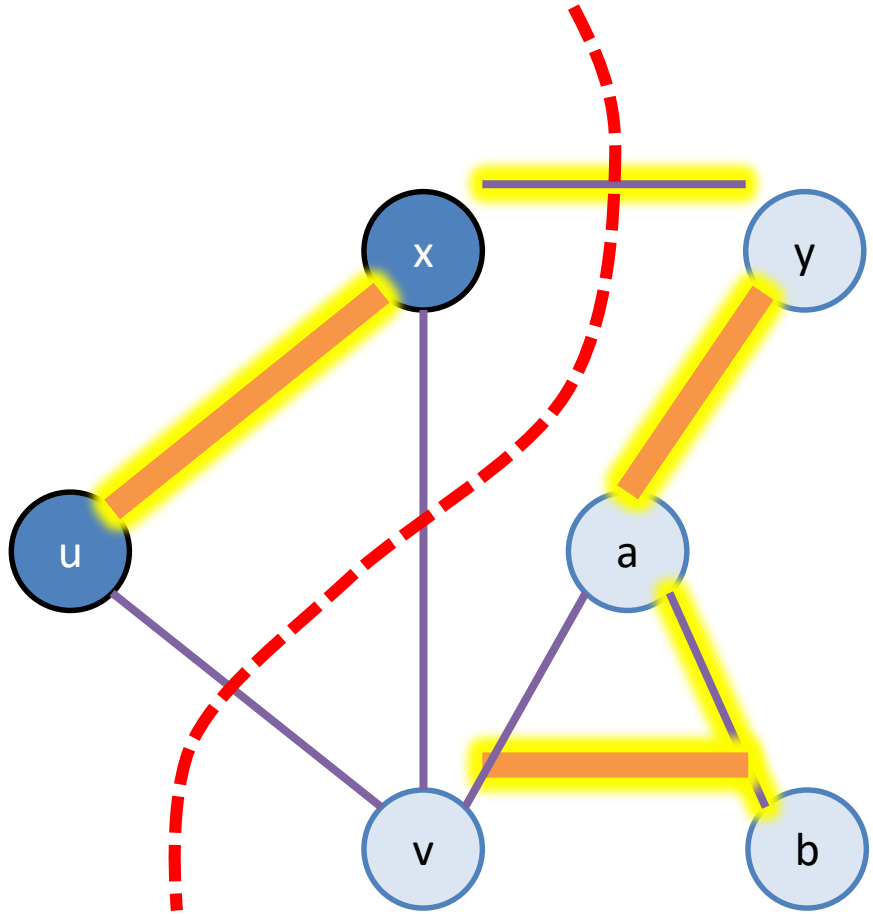
引理1

引理1：令S是图G的边的子集，并且存在一个割尊重S，
- 如果存在G的一个最小生成树包含S
- 令(u, v)是该割的轻量级边

则存在一个G的最小生成树包含$S \cup \{(u, v)\}$。

证明：

- 假设我们有：
  - 图G的一个割尊重S
  - S是图G的一个最小生成树T的子集
- 令(u, v)是横跨该割的轻量级边
  - 也即(u, v)在所有横跨割的边中，权重最小



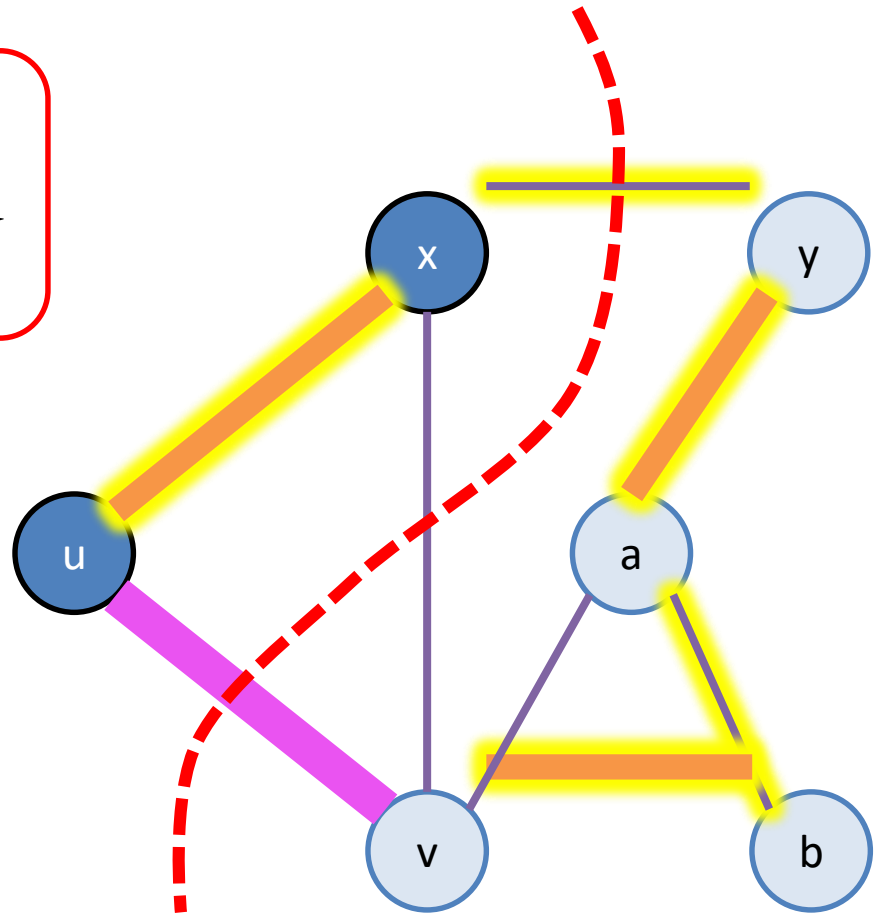
引理1

引理1：令S是图G的边的子集，并且存在一个割尊重S，
- 如果存在G的一个最小生成树包含S
- 令(u, v)是该割的轻量级边

则存在一个G的最小生成树包含$S \cup \{(u, v)\}$。

证明：

- 假设我们有：
  - 图G的一个割尊重S
  - S是图G的一个最小生成树T的子集
- 令(u, v)是横跨该割的轻量级边
  - 也即(u, v)在所有横跨割的边中，权重最小
- 假设(u, v)不在T中，不然引理成立
  - 将(u, v)添加到T中，将会产生一个环（因为T是一个最小生成树，含有$|V-1|$个边）



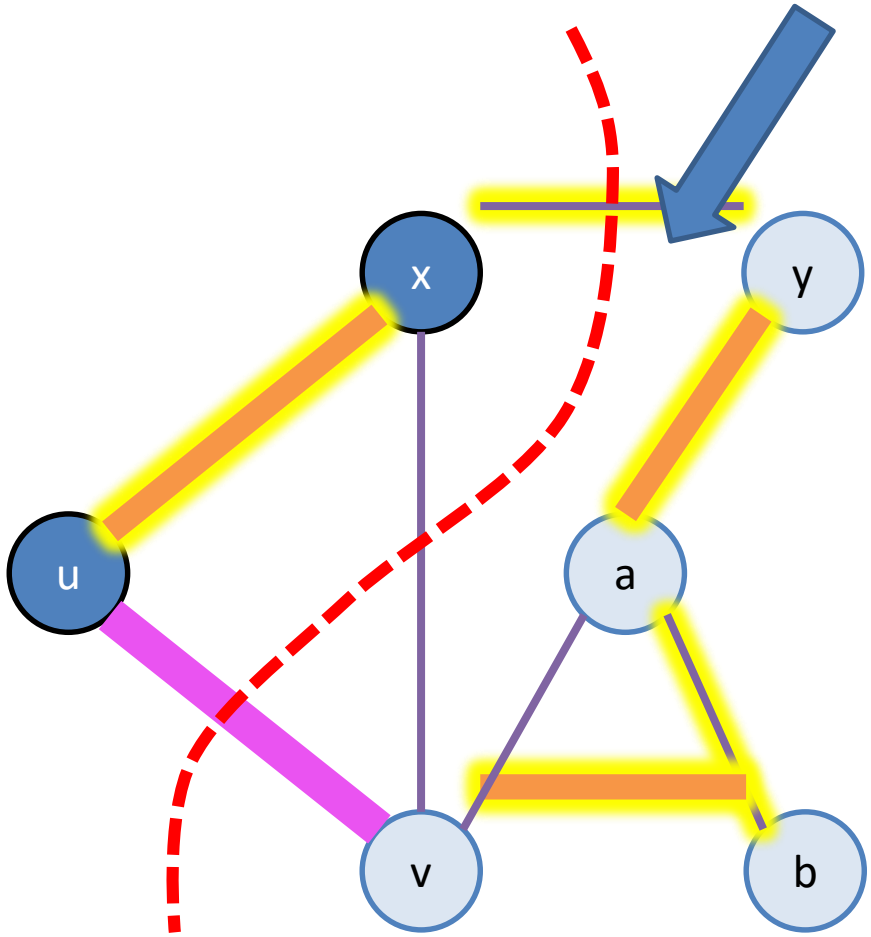
引理1

引理1：令S是图G的边的子集，并且存在一个割尊重S，
- 如果存在G的一个最小生成树包含S
- 令（u, v）是该割的轻量级边

则存在一个G的最小生成树包含$S \cup \{(u, v)\}$。

证明：

- 假设我们有：
  - 图G的一个割尊重 S
  - S是图G的一个最小生成树T的子集
- 令(u, v) 是横跨该割的轻量级边
  - 也即(u, v)在所有横跨割的边中，权重最小
- 假设(u, v) 不在 T中，不然引理成立
  - 将(u, v)添加到T中，将会产生一个环
- 因此T至少存在另外一条边横跨该切割
  - 令其为(x, y)



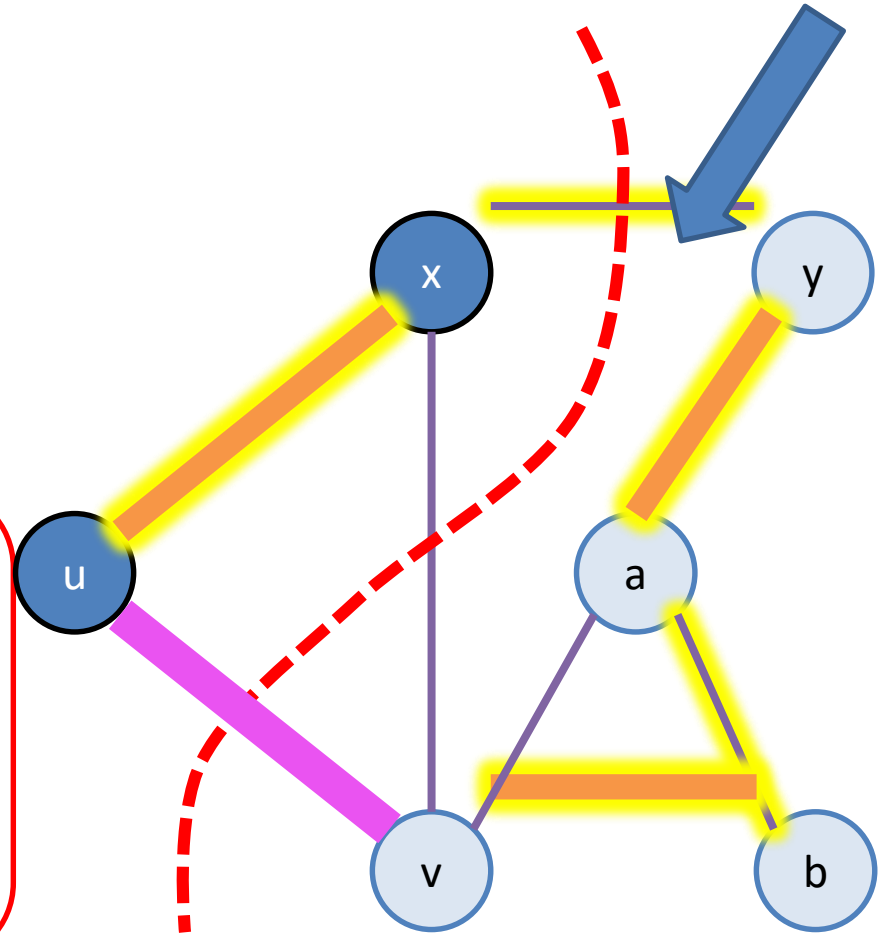
引理1

引理1：令S是图G的边的子集，并且存在一个割尊重S，
- 如果存在G的一个最小生成树包含S
- 令（u, v）是该割的轻量级边

则存在一个G的最小生成树包含$S \cup \{(u, v)\}$。

证明：

- 假设我们有：
  - 图G的一个割尊重 S
  - S是图G的一个最小生成树T的子集
- 令(u, v) 是横跨该割的轻量级边
  - 也即(u, v)在所有横跨割的边中，权重最小
- 假设(u, v) 不在 T中，不然引理成立
  - 将(u, v)添加到T中，将会产生一个环
- 因此T至少存在另外一条边横跨该切割
  - 令其为(x, y)
- 删除 (x, y) 得到生成树 T'
- 由于 (u, v) 是轻量级的边，所以 (u, v) 的代价小于等于 (x, y)。也即， T' 的代价小于等于 T，由于 T 是最小生成树，所以 T' 的代价只能等于 T。即 T' 也是最小生成树且包含$S \cup \{(u, v)\}$

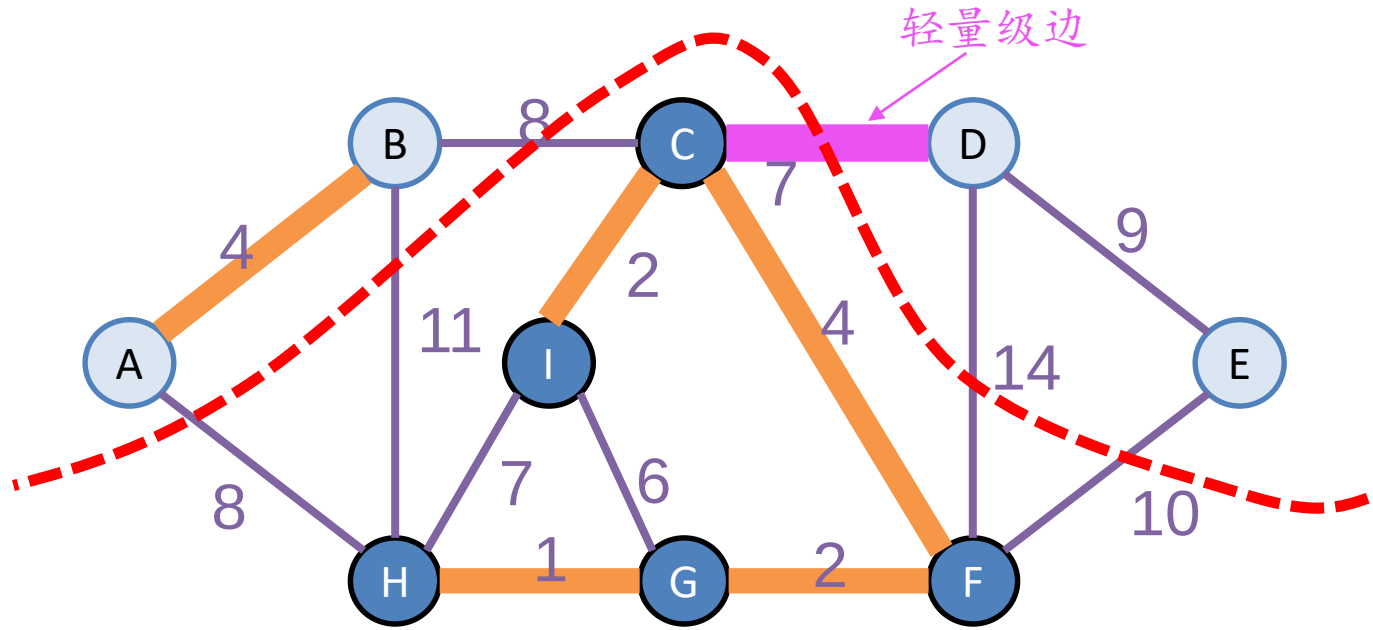


引理1

引理1：令S是图$G=(V,E)$的边集合E的子集，并且存在一个割尊重S，

- 如果存在G的一个最小生成树包含S
- 令(u,v)是该割的轻量级边

则**存在**一个G的最小生成树包含$S \cup \{(u,v)\}$。



S是橙色的边

最小生成树的贪心算法

贪心思想是什么？
- 贪“点”
- 贪“边”

Prim算法

- 贪心思想
  - 也称“加点法”
  - 每次迭代选择轻量级边对应的点，加入到最小生成树中。算法从某一个顶点开始，逐渐长大直到覆盖整个连通图的所有顶点。

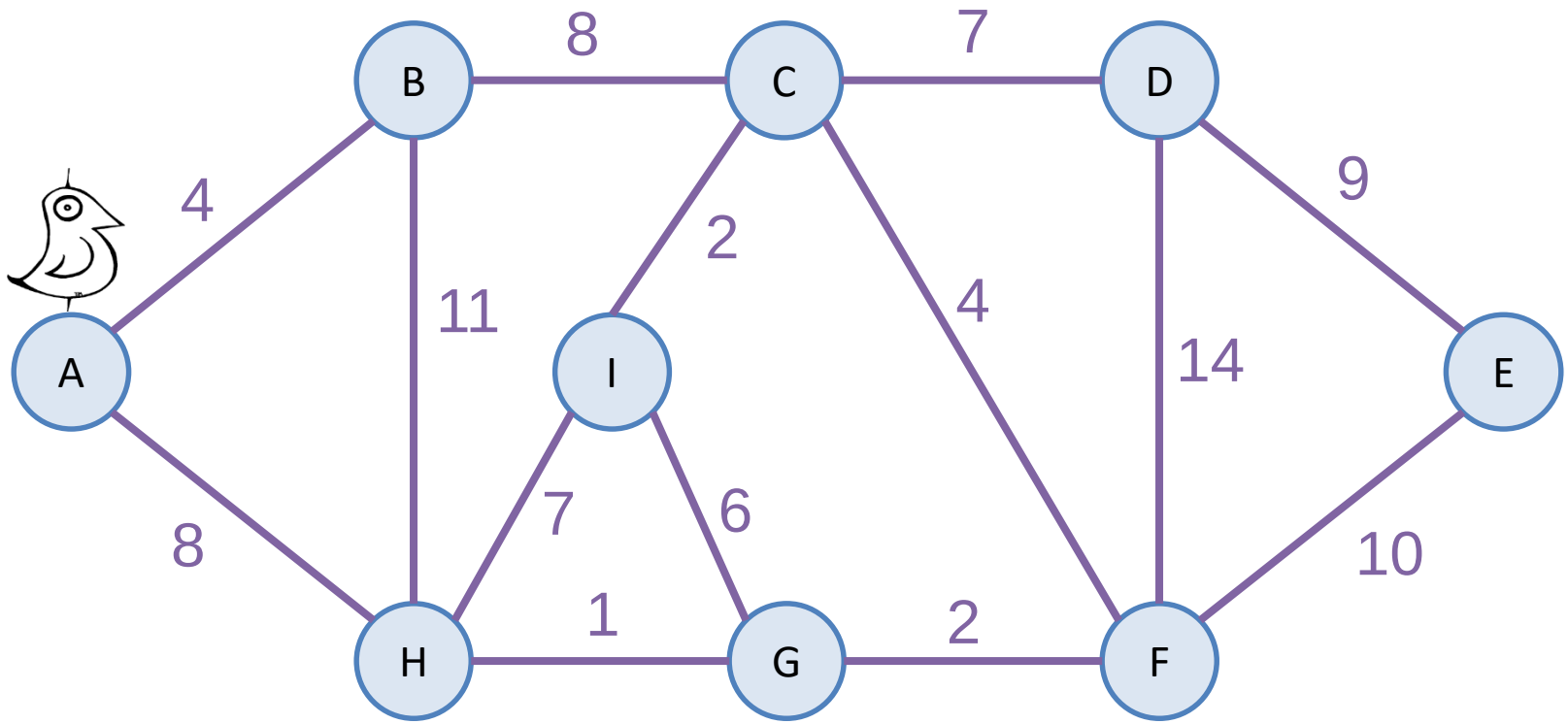
普里姆算法的基本思想:

1. 取图中任意一个顶点 v 作为生成树的根，之后往生成树上添加新的顶点 w。

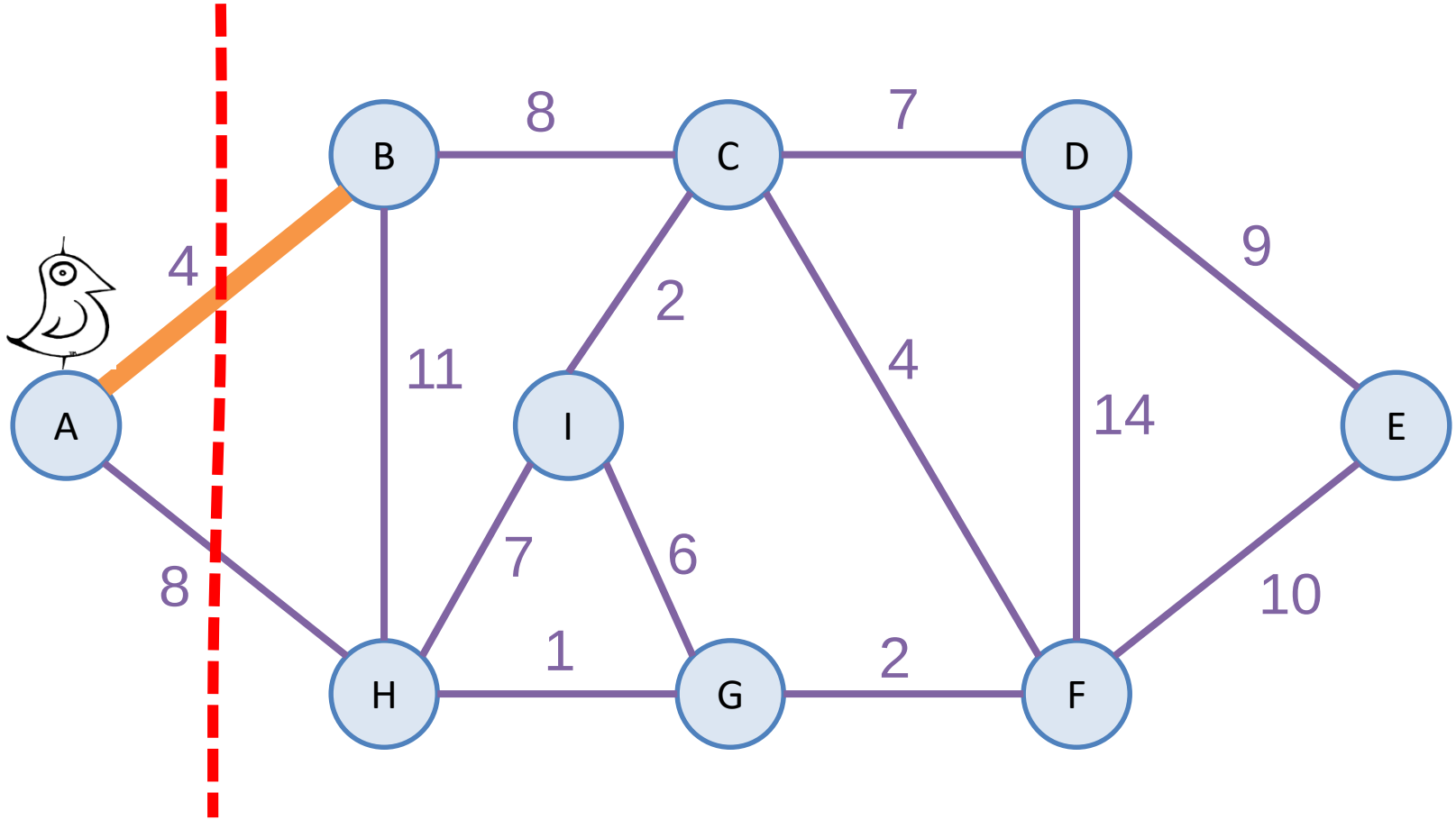
2.在添加的顶点 w 和已经在生成树上的顶点v 之间必定存在一条边，并且该边的权值在所有连通顶点 v 和 w 之间的边中取值最小。

3.继续往生成树上添加顶点，直至生成树上含有 n 个顶点为止。

Prim算法

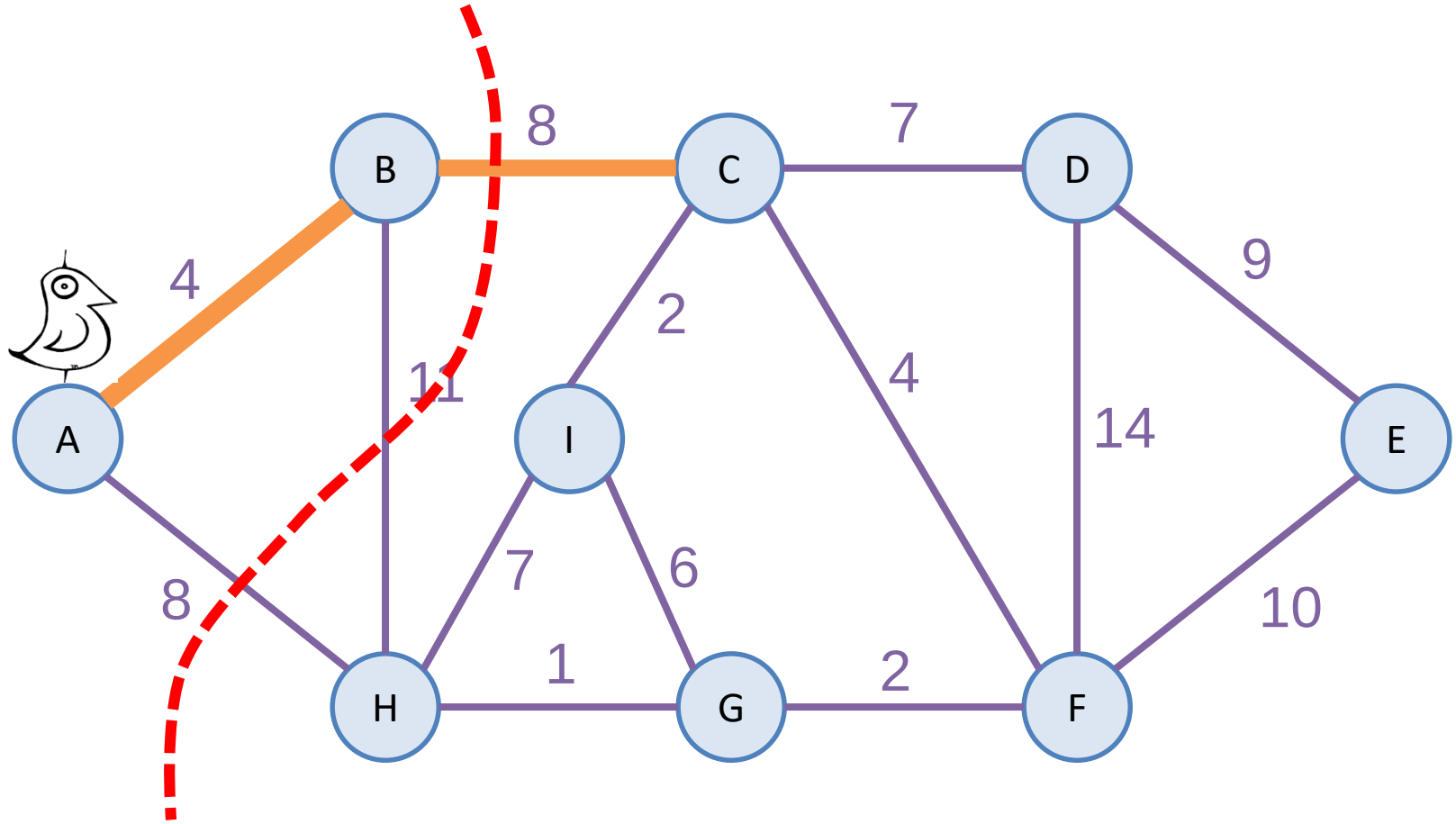


Prim算法



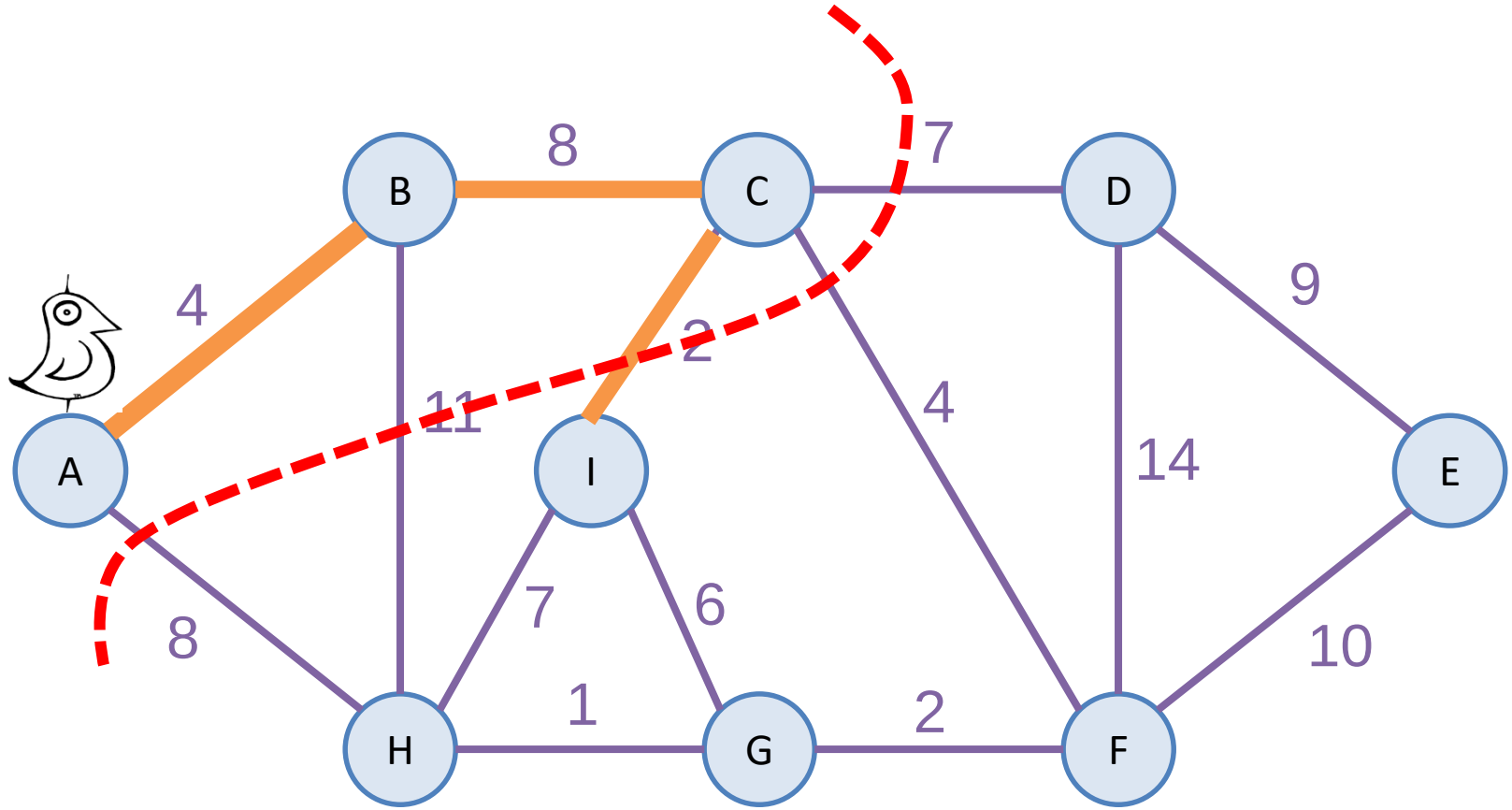
$S = \emptyset$ $(u, v) = (A, B)$

Prim算法



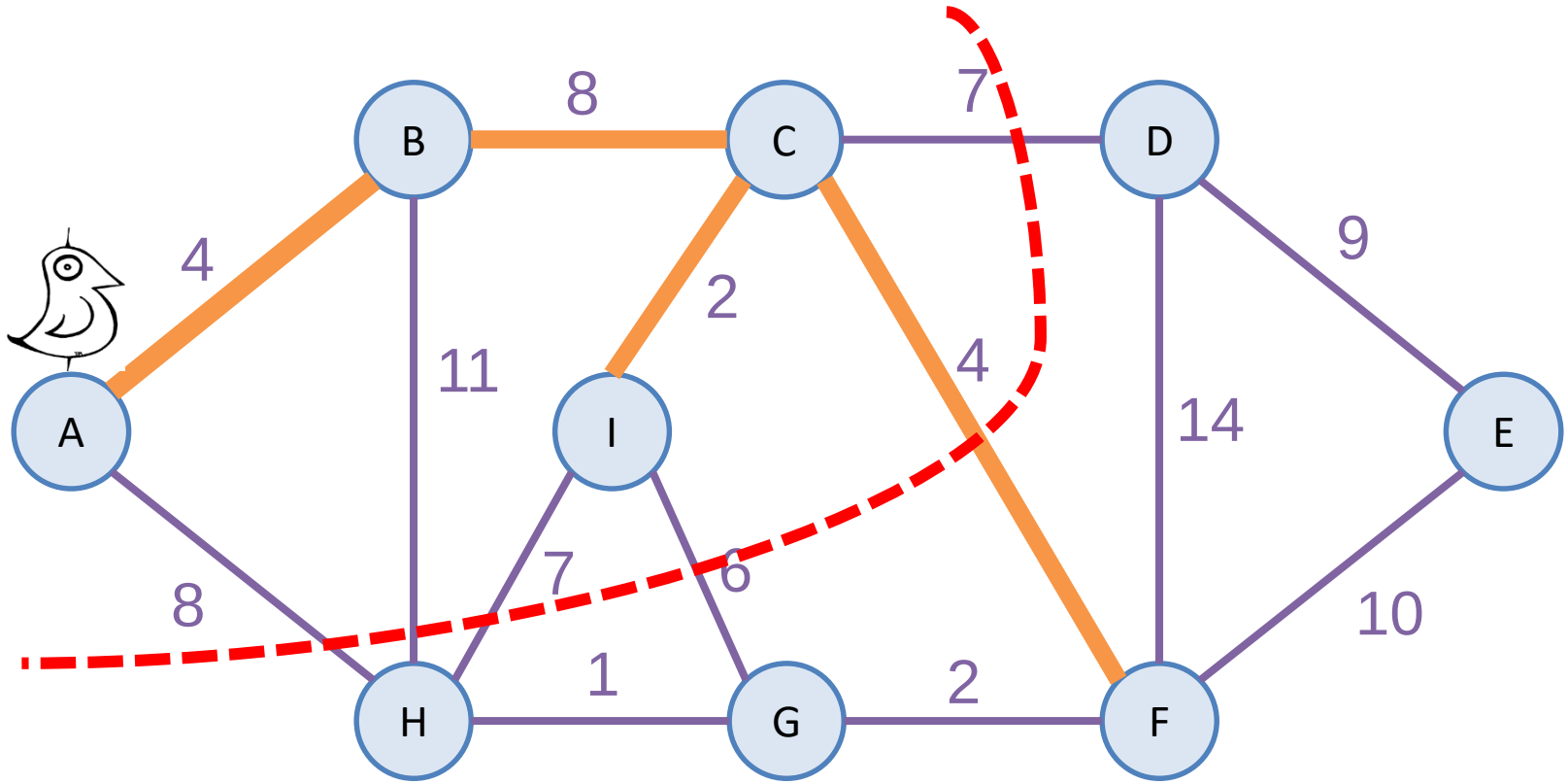
$S = \{(A, B)\}$ $(u, v) = (B, C)$

Prim算法



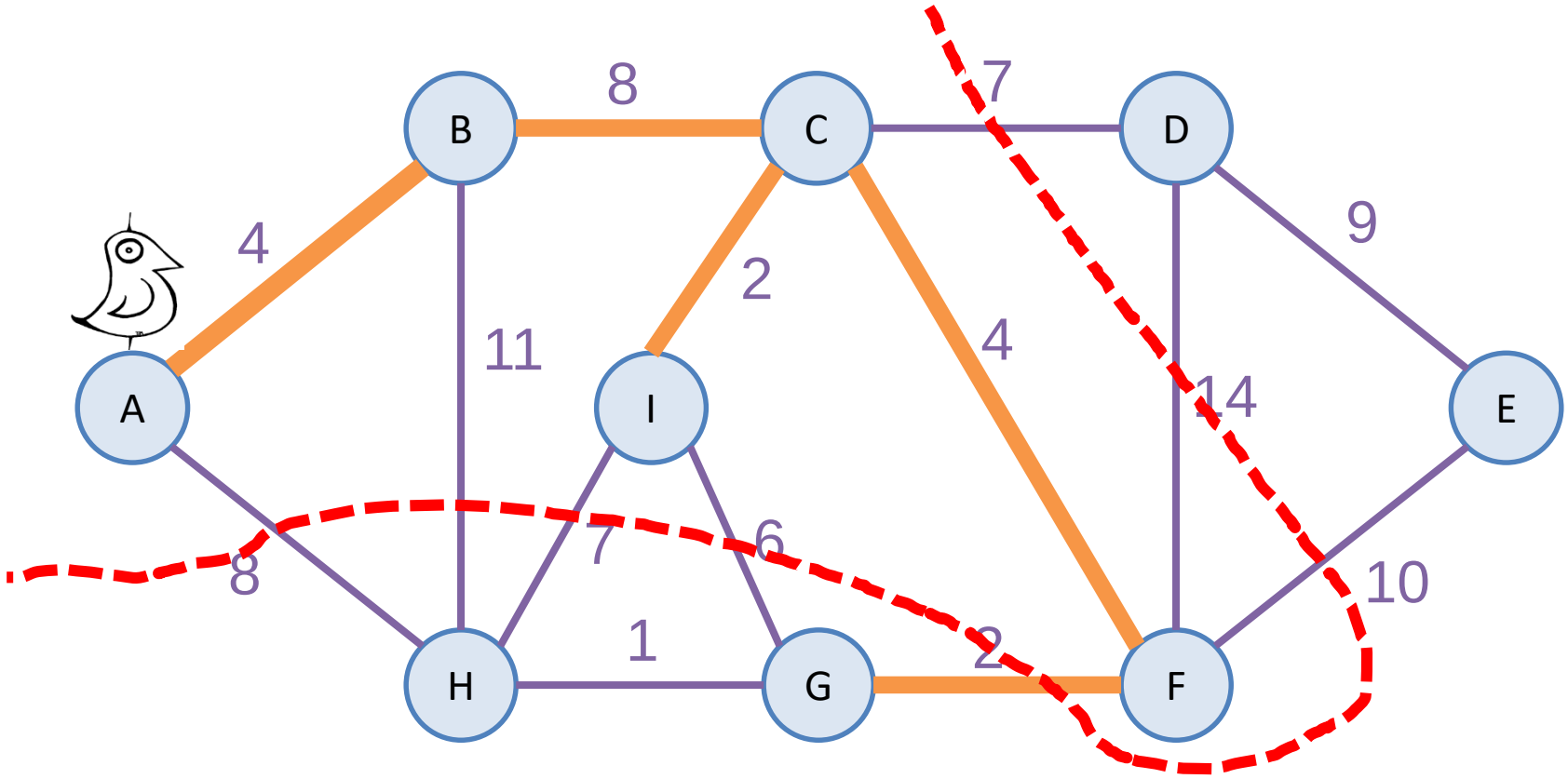
$S = \{(A, B), (B, C)\}$ $(u, v) = (C, I)$

Prim算法



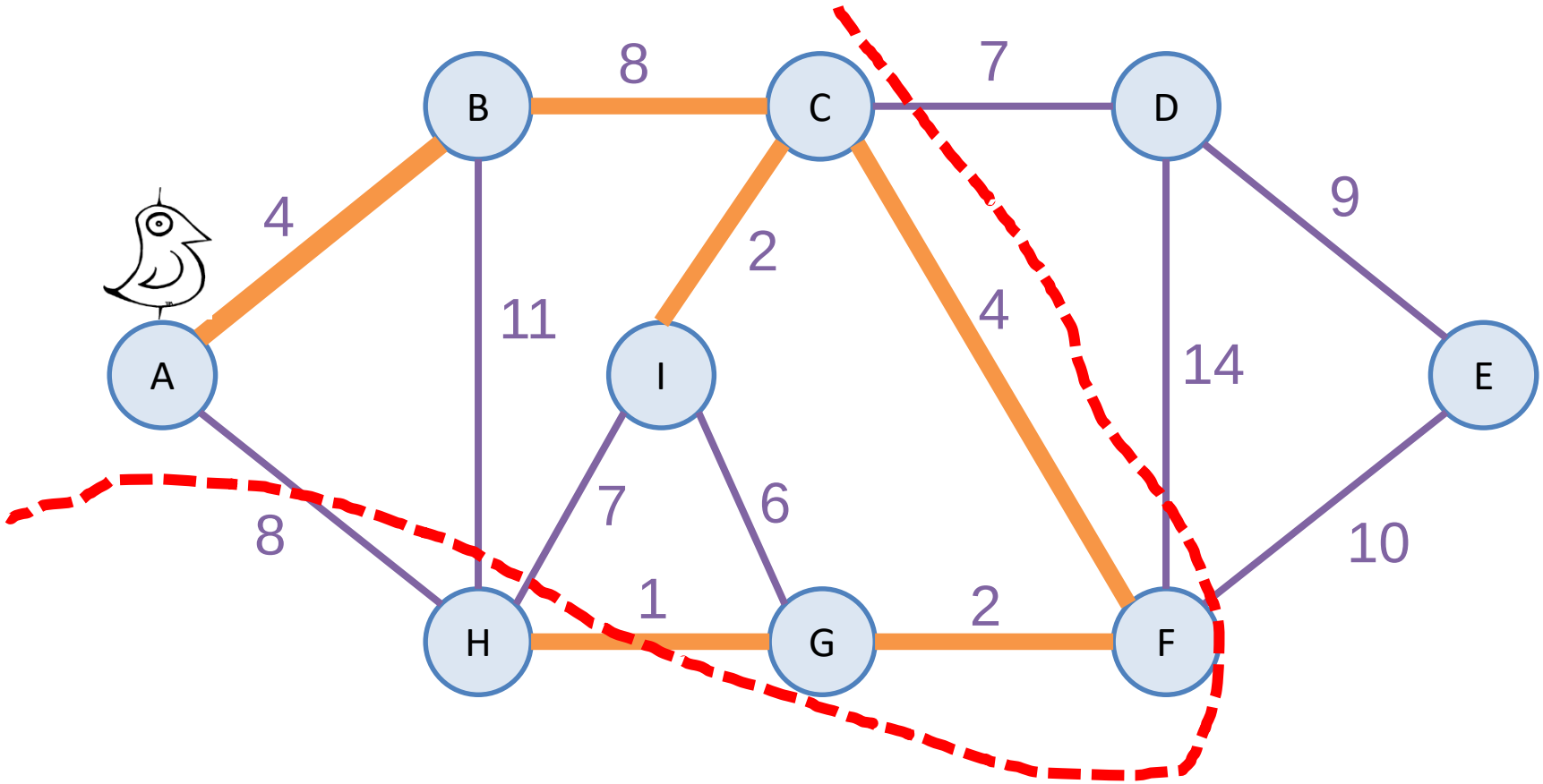
$S = \{(A, B), (B, C), (C, I)\}$ $(u, v) = (C, F)$

Prim算法



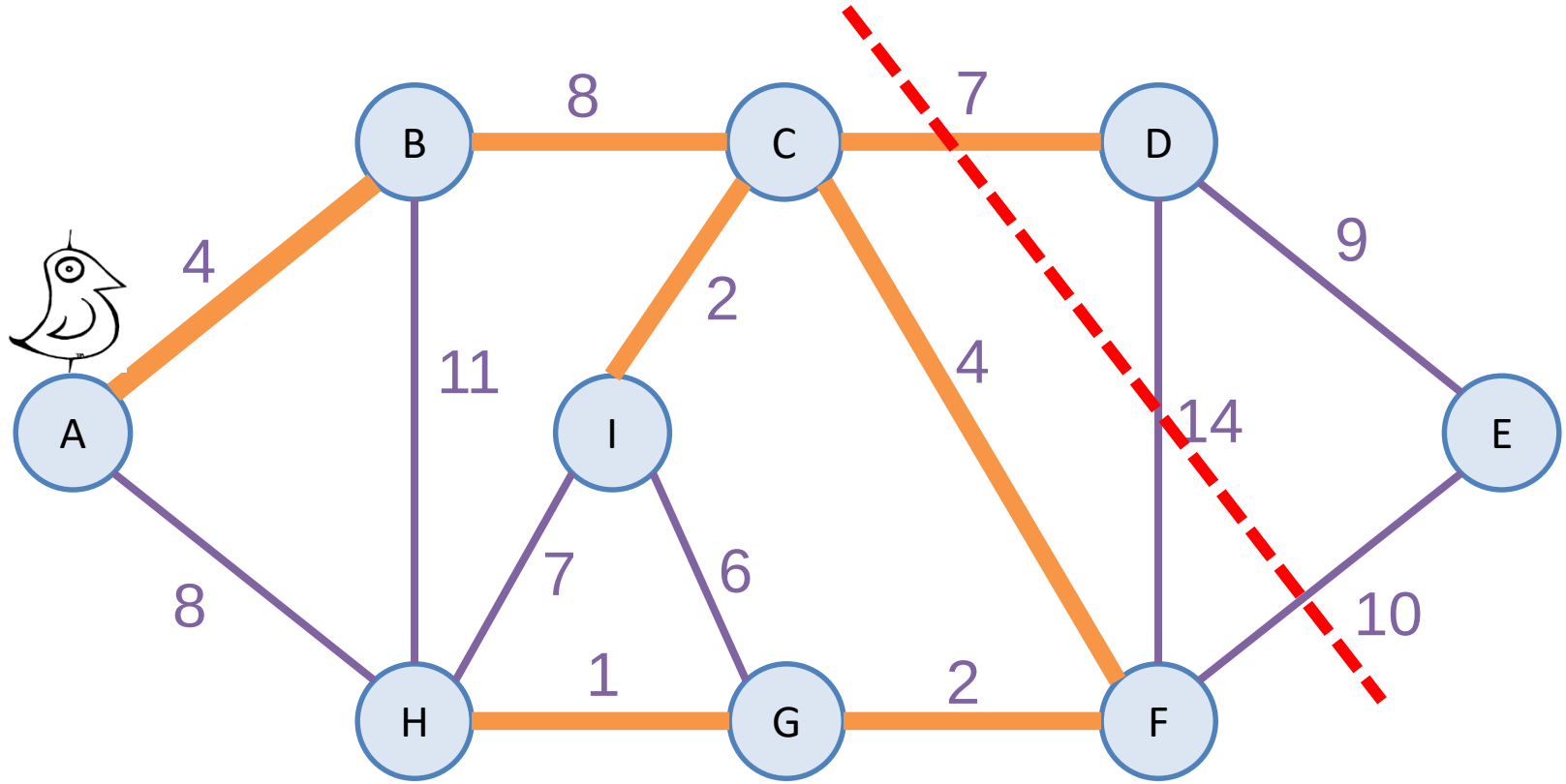
$S = \{(A,B),(B,C),(C,I),(C,F)\}$ $(u,v) = (G,F)$

Prim算法



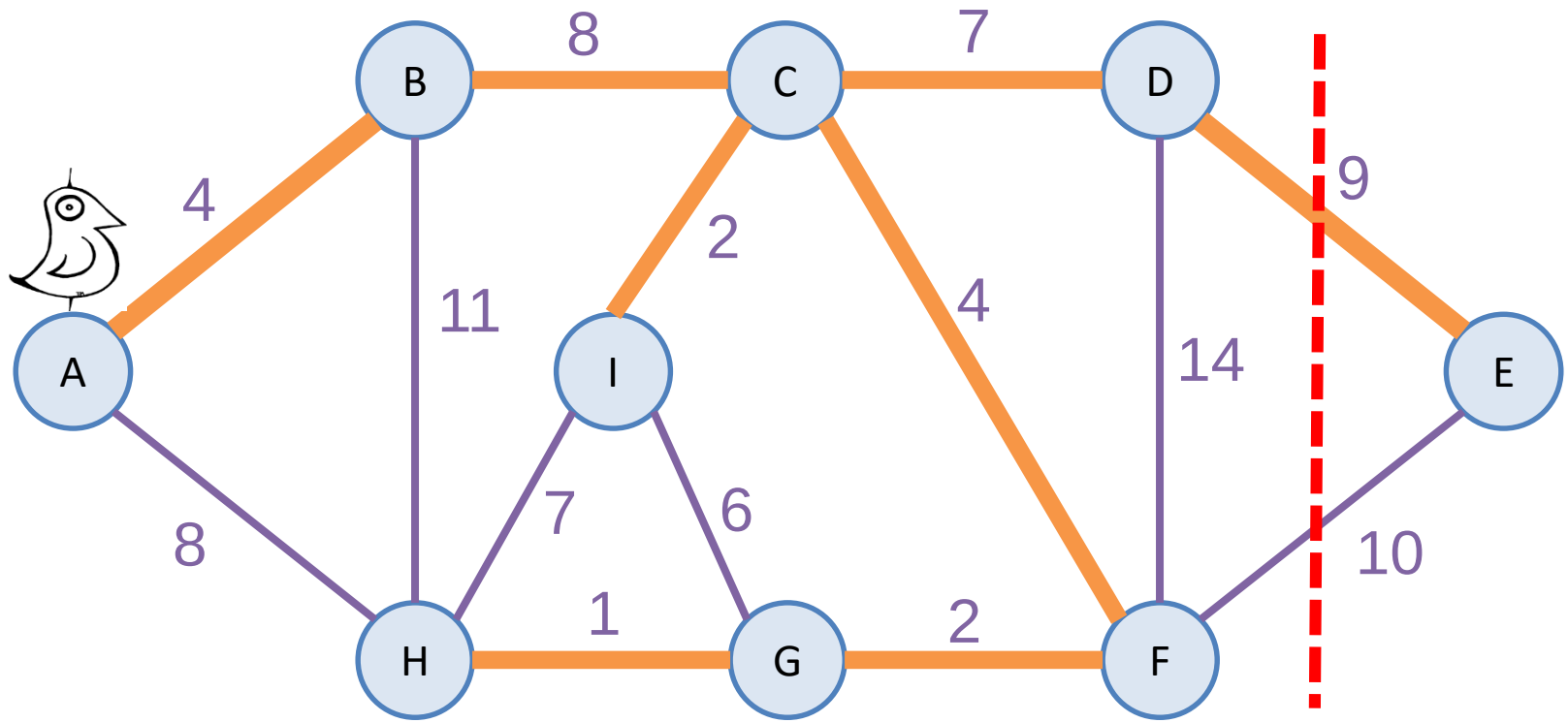
$S = \{(A,B),(B,C),(C,I),(C,F),(G,F)\}$ $(u,v) = (H,G)$

Prim算法



$S = \{(A,B),(B,C),(C,I),(C,F),(G,F),(H,G)\}$ $(u,v) = (C,D)$

Prim算法



逐渐长大直到覆盖整个连通图的所有顶点。

$S = \{(A,B),(B,C),(C,I),(C,F),(G,F),(H,G),(C,D)\}$ $(u,v) = (D,E)$

Prim算法

```
MST-Prim(G, W, r)
Input: 无向连通图G = (V, E)，权值函数W，初始结点r
Output: G的最小生成树
1.   FOR ∀v ∈ V DO
2.       key[v] ← ∞; /*保存结点v和树中结点的所有边中最小边的权重，即
     轻量级边的权重*/
3.       π(v) ← null; /*结点v在树中的父结点*/
4.   key[r] ← 0;        /*保证结点r第一个被处理*/
5.   Q ← V[G];          /*最小优先队列*/
6.   WHILE Q ≠ ∅ DO
7.       u ← Extract-Min(Q);/*找到Q中横跨割（V-Q, Q）的轻量级边的端点*/
8.   FOR ∀v ∈ Adj[u] DO  /*更新与u邻接的，但不在树中结点v的key及
                                                其父结点*/
9.           IF v ∈ Q and w(u, v) < key[v] Then
10.              π(v) ← u;
11.              key[v] ← w(u, v); /*更新信息*/
12. Return A={(v, π(v))| v ∈ V - r}.
```

《算法导论》90页有关于优先队列的解释

Prim算法是否正确？

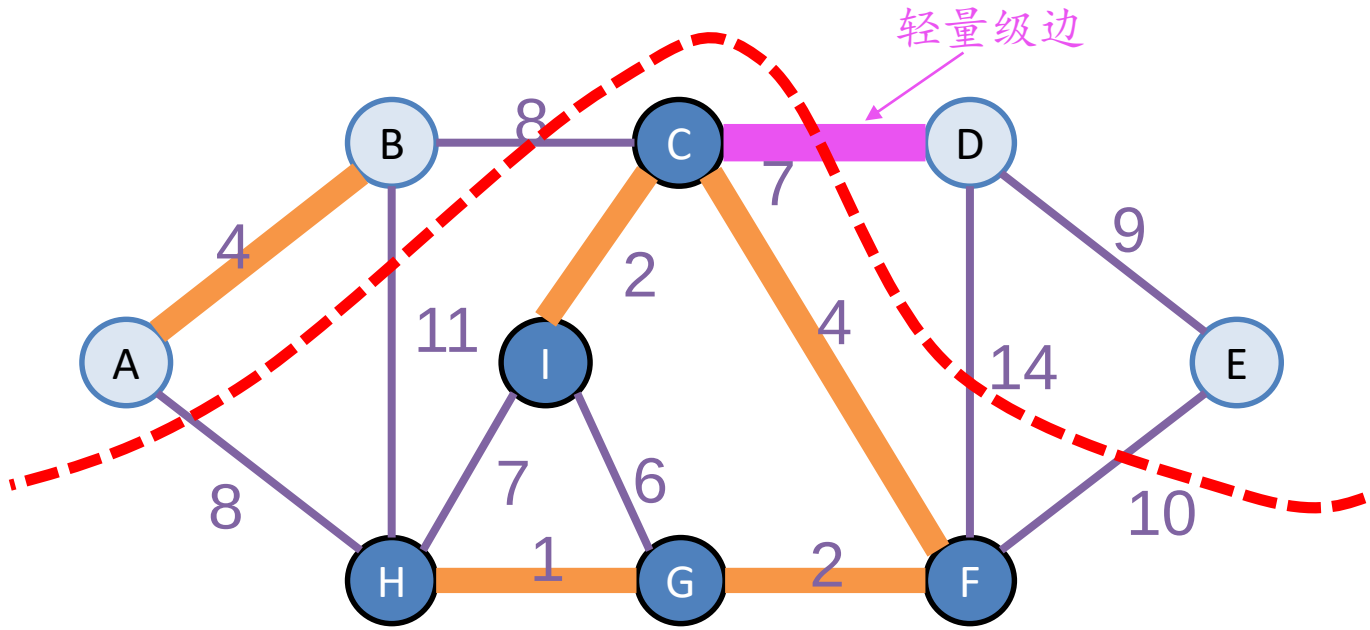
- 贪心选择性：
  - 每一步得到的边的集合，都是最小生成树的子集

引理1

引理1：令S是图$G=(V,E)$的边集合E的子集，并且存在一个割尊重S，

- 如果存在G的一个最小生成树包含S
- 令(u,v)是该割的轻量级边

则**存在**一个G的最小生成树包含$S \cup \{(u,v)\}$。



S是橙色的边

归纳法



- 基本情况：
  - 增加第0条边时，边集为空，显然是最小生成树的一部分
- 归纳步骤：
  - 如果增加第t条边之后，得到的边集是最小生成树的一部分，那么再增加一条边，依然是最小生成树的一部分。（前面已证）
- 结论：
  - 每一步得到的边集都是最小生成树的一部分，当所有点都包含之后，得到的边集就是最小生成树

Kruskal算法

- **贪心思想**
  - 也称“加边法”
  - 初始时，最小生成树集合里，包含所有顶点，但不包含任何边，每迭代一次就选择一条满足条件的边加入到集合里。
- **考虑问题的出发点：**为使生成树上边的权值之和达到最小，则应使生成树中每一条边的权值尽可能地小。

Kruskal算法

克鲁斯卡尔算法的基本思想：

设G=(V,E)是连通网，用T来记录G上最小生成树边的集合。

(1)从G中取最短边e，如果边e所关联的两个顶点不在T的同一个连通分量中，则将该边加入T；

(2)从G中删除边e；

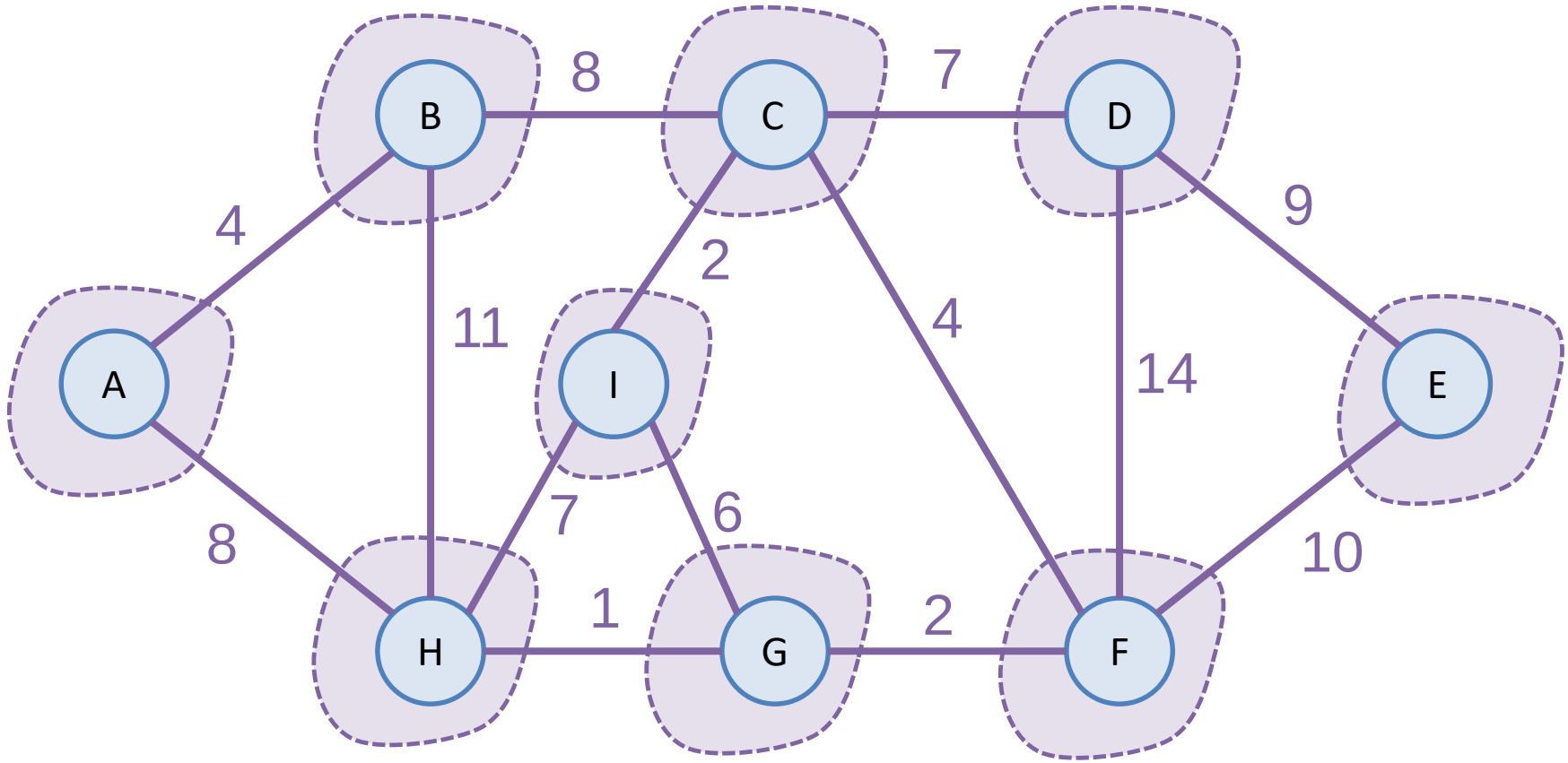
(3)重复(1)和(2)两个步骤，直到T中有n-1条边。

Kruskal算法

Kruskal算法是从森林到树：

每一步都通过合并规模小的树，减小森林的规模，得到更大的树

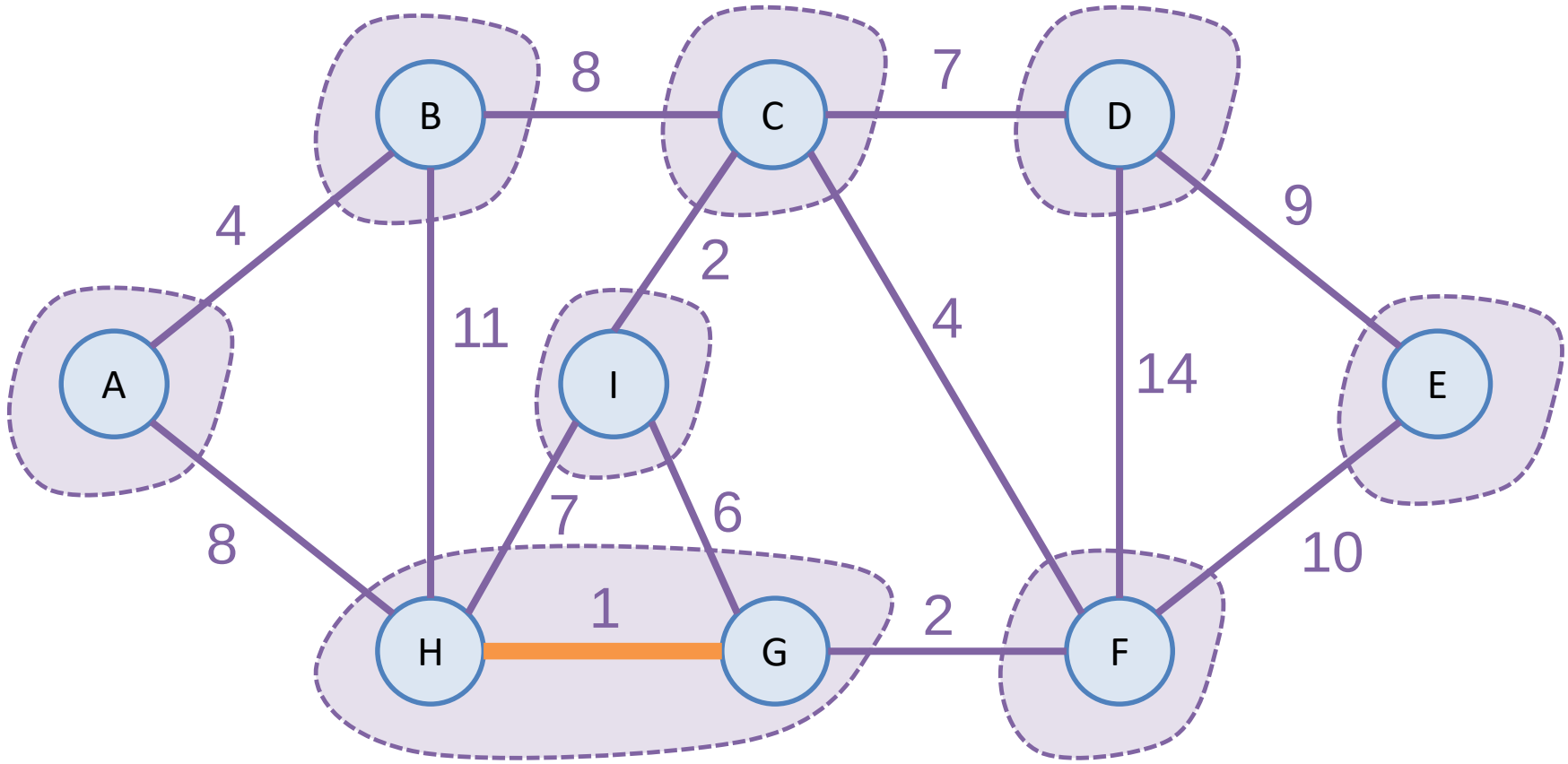
开始时，每个顶点都在自己的树中



Kruskal算法

Kruskal算法是从森林到树：

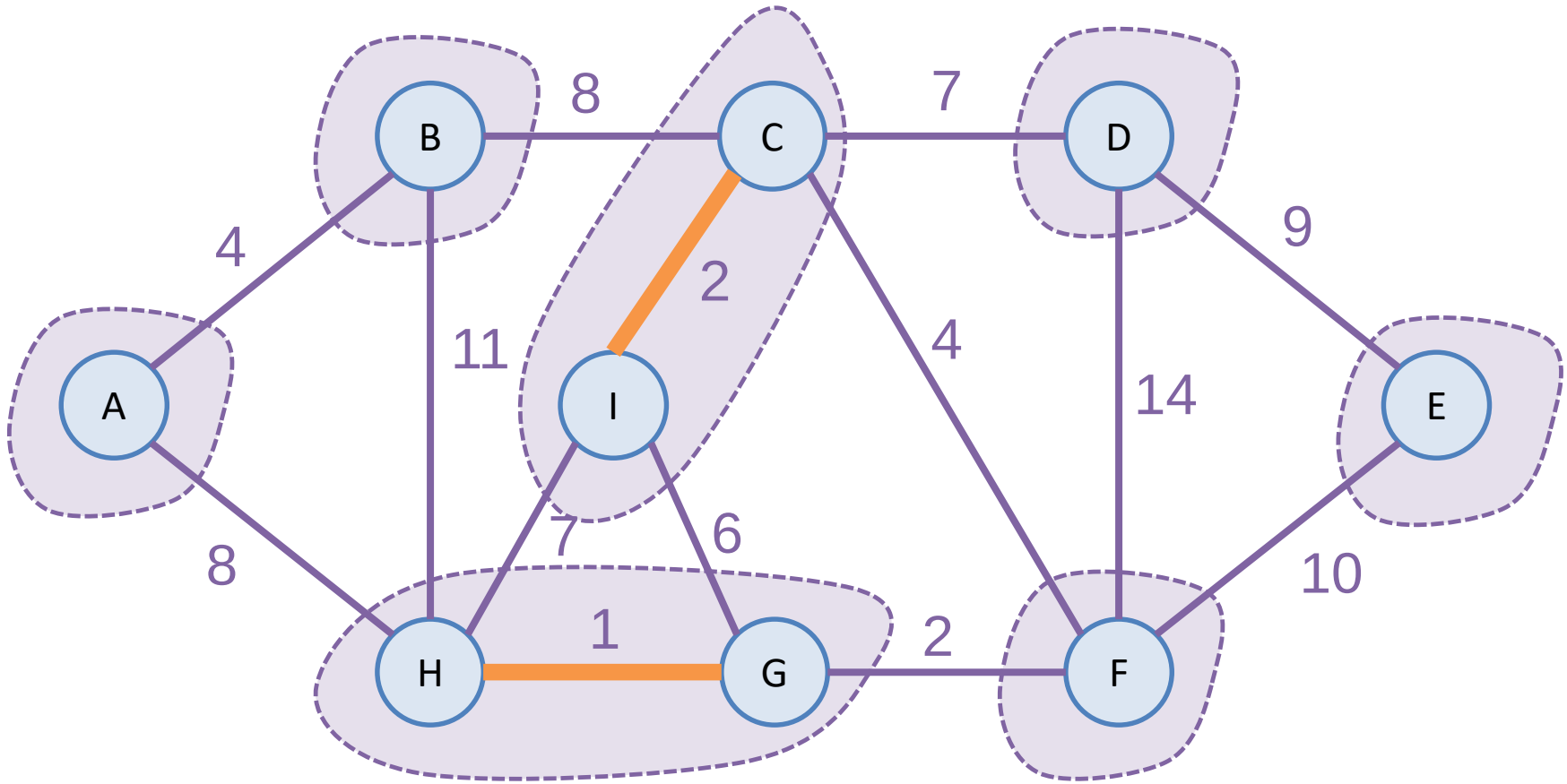
每一步都通过合并规模小的树，减小森林的规模，得到更大的树



Kruskal算法

Kruskal算法是从森林到树：

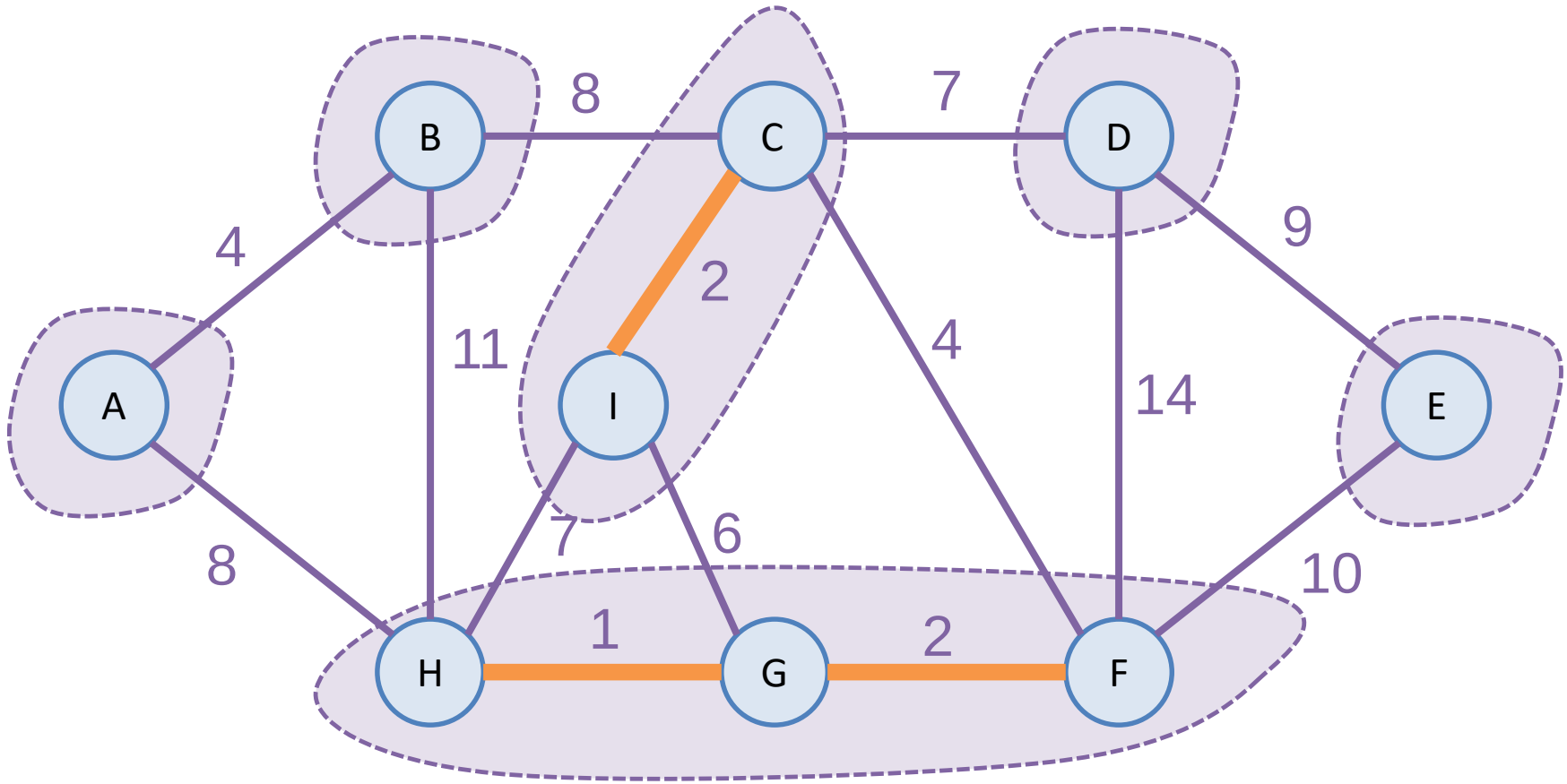
每一步都通过合并规模小的树，减小森林的规模，得到更大的树



Kruskal算法

Kruskal算法是从森林到树：

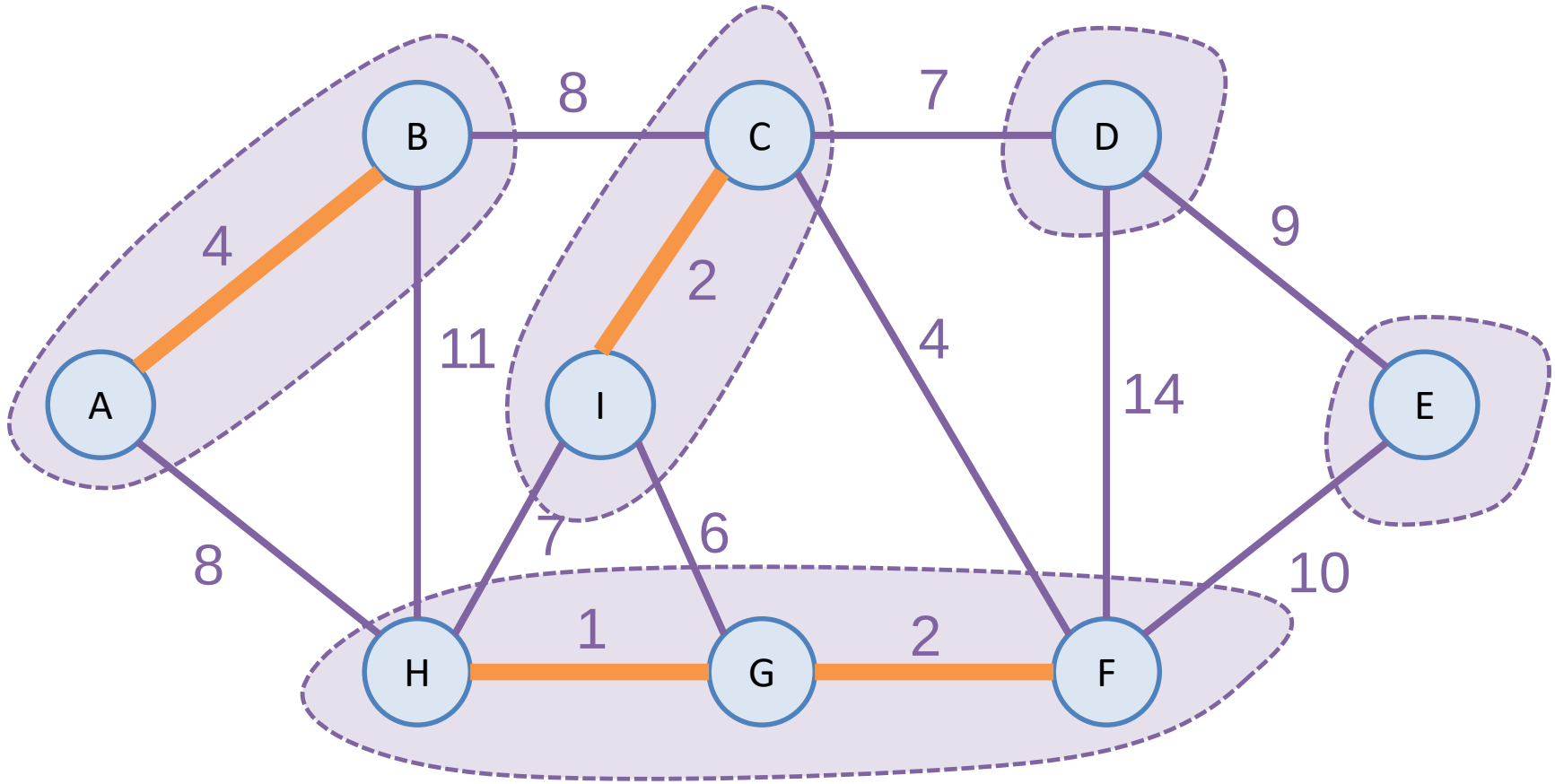
每一步都通过合并规模小的树，减小森林的规模，得到更大的树



Kruskal算法

Kruskal算法是从森林到树：

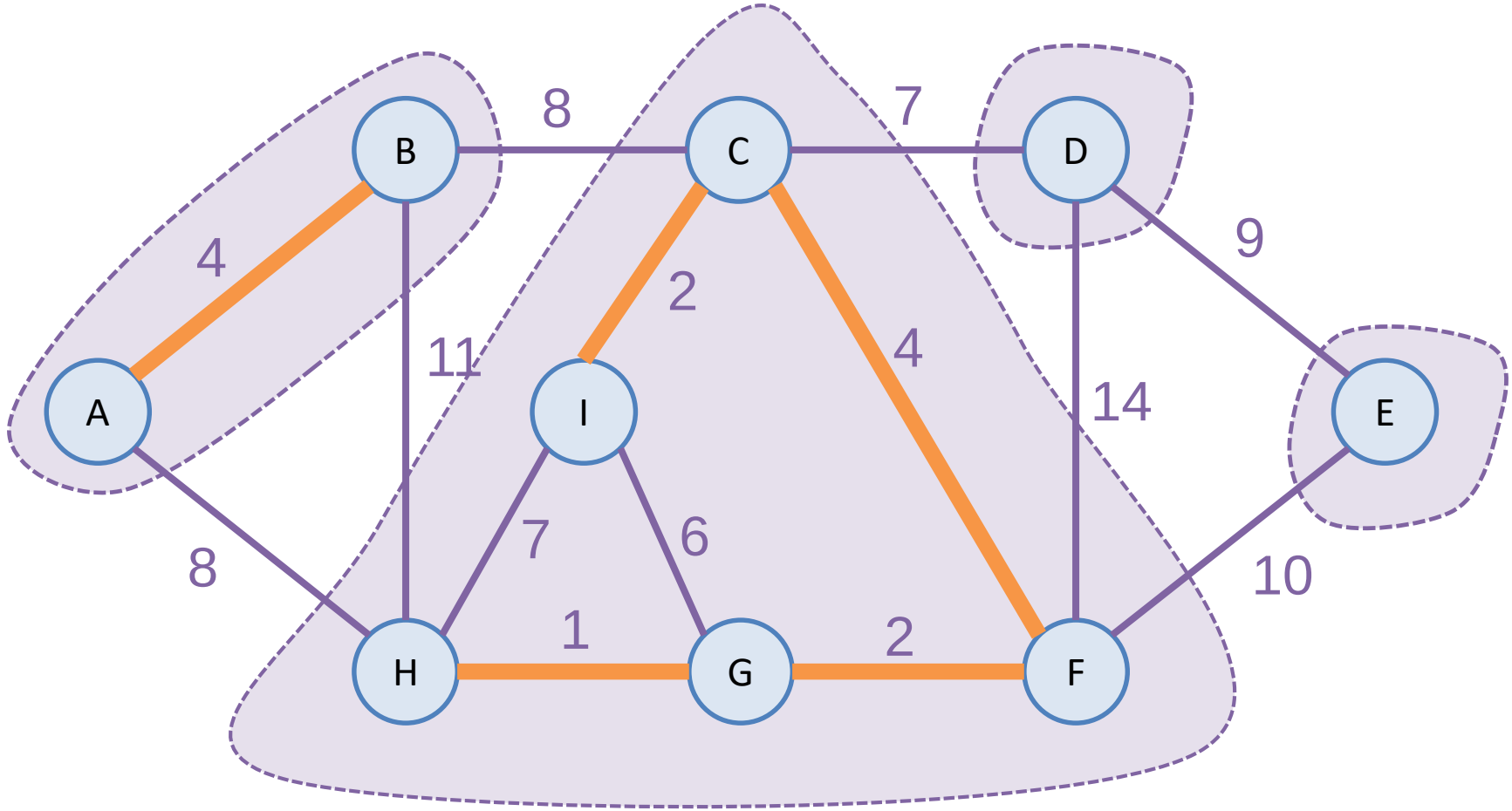
每一步都通过合并规模小的树，减小森林的规模，得到更大的树



Kruskal算法

Kruskal算法是从森林到树：

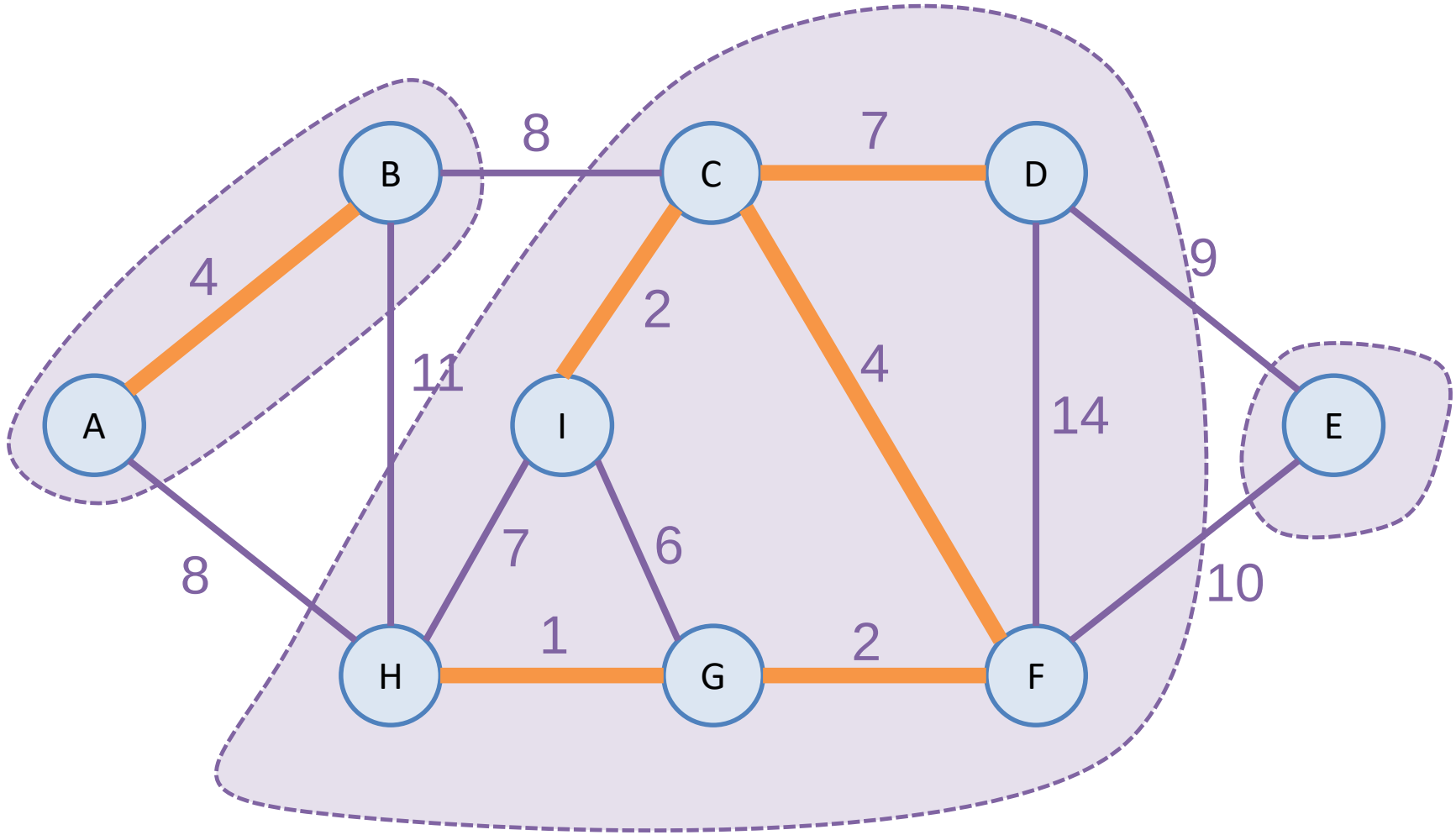
每一步都通过合并规模小的树，减小森林的规模，得到更大的树



Kruskal算法

Kruskal算法是从森林到树：

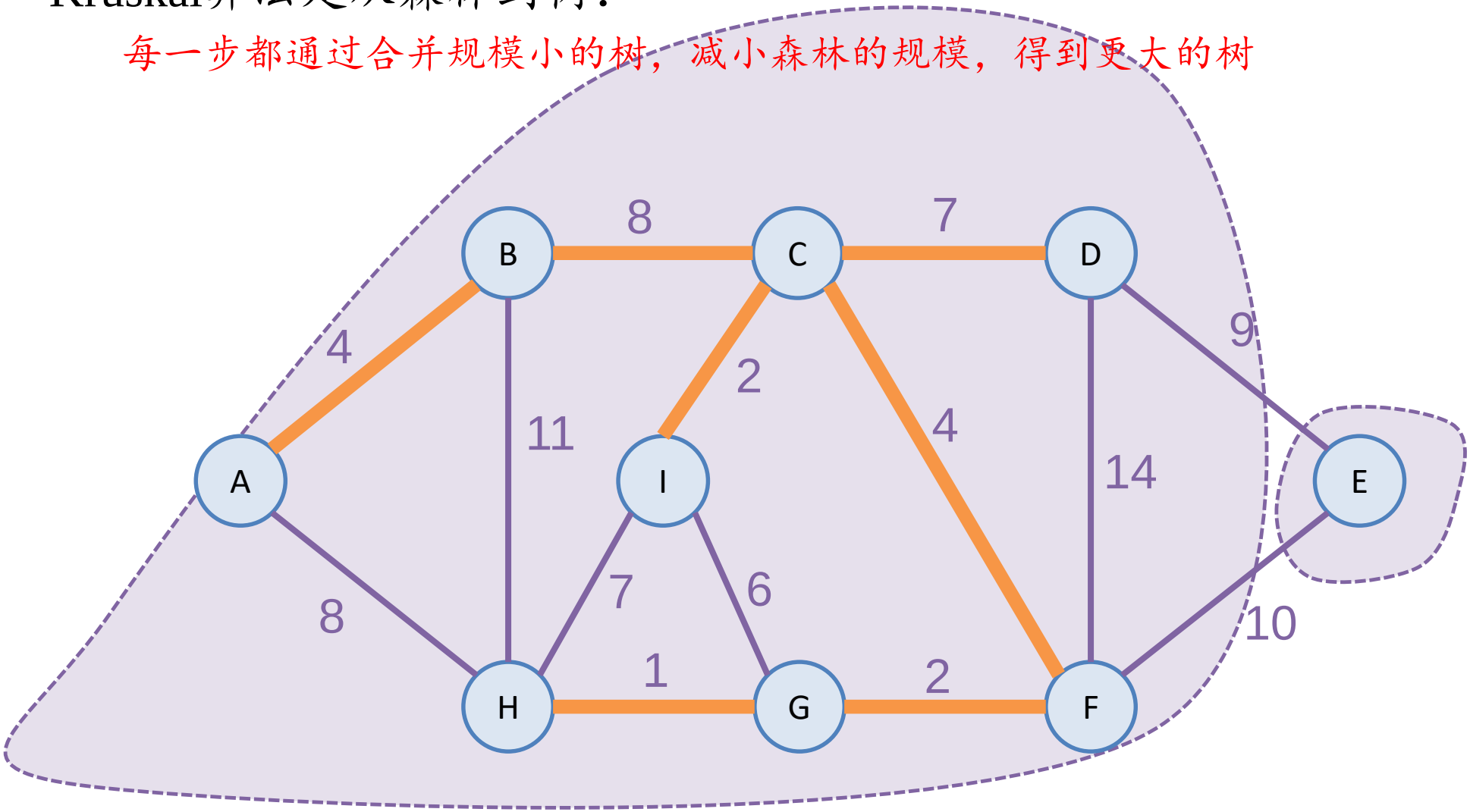
每一步都通过合并规模小的树，减小森林的规模，得到更大的树



Kruskal算法

Kruskal算法是从森林到树：

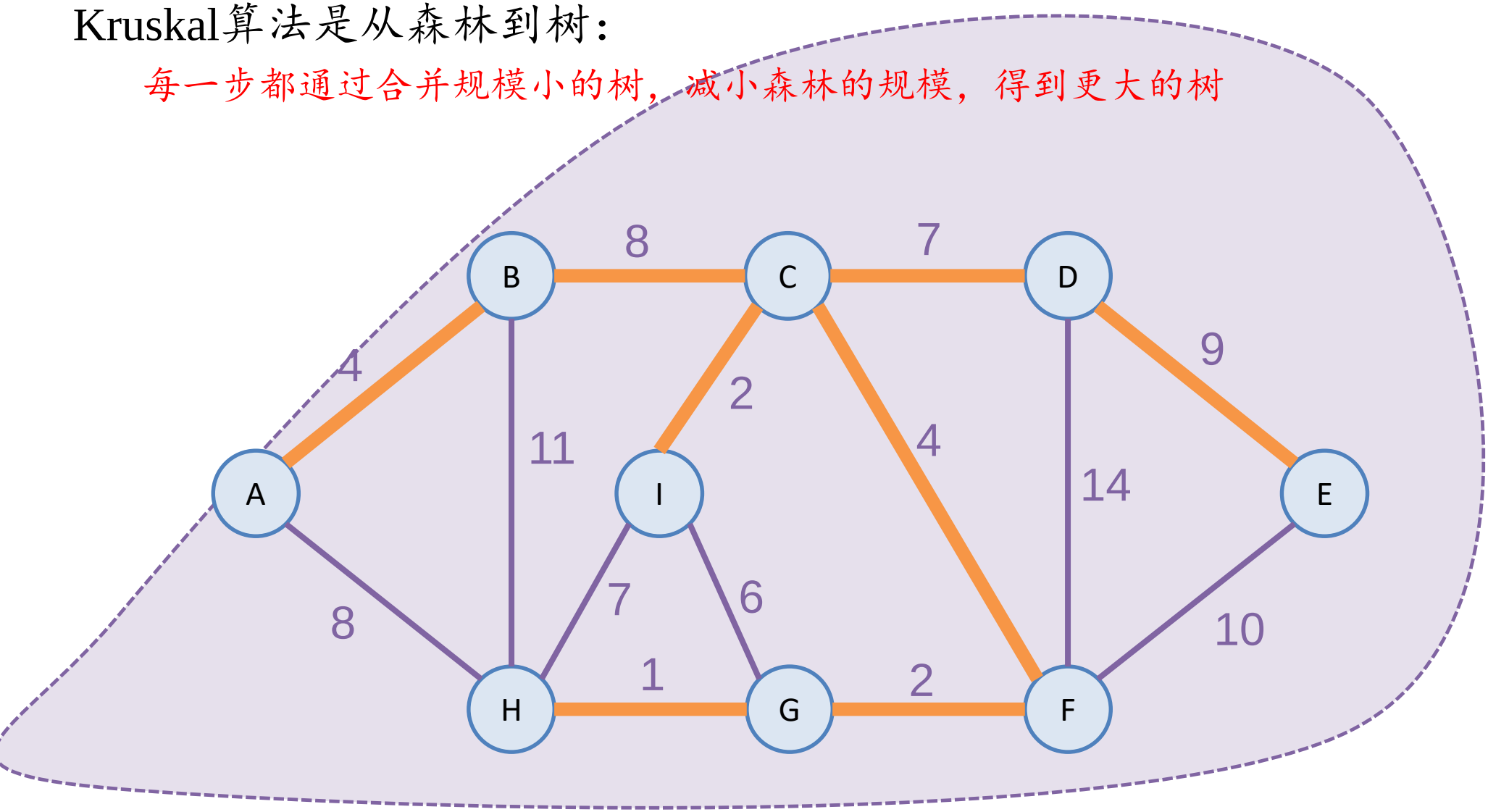
每一步都通过合并规模小的树，减小森林的规模，得到更大的树



Kruskal算法

Kruskal算法是从森林到树：

每一步都通过合并规模小的树，减小森林的规模，得到更大的树



当只有一棵树的时候，停止！

Kruskal算法

```
MST-Kruskal(G, W)
Input: 无向连通图G = (V,E), 权值函数W
Output: G的最小生成树
1. S = ∅; /*初始化为空树*/
2. FOR ∀v ∈ V DO
3.     Make-Set(V);      /*创建|V|棵树*/
4. 按照W权值的非递减顺序排序E;
5. FOR ∀(u,v) ∈ E (按照W权值的非递减顺序) DO
6.     IF Find-Set(u) ≠ Find-Set(v)
                /*检查节点u,v是否属于同一棵树 */
7.         S = SU{(u,v)}; /*边(u,v)加入到集合S*/
8.         Union(u,v);/*对节点u,v所属的子树进行合并*/
9.  Return S.
```

Kruskal算法复杂性分析

- 令$n = |V|, m = |E|$
- 第4步需要时间：$O(m\lg m)$
- 第2-3步执行$O(n)$个Make-Set操作
- 第5-8步执行$O(m)$个Find-Set和Union操作

  需要时间：$O((n+m)\alpha(n))$
- $m \geq n-1$（因为G连通），$\alpha(n) = \lg n = \lg m$
- 总时间复杂性：$O(m\lg m)$

Kruskal 算法是否正确？

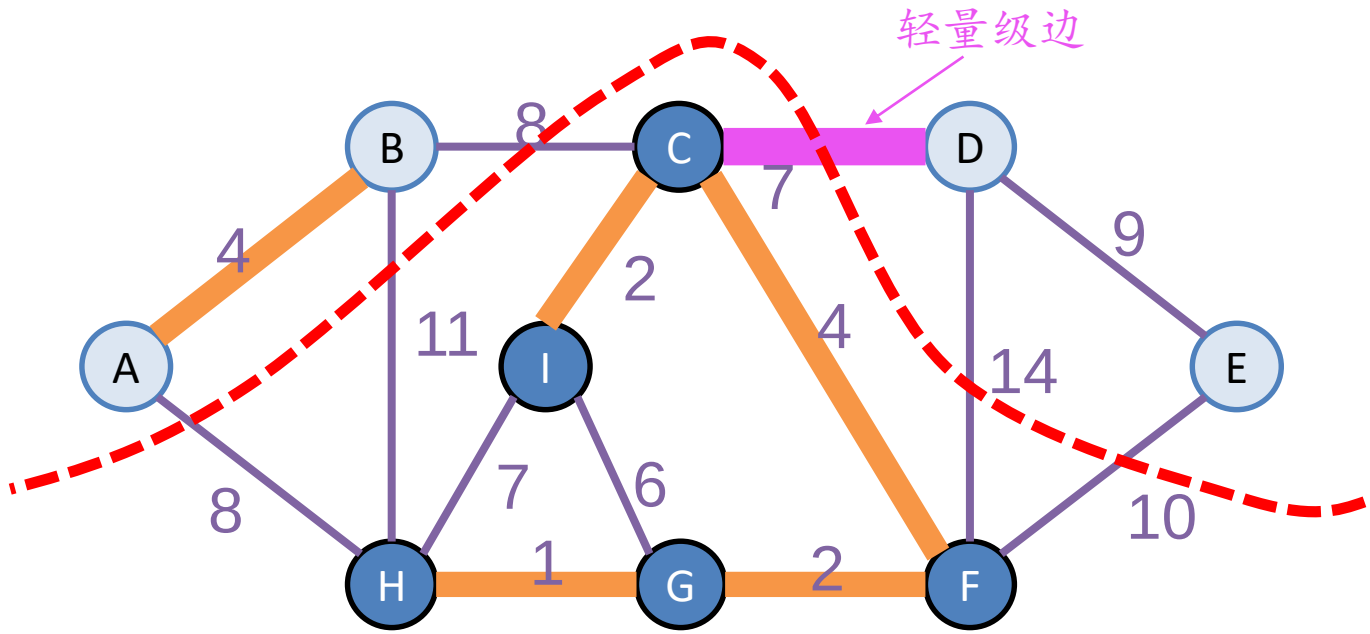
- 贪心选择性：
  - 每一步得到的边的集合，都是最小生成树的子集

引理1

引理1：令S是图$G=(V,E)$的边集合E的子集，并且存在一个割尊重S，

- 如果存在G的一个最小生成树包含S
- 令(u,v)是该割的轻量级边

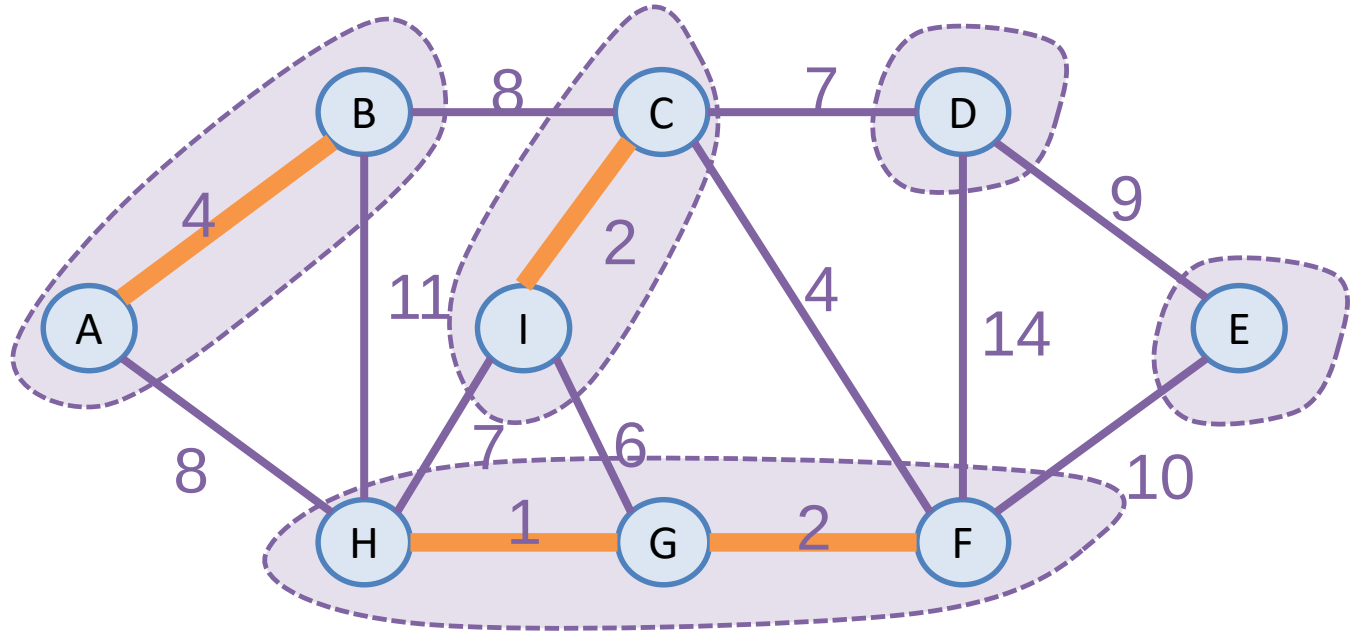
则**存在**一个G的最小生成树包含$S \cup \{(u,v)\}$。



S是橙色的边

Kruskal算法的贪心选择性

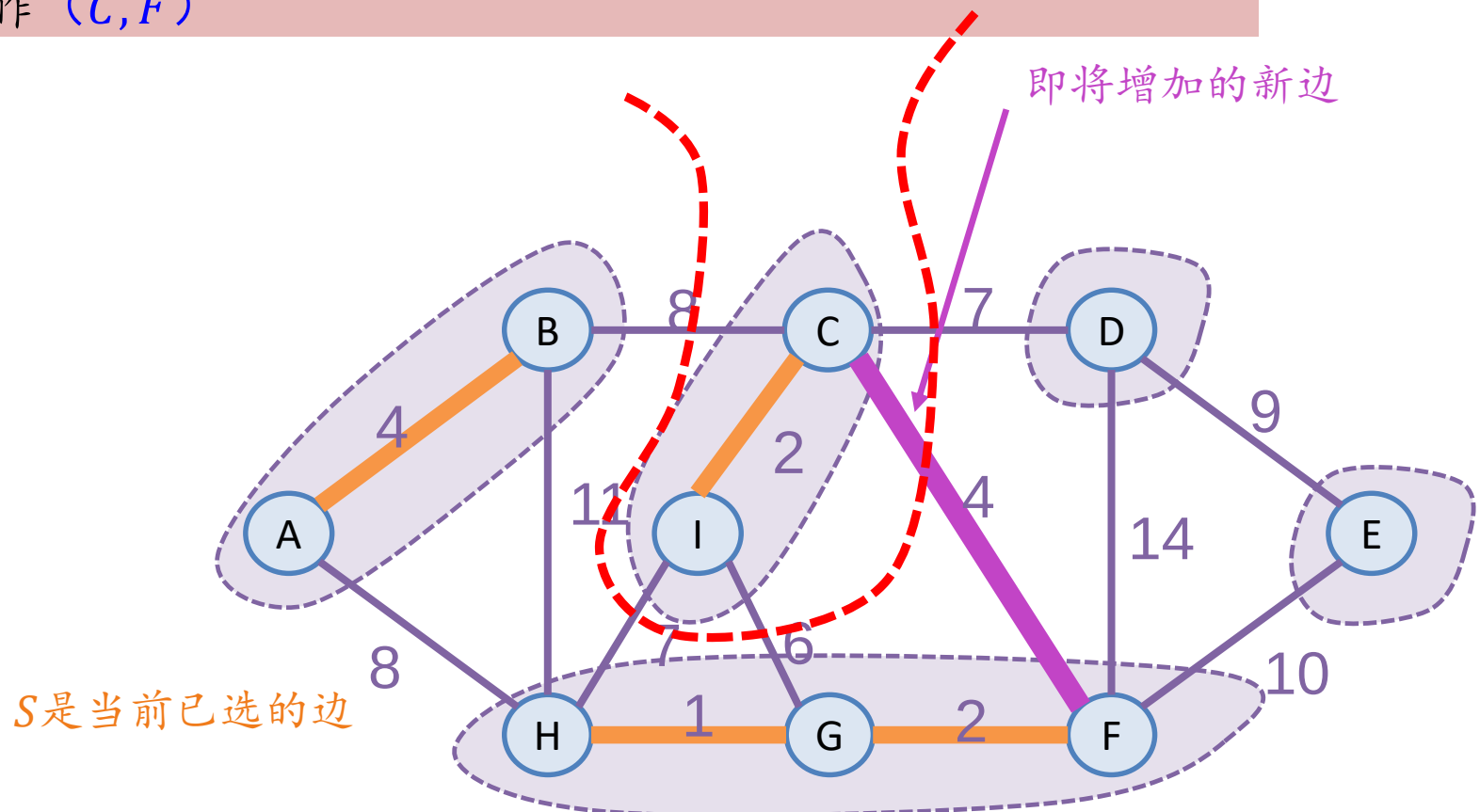
- 假设当前 S是最小生成树的一部分
- 接下来增加的边将会合并两个树不妨记作， $T1, T2$



S是当前选定的边组成的集合

Kruskal算法的贪心选择性

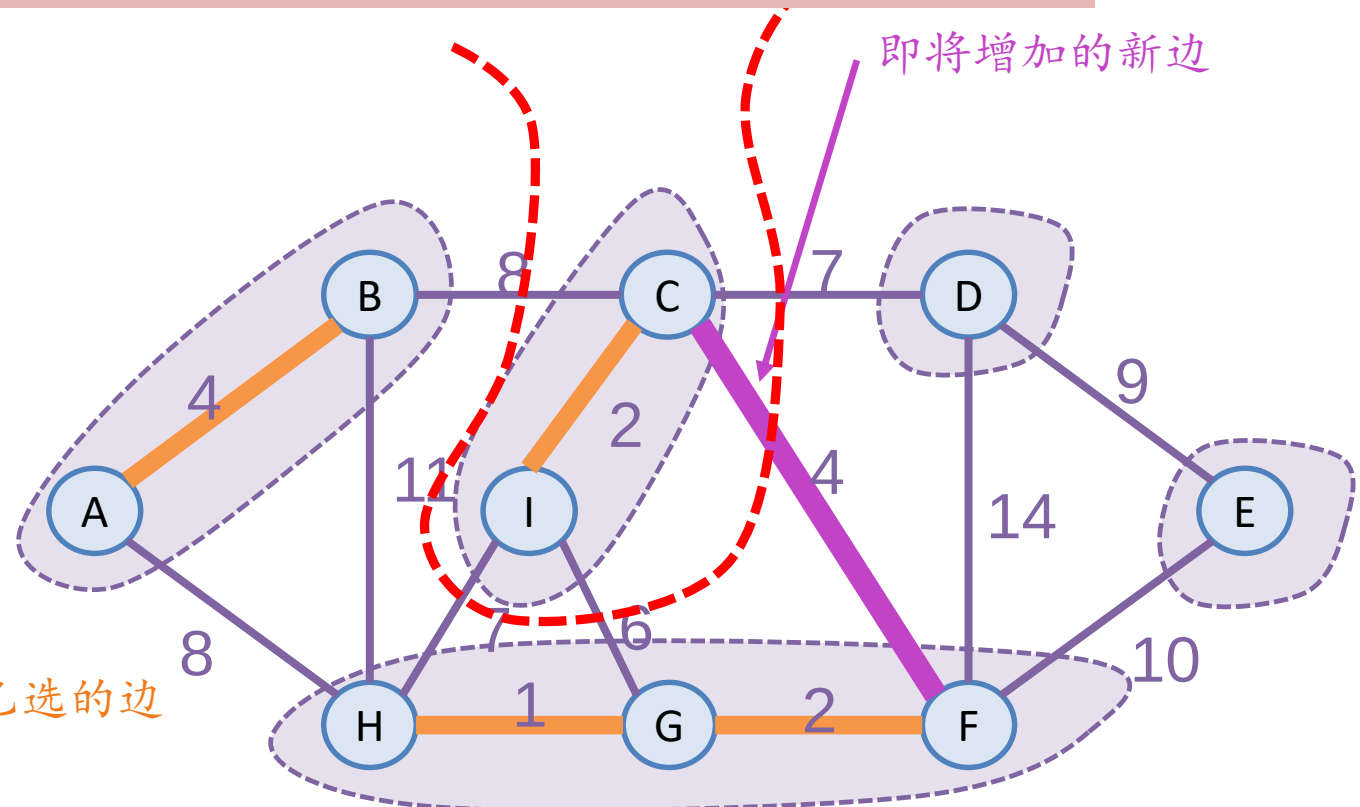
- 假设当前S是最小生成树的一部分
- 接下来增加的边将会合并两个树不妨记作，$T1, T2$
- 则存在图的割$\{T1, V - T1\}$，并且我们新增加的边是该割的轻量级边不妨记作(C, F)



Kruskal算法的贪心选择性

- 假设当前S是最小生成树的一部分
- 接下来增加的边将会合并两个树不妨记作，$T1, T2$
- 则存在图的割$\{T1, V - T1\}$，并且我们新增加的边是该割的轻量级边不妨记作(C, F)

- 根据引理1，则存在一个G的最小生成树包含$S \cup \{(C, F)\}$



归纳法

与prim的证明完全一样



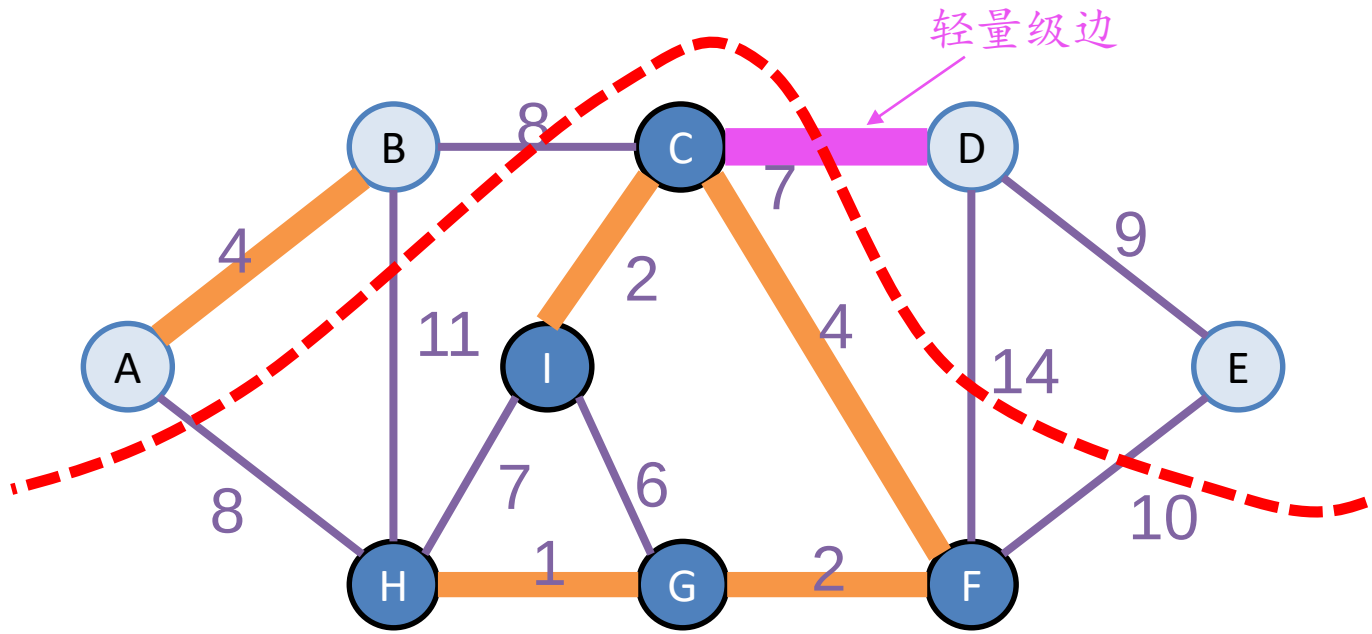
- 基本情况：
  - 增加第0条边时，边集为空，显然是最小生成树的一部分
- 归纳步骤：
  - 如果增加第t条边之后，得到的边集是最小生成树的一部分，那么再增加一条边，依然是最小生成树的一部分。(前面已证)
- 结论：
  - 每一步得到的边集都是最小生成树的一部分，当所有点都包含之后，得到的边集就是最小生成树

Prim vs Kruskal
（不同点）

- Prim算法:
  - 逐步增加单一树的规模得到最小生成树
- Kruskal算法:
  - 从森林中合并小树，得到大树最终得到最小生成树

Prim vs Kruskal
（相同点）

- 都是通过贪心思想得到最小生成树
- 优化子结构都是通过图的割得到
- 都是通过选择割的轻量级边扩展边集



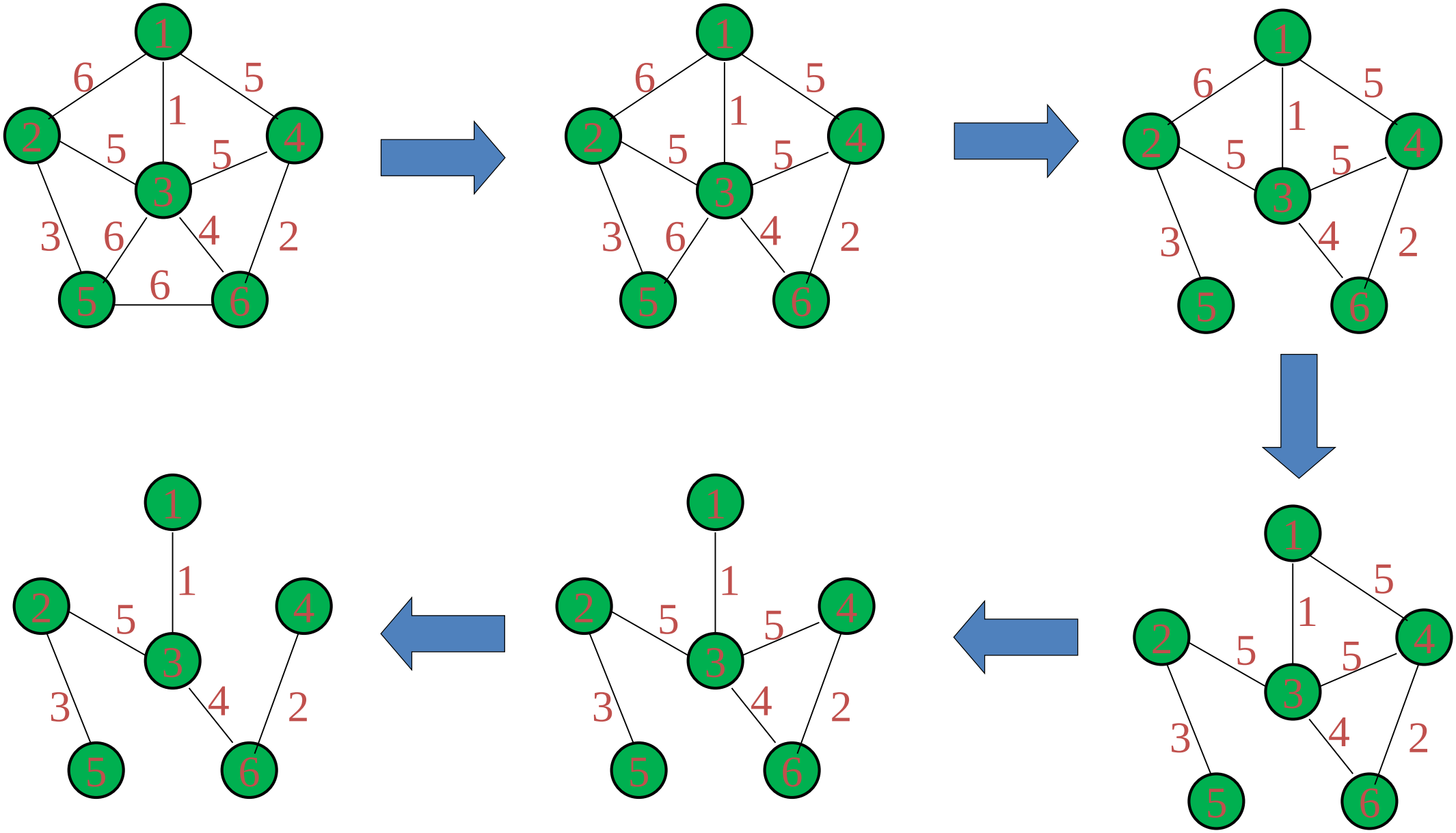
破圈法（管梅谷算法）

- Prim算法和Kruskal算法属于**避圈法**
- 1975年，我国管梅谷教授提出了破圈法，“见圈破圈”

算法思想

- 先从图G任取一个圈，并从圈中去掉一条权最大的边。若在同一圈中有几条都是权最大边，则任选其中一边去掉。
- 在余下的子圈中，重复上述步骤，直至没有圈但仍然连通止。

破圈法（管梅谷算法）


1
6
5
2
1
4
5
5
3
3
6
4
2
5
6
1
6
5
2
1
4
5
5
3
3
6
4
2
5
6
1
6
5
2
1
4
5
5
3
3
6
4
2
5
6
1
5
2
1
4
5
3
3
4
2
5
6
1
5
2
1
4
5
3
3
4
2
5
6
1
5
2
1
4
5
3
3
4
2
5
6

破圈法（管梅谷算法）

